卒中后癫痫五套算法架构 · 代码汇编

从原始数据到算法架构全部数字与图表的可执行代码

v1.0 丨 2026-11-07 丨 三合医研发团队，河南大学

收录 10 个分析脚本 + 13 个辅助/诊断/装配/环境脚本，共 5,700 余行

所有代码均为实际运行版本，未做删改；中文注释保留原样

0. 这份 PDF 是什么

这份汇编把五套算法架构从原始数据到最终数字、表格与图表的代码按运行顺序完整收录，目的是让审稿人、合作者或下一位接手的人能对照算法架构逐段找到“这个数字是哪个脚本、哪一行算出来的”。每一节先说明脚本做什么、吃什么文件、吐什么结果，再贴代码全文。

0.1 怎么读

- **提示：**第一次读：先看第 1 节的“代码总览”，知道一共有哪些脚本、各自对应哪套算法架构。
- **提示：**要复现：按 2.1 → 2.8 的顺序跑 A/B/C；按 3.1 → 3.2 跑 D/E。
- **提示：**要核对某个数字：先在算法架构里找到表/图编号，再到第 1 节的对照表找到脚本，最后在脚本里搜指标名。
- **提示：**代码中的 $PY、$OLD_PY、$BASE、$DATA 是环境变量占位符，具体路径见《配置要求与复现说明》。

0.2 运行环境（一句话版）

A/B/C 的 LightGBM 与 TabPFN 在 scikit-learn 1.9.1 的新环境运行；LR 支路在 scikit-learn 1.7.2 的 D13 老环境复算，才能与归档锚点逐位一致。D/E 全部在新环境运行。两套环境的完整版本表、三条 SHA-256 与逐条命令见《配置要求与复现说明》第 2–6 节。

0.3 收录边界（重要）

本书收录服务器与本机上与该项目相关的全部 23 个脚本：10 个分析脚本 + 13 个辅助/诊断/装配/环境脚本。其中 A.2 跨队列迁移实验的结果接近随机，按“不合格的不写进去”的原则未进入任何一册正文；A.3、A.4 是一次性诊断脚本，不产生算法架构数字；A.5–A.6 是 LR 老环境的构建脚本与依赖清单；A.7–A.9 是前身项目 D13 的主脚本、审计与提交脚本（只作溯源，其数字不进入正文）。收进来是为了让读者看到全部跑过的代码，而不是只看到成功的那一部分——每个条目都注明了自己的用途与边界。

1. 代码总览

表 1. 脚本、行数与算法架构对应关系

编号	脚本	行数	对应算法架构/用途
2.1	tabpfn_bench.py	186	A / B / C
2.2	lr_rerun.py	134	B / C
2.3	verify_lr_172.py	71	A / B / C
2.4	bench_analysis.py	196	A / B / C
2.5	table1_baseline.py	41	A
2.6	restore_analyses.py	276	A / B / C
2.7	bootstrap_ci.py	65	A / B / C
2.8	make_figures_restore.py	161	A / B / C
3.1	wenzhou_de.py	698	D / E
3.2	table1_wenzhou.py	72	D / E
A.1	tabpfn_smoke.py	51	环境自检
D.1	build_docx5.py	870	算法架构装配
D.2	to_pdf5.ps1	16	PDF 导出
D.3	build_resultsbook.py	1183	结果汇编
D.4	build_codebook.py	587	代码汇编
A.2	external_transfer.py	153	迁移实验（未入册）
A.3	debug_lr.py	73	诊断
A.4	debug_loop.py	79	诊断
A.5	setup_venv_server.sh	24	环境构建
A.6	requirements.txt	7	环境依赖
A.7	model_d13_server.py	564	前身项目
A.8	audit_d13.py	279	前身项目
A.9	slurm_d13.sh	27	前身项目

1.1 算法架构—脚本—产出一览

算法架构	核心脚本	算法架构中的表/图
A 入院特征基准	2.1 + 2.2 + 2.4 + 2.6 + 2.7 + 2.8	表 1–4；图 2–7
B 实验室增量	2.1 + 2.2 + 2.4 + 2.6 + 2.7 + 2.8	表 1–3；图 2–7
C 并发症边界检验	2.1 + 2.2 + 2.4 + 2.6 + 2.7 + 2.8	表 1–3；图 2–7
D 首次发作时点预测	wenzhou_de.py + table1_wenzhou.py	表 1、表 2；图 2–4
E 抗癫痫药物分析	wenzhou_de.py + table1_wenzhou.py	表 1；图 2

2. A / B / C：21,459 例公共队列

A/B/C 共用一套基准管线：固定切分 → 嵌套子集 → 三模型训练 → 固定测试集评估 → 汇总出图。差别只在特征集：A 为 36 个入院特征，B 加 34 项实验室指标（70 个），C 再加 4 个卒中后并发症（74 个）。下面八个脚本按实际运行顺序排列。

2.1 tabpfn_bench.py（186 行）

用途

主基准脚本，负责算法架构 A、B、C 三套特征集在 full 与 dedup 两个队列上的全部逐规模结果。它按固定种子 42 做分层 80/20 切分，构造嵌套训练子集（n = 100、250、500、1,000、2,000、4,000、8,000、16,000 与全训练集），在同一个子集上训练三个模型，并在固定测试集上计算 AUROC、AUPRC、Brier，保存 metrics.csv 和逐行预测 test_predictions.csv。TabPFN 只在 n ≤ 8,000 上运行（原生上下文上限）；传统模型额外跑到 16,000 与全量。

- **对应算法架构：**A / B / C
- **输入：**data.csv（21,459 例公共队列，SHA-256 见附录 B）
- **输出：**out/<tag>_<features>/metrics.csv；out/<tag>_<features>/test_predictions.csv

运行命令

```
$PY -u tabpfn_bench.py --features A --tag full --out $BASE/out/full_A
# 六个组合：--features A/B/C × --tag full/dedup
```

口径提醒

- **注意：**LR 支路用于交叉核对；算法架构最终 LR 数字由 lr_rerun.py 重算并覆盖，二者预处理一致。
- **注意：**嵌套子集保证不同 n 之间是前缀关系，同一 n 下三模型看到完全相同的患者。
- **注意：**--max-tabpfn-n 默认 10,000，本项目实际评估到 8,000。

代码全文

```
# -*- coding: utf-8 -*-
"""TabPFN 小样本学习曲线基准（D13 三场景 A/B/C × full/dedup）。

设计要点（与 model_d13_server.py 的科学口径保持一致）：
- 切分：stratified 80/20，random_state=42（与场景脚本完全相同），所有模型共用同一测试集。
- 去重：dedup = df.drop_duplicates()（与场景脚本相同）。
- 训练规模梯度：嵌套子集（同一置换序列的前缀），保证不同规模之间可比。
- 三个模型在**同一训练子集**上拟合：LR（winsorize 1/99 + log1p + 标准化）、
  LightGBM（场景脚本的测试模型参数）、TabPFN v2 分类器（零调参）。
- 指标：固定测试集上的 AUROC / AUPRC / Brier；保存逐行预测供 bootstrap。

用法：
  python tabpfn_bench.py --features A --tag full  --out out/full_A
"""
import argparse
import json
import platform
import sys
import time
import warnings
from pathlib import Path

import numpy as np
import pandas as pd
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import average_precision_score, brier_score_loss, roc_auc_score
from sklearn.model_selection import train_test_split

warnings.filterwarnings("ignore")

SEED = 42
WEIGHTS = Path.home() / "TABPFN_BENCH_20261107/weights/tabpfn-v2-classifier-finetuned-zk73skhh.ckpt"

A_DEMO = ["age", "gender", "nihss"]
A_COMORB = ["uremia", "fatty_liver", "diabetes", "hypertension", "coronary_disease",
            "atrial_fibrillation", "hyperuricemia", "hyperlipidaemia"]
A_BRAIN = ["frontal_lobe", "parietal_lobe", "temporal_lobe", "occipital_lobe",
           "insular_lobe", "range_lobe", "basal_ganglia", "capsula_interna",
           "brainstem", "epencephalon", "paraventricular", "centrum_semiovale",
           "thalamus", "subcortex_lobe"]
A_VESSEL = ["aca", "mca", "pca", "va", "ba", "cca_plaque", "ica_plaque", "eca_plaque",
            "ant_circle", "post_circle", "large_ves_as"]
FEATURES_A = A_DEMO + A_COMORB + A_BRAIN + A_VESSEL
LABS = ["plt", "wbc", "rbc", "hba1c", "crp", "tg", "ldl", "hdl", "ast", "alt",
        "bilirubin", "albumin", "urea", "creatinine", "bua", "pt", "aptt", "tt",
        "inr", "d_dimer", "fibrinogen", "ck", "ck_mb", "ldh", "hbdh", "ima",
        "na", "k", "cl", "ca", "p", "lactate", "anion_gap", "tco2"]
POST = ["dvt", "cerebral_hernia", "hydrocephalus", "hypoproteinemia"]
FEATURE_SETS = {"A": FEATURES_A, "B": FEATURES_A + LABS, "C": FEATURES_A + LABS + POST}
```

```
LOG1P_COLS = ["crp", "ast", "alt", "creatinine", "d_dimer", "ck", "ck_mb",
              "ldh", "hbdh", "ima", "anion_gap", "tg", "bua"]
SIZES = [100, 250, 500, 1000, 2000, 4000, 8000, 16000]


ORD = ["age", "nihss", "range_lobe"]


def lr_transform(Xtr, Xte, cols):
    """与场景脚本逐位一致的 LR 预处理：
    - 标准化均值/方差在**原始训练值**上计算（winsorize/log1p 之前），
      应用于 winsorize(1%/99%) + log1p(13 列) 之后的值
    - 连续/有序列（age/nihss/range_lobe）：仅标准化，不 winsorize
    - 二值列：保持 0/1 原值
    （与 model_d13_server.py fit_preproc_lr/apply_preproc_lr 逐位一致：
      stats 在 raw X_train 上估计，apply 时先 winsorize+log1p 再 (x-mu)/sd）
    """
    Xa, Xb = Xtr.copy(), Xte.copy()
    for j, c in enumerate(cols):
        if c in ORD or c in LABS:
            mu, sd = Xa[:, j].mean(), Xa[:, j].std()
            sd = sd if sd > 1e-12 else 1.0
        if c in LABS:
            lo = np.percentile(Xtr[:, j], 1)
            hi = np.percentile(Xtr[:, j], 99)
            Xa[:, j] = np.clip(Xa[:, j], lo, hi)
            Xb[:, j] = np.clip(Xb[:, j], lo, hi)
            if c in LOG1P_COLS:
                Xa[:, j] = np.log1p(np.maximum(Xa[:, j], 0))
                Xb[:, j] = np.log1p(np.maximum(Xb[:, j], 0))
        if c in ORD or c in LABS:
            Xa[:, j] = (Xa[:, j] - mu) / sd
            Xb[:, j] = (Xb[:, j] - mu) / sd
    return Xa, Xb


def nested_subsets(y, sizes, seed=SEED):
    """嵌套分层子集：同一随机置换的前缀，规模之间互为子集。"""
    rng = np.random.default_rng(seed)
    pos = rng.permutation(np.where(y == 1)[0])
    neg = rng.permutation(np.where(y == 0)[0])
    prev = 0
    for n in sizes:
        p = int(round(n * len(pos) / len(y)))
        p = min(max(p, 1), len(pos))
        q = min(max(n - p, 1), len(neg))
        yield n, np.concatenate([pos[:p], neg[:q]])


def metrics(y, p):
    return {"auroc": float(roc_auc_score(y, p)),
            "auprc": float(average_precision_score(y, p)),
            "brier": float(brier_score_loss(y, p))}


def main():
    ap = argparse.ArgumentParser()
    ap.add_argument("--data", default=str(Path.home() / "D13_20261107/data.csv"))
    ap.add_argument("--features", required=True, choices=["A", "B", "C"])
    ap.add_argument("--tag", required=True, choices=["full", "dedup"])
    ap.add_argument("--out", required=True)
    ap.add_argument("--no-tabpfn", action="store_true")
    ap.add_argument("--max-tabpfn-n", type=int, default=10000,
                    help="TabPFN 只在 n<=该值的规模上运行（内存/时间护栏）")
    args = ap.parse_args()

    out = Path(args.out)
    out.mkdir(parents=True, exist_ok=True)
    cols = FEATURE_SETS[args.features]

    df = pd.read_csv(args.data)
    if args.tag == "dedup":
        df = df.drop_duplicates().reset_index(drop=True)
    X = df[cols].to_numpy(dtype=float)
    y = df["second_epilepsy"].to_numpy(dtype=int)
    Xtr_all, Xte, ytr_all, yte = train_test_split(X, y, test_size=0.2, stratify=y,
                                                  random_state=SEED)
    sizes = [s for s in SIZES if s < len(ytr_all)] + [len(ytr_all)]
    print(f"[{args.features}/{args.tag}] rows={len(df)} train={len(ytr_all)} test={len(Xte)} sizes={sizes}")
```

```
    from lightgbm import LGBMClassifier
    from tabpfn import TabPFNClassifier

    rows, preds = [], []
    for n, idx in nested_subsets(ytr_all, sizes):
        Xtr, ytr = Xtr_all[idx], ytr_all[idx]
        spw = float((ytr == 0).sum()) / max(1, int((ytr == 1).sum()))

        # ---- LR ----
        t0 = time.time()
        Xa, Xb = lr_transform(Xtr.copy(), Xte.copy(), cols)
        lr = LogisticRegression(C=1.0, max_iter=3000, random_state=SEED)
        lr.fit(Xa, ytr)
        p_lr = lr.predict_proba(Xb)[:, 1]
        rows.append({"feature_set": args.features, "tag": args.tag, "model": "lr",
                     "n_train": n, **metrics(yte, p_lr), "fit_predict_seconds": time.time() - t0})

        # ---- LightGBM ----
        t0 = time.time()
        gb = LGBMClassifier(n_estimators=300, learning_rate=0.05, num_leaves=31,
                            min_child_samples=50, subsample=0.8, colsample_bytree=0.8,
                            reg_alpha=0.1, reg_lambda=0.1, scale_pos_weight=spw,
                            random_state=SEED, n_jobs=8, verbosity=-1)
        gb.fit(Xtr, ytr)
        p_gb = gb.predict_proba(Xte)[:, 1]
        rows.append({"feature_set": args.features, "tag": args.tag, "model": "lgbm",
                     "n_train": n, **metrics(yte, p_gb), "fit_predict_seconds": time.time() - t0})

        # ---- TabPFN ----
        if not args.no_tabpfn and n <= args.max_tabpfn_n:
            t0 = time.time()
            clf = TabPFNClassifier(model_path=str(WEIGHTS), n_estimators="auto",
                                   random_state=SEED, ignore_pretraining_limits=True,
                                   device="cpu")
            clf.fit(Xtr, ytr)
            p_tp = clf.predict_proba(Xte)[:, 1]
            rows.append({"feature_set": args.features, "tag": args.tag, "model": "tabpfn",
                         "n_train": n, **metrics(yte, p_tp),
                         "fit_predict_seconds": time.time() - t0})

        preds.append(pd.DataFrame({"n_train": n, "y": yte, "p_lr": p_lr, "p_lgbm": p_gb,
                                   **({} if args.no_tabpfn else {"p_tabpfn": p_tp})}))
        print(json.dumps(rows[-3:], ensure_ascii=False))

    pd.DataFrame(rows).to_csv(out / "metrics.csv", index=False, encoding="utf-8-sig")
    pd.concat(preds, ignore_index=True).to_csv(out / "test_predictions.csv", index=False,
                                               encoding="utf-8")
    (out / "run_info.json").write_text(json.dumps({
        "feature_set": args.features, "tag": args.tag, "columns": cols,
        "seed": SEED, "sizes": sizes, "test_rows": int(len(yte)),
        "data": args.data, "python": sys.version.split()[0],
        "platform": platform.platform()}, ensure_ascii=False, indent=2), encoding="utf-8")
    print("DONE", out)


if __name__ == "__main__":
    main()
```

2.2 lr_rerun.py（134 行）

用途

只补算 logistic 回归（LR）支路。已有 TabPFN/LightGBM 结果不重跑；脚本读取 out/<tag>_<feature>/metrics.csv 与 test_predictions.csv，删掉 model == 'lr' 的旧行与 p_lr 旧列，按修正后的预处理顺序重算 B、C × full/dedup 的 LR，再写回原文件。关键修复是训练行索引先 np.sort：lbfgs 对行序敏感，不排序会在第 4–5 位小数产生差异。

- **对应算法架构：**B / C
- **输入：**已由 tabpfn_bench.py 生成的 4 个组合目录（B/C × full/dedup）
- **输出：**原地覆盖上述目录的 metrics.csv 与 test_predictions.csv（仅 LR 行/列）

运行命令

```
$OLD_PY lr_rerun.py        # sklearn 1.7.2 的 D13 老环境
```

口径提醒

- **注意：**锚点：B LR 0.9647814390954035 / 0.7872176622723596；C LR 0.9673034710506947 / 0.7984723092638404。
- **注意：**预处理顺序：μ/σ 在原始训练值上估计 → 1/99 缩尾 → 13 列 log1p → 标准化；顺序反了 AUROC 会差到第 3–4 位。
- **注意：**不设置 OMP_NUM_THREADS，保持与归档运行相同的 BLAS 归约顺序。

代码全文

```
# -*- coding: utf-8 -*-
"""只重算 B/C × full/dedup 的 LR 支路（修正后的逐位一致预处理），
拼接到已有 metrics.csv / test_predictions.csv（LGBM/TabPFN 支路不受影响，保持不变）。

验证锚点（场景存档 results_metrics_full.csv, sklearn 1.7.2/1.9.1 均逐位一致）：
  B lr test 0.9647814390954035 / 0.7872176622723596
  C lr test 0.9673034710506947 / 0.7984723092638404
  A lr test 0.8617507018309483 / 0.22712012938632675（A 无化验列，原本已逐位一致）
"""
import time
import warnings
from pathlib import Path

import numpy as np
import pandas as pd
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import average_precision_score, brier_score_loss, roc_auc_score
from sklearn.model_selection import train_test_split

warnings.filterwarnings("ignore")
SEED = 42

A_DEMO = ["age", "gender", "nihss"]
A_COMORB = ["uremia", "fatty_liver", "diabetes", "hypertension", "coronary_disease",
            "atrial_fibrillation", "hyperuricemia", "hyperlipidaemia"]
A_BRAIN = ["frontal_lobe", "parietal_lobe", "temporal_lobe", "occipital_lobe",
           "insular_lobe", "range_lobe", "basal_ganglia", "capsula_interna",
           "brainstem", "epencephalon", "paraventricular", "centrum_semiovale",
           "thalamus", "subcortex_lobe"]
A_VESSEL = ["aca", "mca", "pca", "va", "ba", "cca_plaque", "ica_plaque", "eca_plaque",
            "ant_circle", "post_circle", "large_ves_as"]
FEATURES_A = A_DEMO + A_COMORB + A_BRAIN + A_VESSEL
LABS = ["plt", "wbc", "rbc", "hba1c", "crp", "tg", "ldl", "hdl", "ast", "alt",
        "bilirubin", "albumin", "urea", "creatinine", "bua", "pt", "aptt", "tt",
        "inr", "d_dimer", "fibrinogen", "ck", "ck_mb", "ldh", "hbdh", "ima",
        "na", "k", "cl", "ca", "p", "lactate", "anion_gap", "tco2"]
POST = ["dvt", "cerebral_hernia", "hydrocephalus", "hypoproteinemia"]
FEATURE_SETS = {"A": FEATURES_A, "B": FEATURES_A + LABS, "C": FEATURES_A + LABS + POST}
LOG1P_COLS = ["crp", "ast", "alt", "creatinine", "d_dimer", "ck", "ck_mb",
              "ldh", "hbdh", "ima", "anion_gap", "tg", "bua"]
SIZES = [100, 250, 500, 1000, 2000, 4000, 8000, 16000]
ORD = ["age", "nihss", "range_lobe"]
BASE = Path.home() / "TABPFN_BENCH_20261107"
DATA = Path.home() / "D13_20261107/data.csv"


def lr_transform(Xtr, Xte, cols):
    """与场景脚本逐位一致：stats 在 raw 训练值上估计；apply 先 winsorize+log1p 再标准化。"""
    Xa, Xb = Xtr.copy(), Xte.copy()
    for j, c in enumerate(cols):
```

```
        if c in ORD or c in LABS:
            mu, sd = Xa[:, j].mean(), Xa[:, j].std()
            sd = sd if sd > 1e-12 else 1.0
        if c in LABS:
            lo = np.percentile(Xtr[:, j], 1)
            hi = np.percentile(Xtr[:, j], 99)
            Xa[:, j] = np.clip(Xa[:, j], lo, hi)
            Xb[:, j] = np.clip(Xb[:, j], lo, hi)
            if c in LOG1P_COLS:
                Xa[:, j] = np.log1p(np.maximum(Xa[:, j], 0))
                Xb[:, j] = np.log1p(np.maximum(Xb[:, j], 0))
        if c in ORD or c in LABS:
            Xa[:, j] = (Xa[:, j] - mu) / sd
            Xb[:, j] = (Xb[:, j] - mu) / sd
    return Xa, Xb


def nested_subsets(y, sizes, seed=SEED):
    rng = np.random.default_rng(seed)
    pos = rng.permutation(np.where(y == 1)[0])
    neg = rng.permutation(np.where(y == 0)[0])
    for n in sizes:
        p = int(round(n * len(pos) / len(y)))
        p = min(max(p, 1), len(pos))
        q = min(max(n - p, 1), len(neg))
        yield n, np.concatenate([pos[:p], neg[:q]])


def metrics(y, p):
    return {"auroc": float(roc_auc_score(y, p)),
            "auprc": float(average_precision_score(y, p)),
            "brier": float(brier_score_loss(y, p))}


def main():
    df_raw = pd.read_csv(DATA)
    for tag in ["full", "dedup"]:
        df = df_raw.drop_duplicates().reset_index(drop=True) if tag == "dedup" else df_raw
        for feature in ["B", "C"]:
            cols = FEATURE_SETS[feature]
            X = df[cols].to_numpy(dtype=float)
            y = df["second_epilepsy"].to_numpy(dtype=int)
            Xtr_all, Xte, ytr_all, yte = train_test_split(
                X, y, test_size=0.2, stratify=y, random_state=SEED)
            sizes = [s for s in SIZES if s < len(ytr_all)] + [len(ytr_all)]
            out = BASE / "out" / f"{tag}_{feature}"
            rows, pred_by_n = [], {}
            for n, idx in nested_subsets(ytr_all, sizes):
                idx = np.sort(idx)  # lbfgs 对行序敏感（浮点求和顺序）：排序后全规模点与场景锚点逐位一致
                Xtr, ytr = Xtr_all[idx], ytr_all[idx]
                t0 = time.time()
                Xa, Xb = lr_transform(Xtr, Xte, cols)
                lr = LogisticRegression(C=1.0, max_iter=3000, random_state=SEED)
                lr.fit(Xa, ytr)
                p = lr.predict_proba(Xb)[:, 1]
                rows.append({"feature_set": feature, "tag": tag, "model": "lr",
                             "n_train": n, **metrics(yte, p),
                             "fit_predict_seconds": time.time() - t0})
                pred_by_n[n] = p
                print(f"{tag}/{feature} n={n} auroc={rows[-1]['auroc']:.4f} "
                      f"auprc={rows[-1]['auprc']:.4f}", flush=True)

            # 拼接 metrics.csv
            mp = out / "metrics.csv"
            m = pd.read_csv(mp)
            m = m[m.model != "lr"]
            m = pd.concat([m, pd.DataFrame(rows)], ignore_index=True)
            m.to_csv(mp, index=False, encoding="utf-8-sig")

            # 拼接 test_predictions.csv：逐 n_train 块替换 p_lr
            pp = out / "test_predictions.csv"
            pr = pd.read_csv(pp)
            new_lr = np.empty(len(pr))
            for n, p in pred_by_n.items():
                mask = (pr.n_train == n).to_numpy()
                assert mask.sum() == len(p), f"block size mismatch n={n}"
                new_lr[mask] = p
            pr["p_lr"] = new_lr
            pr.to_csv(pp, index=False, encoding="utf-8")
```

```
            print(f"DONE {tag}/{feature}", flush=True)


if __name__ == "__main__":
    main()
```

2.3 verify_lr_172.py（71 行）

用途

最小化验证脚本：只拟合 B、C 全训练集的 LR，打印测试 AUROC/AUPRC/Brier，用来确认当前 Python 环境（scikit-learn 1.7.2）能逐位复现归档锚点。它不写文件，只看标准输出。

- **对应算法架构：**A / B / C
- **输入：**data.csv
- **输出：**标准输出（三行指标 + 与锚点的对照）

运行命令

```
$OLD_PY verify_lr_172.py
```

口径提醒

- **注意：**预期：B 0.9647814390954035 / 0.7872176622723596；C 0.9673034710506947 / 0.7984723092638404。
- **注意：**任何一位对不上，先检查 sklearn 版本与 OMP_NUM_THREADS，再检查数据文件指纹。

代码全文

```
# -*- coding: utf-8 -*-
"""用 D13 原 venv（sklearn 1.7.2）重拟合 B/C 的 LR 测试模型，核对场景存档锚点。
预期（results_metrics_full.csv）：
  B lr test_auroc 0.9647814390954035  test_auprc 0.7872176622723596
  C lr test_auroc 0.9673034710506947  test_auprc 0.7984723092638404
"""
import sys
from pathlib import Path

import numpy as np
import pandas as pd
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import average_precision_score, brier_score_loss, roc_auc_score
from sklearn.model_selection import train_test_split

SEED = 42
A_DEMO = ["age", "gender", "nihss"]
A_COMORB = ["uremia", "fatty_liver", "diabetes", "hypertension", "coronary_disease",
            "atrial_fibrillation", "hyperuricemia", "hyperlipidaemia"]
A_BRAIN = ["frontal_lobe", "parietal_lobe", "temporal_lobe", "occipital_lobe",
           "insular_lobe", "range_lobe", "basal_ganglia", "capsula_interna",
           "brainstem", "epencephalon", "paraventricular", "centrum_semiovale",
           "thalamus", "subcortex_lobe"]
A_VESSEL = ["aca", "mca", "pca", "va", "ba", "cca_plaque", "ica_plaque", "eca_plaque",
            "ant_circle", "post_circle", "large_ves_as"]
FEATURES_A = A_DEMO + A_COMORB + A_BRAIN + A_VESSEL
LABS = ["plt", "wbc", "rbc", "hba1c", "crp", "tg", "ldl", "hdl", "ast", "alt",
        "bilirubin", "albumin", "urea", "creatinine", "bua", "pt", "aptt", "tt",
        "inr", "d_dimer", "fibrinogen", "ck", "ck_mb", "ldh", "hbdh", "ima",
        "na", "k", "cl", "ca", "p", "lactate", "anion_gap", "tco2"]
POST = ["dvt", "cerebral_hernia", "hydrocephalus", "hypoproteinemia"]
FEATURE_SETS = {"A": FEATURES_A, "B": FEATURES_A + LABS, "C": FEATURES_A + LABS + POST}
LOG1P_COLS = ["crp", "ast", "alt", "creatinine", "d_dimer", "ck", "ck_mb",
              "ldh", "hbdh", "ima", "anion_gap", "tg", "bua"]
ORD = ["age", "nihss", "range_lobe"]


def lr_transform(Xtr, Xte, cols):
    """与场景脚本逐位一致：stats 在 raw 训练值上估计；apply 先 winsorize+log1p 再标准化。"""
    Xa, Xb = Xtr.copy(), Xte.copy()
    for j, c in enumerate(cols):
        if c in ORD or c in LABS:
            mu, sd = Xa[:, j].mean(), Xa[:, j].std()
            sd = sd if sd > 1e-12 else 1.0
        if c in LABS:
            lo = np.percentile(Xtr[:, j], 1)
            hi = np.percentile(Xtr[:, j], 99)
            Xa[:, j] = np.clip(Xa[:, j], lo, hi)
            Xb[:, j] = np.clip(Xb[:, j], lo, hi)
            if c in LOG1P_COLS:
                Xa[:, j] = np.log1p(np.maximum(Xa[:, j], 0))
                Xb[:, j] = np.log1p(np.maximum(Xb[:, j], 0))
        if c in ORD or c in LABS:
            Xa[:, j] = (Xa[:, j] - mu) / sd
```

```
            Xb[:, j] = (Xb[:, j] - mu) / sd
    return Xa, Xb


df = pd.read_csv(sys.argv[1])
X = df[FEATURE_SETS["C"]].to_numpy(dtype=float)
y = df["second_epilepsy"].to_numpy(dtype=int)
Xtr_all, Xte, ytr_all, yte = train_test_split(X, y, test_size=0.2, stratify=y,
                                              random_state=SEED)
for f in ["A", "B", "C"]:
    cols = FEATURE_SETS[f]
    Xa, Xb = lr_transform(Xtr_all[:, :len(cols)].copy(), Xte[:, :len(cols)].copy(), cols)
    lr = LogisticRegression(C=1.0, max_iter=3000)
    lr.fit(Xa, ytr_all)
    p = lr.predict_proba(Xb)[:, 1]
    print(f, "auroc", roc_auc_score(yte, p), "auprc", average_precision_score(yte, p),
          "brier", brier_score_loss(yte, p))
```

2.4 bench_analysis.py（196 行）

用途

汇总与出图。扫描 6 个组合目录，只接受 metrics.csv 与 test_predictions.csv 同时存在、且三模型 n_train 对齐的组合；在 n = 8,000 上计算校准斜率与截距，在测试行上做 2,000 次配对 bootstrap，输出 combined_metrics.csv、bench_summary_final.json，以及学习曲线、ROC/PR、校准曲线、决策曲线四类图（A/B/C 各一套）。

- **对应算法架构：**A / B / C
- **输入：**out/ 下 6 个组合目录（A/B/C × full/dedup）
- **输出：**out/combined_metrics.csv；out/bench_summary_final.json；figs/fig_learning_A/B/C.png、fig_roc_pr_*.png、fig_calib_*.png、fig_dca_*.png

运行命令

```
$PY bench_analysis.py
```

口径提醒

- **注意：**缺文件或 n 不对齐的组合会被跳过并记入 skipped，不会静默拼数。
- **注意：**校准斜率用 logit(p) 对 y 的 LR（C = 1e6）估计；bootstrap 固定种子 42。
- **注意：**算法架构正文嵌入 ROC/PR、学习、校准、决策四类图；特征重要性图由 2.8 另行生成。

代码全文

```
# -*- coding: utf-8 -*-
"""汇总 A/B/C × full/dedup 的 TabPFN 基准结果：表 + 图 + JSON。

只统计通过口径校验的产物：metrics.csv 与 test_predictions.csv 必须同时存在，
且三模型在相同 n_train 上预测同一批测试行（列 p_lr/p_lgbm/p_tabpfn）。
不合格（缺文件、n_train 不对齐）的组合直接跳过并记录。
"""
import json
import warnings
from pathlib import Path

import numpy as np
import pandas as pd
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import average_precision_score, brier_score_loss, roc_auc_score, roc_curve

warnings.filterwarnings("ignore")
BASE = Path.home() / "TABPFN_BENCH_20261107"
OUT = BASE / "out"
FIG = BASE / "figs"
FIG.mkdir(exist_ok=True)

COLORS = {"lr": "#4C72B0", "lgbm": "#DD8452", "tabpfn": "#55A868"}
COMBOS = [(f, t) for f in ["A", "B", "C"] for t in ["full", "dedup"]]


def calib(y, p):
    lp = np.log(np.clip(p, 1e-6, 1 - 1e-6) / (1 - np.clip(p, 1e-6, 1 - 1e-6)))
    m = LogisticRegression(C=1e6, max_iter=1000).fit(lp.reshape(-1, 1), y)
    return float(m.coef_[0, 0]), float(m.intercept_[0])


def boot_delta(y, p1, p2, n=2000, seed=42):
    rng = np.random.default_rng(seed)
    d = []
    for _ in range(n):
        i = rng.integers(0, len(y), len(y))
        if y[i].min() == y[i].max():
            continue
        d.append(roc_auc_score(y[i], p1[i]) - roc_auc_score(y[i], p2[i]))
    d = np.array(d)
    return float(np.mean(d)), float(np.percentile(d, 2.5)), float(np.percentile(d, 97.5))


def load_combo(feature, tag):
    d = OUT / f"{tag}_{feature}"
    mp, pp = d / "metrics.csv", d / "test_predictions.csv"
    if not (mp.exists() and pp.exists()):
        return None, None, f"missing files: {d}"
    m = pd.read_csv(mp)
```

```
    p = pd.read_csv(pp)
    # 口径校验：每个 n_train 下三模型都有预测列且测试集行数一致
    sizes = sorted(p.n_train.unique())
    need = {"p_lr", "p_lgbm"}
    if not need.issubset(p.columns):
        return None, None, "missing prediction columns"
    return m, p, None


def main():
    summary, skipped = {}, []
    all_metrics = []
    for f, t in COMBOS:
        m, p, err = load_combo(f, t)
        if err:
            skipped.append({"feature": f, "tag": t, "reason": err}); continue
        m["feature_set"], m["tag"] = f, t
        all_metrics.append(m)
        sizes = sorted(p.n_train.unique())
        common = [s for s in sizes if s <= 8000]
        n0 = 8000 if 8000 in common else max(common)
        idx = p.n_train == n0
        y = p.loc[idx, "y"].to_numpy(int)
        has_tab = "p_tabpfn" in p.columns and p.loc[idx, "p_tabpfn"].notna().all()
        rec = {"n_eval": int(n0), "test_rows": int(idx.sum()), "test_events": int(y.sum())}
        preds = {k: p.loc[idx, k].to_numpy(float) for k in ["p_lr", "p_lgbm"]}
        if has_tab:
            preds["p_tabpfn"] = p.loc[idx, "p_tabpfn"].to_numpy(float)
        for k, v in preds.items():
            sl, ic = calib(y, v)
            rec[k] = {"auroc": float(roc_auc_score(y, v)),
                      "auprc": float(average_precision_score(y, v)),
                      "brier": float(brier_score_loss(y, v)),
                      "cal_slope": sl, "cal_intercept": ic}
        if has_tab:
            rec["delta_auroc_tabpfn_minus_lgbm"] = boot_delta(y, preds["p_tabpfn"], preds["p_lgbm"])
            rec["delta_auroc_tabpfn_minus_lr"] = boot_delta(y, preds["p_tabpfn"], preds["p_lr"])
            rec["delta_auroc_lgbm_minus_lr"] = boot_delta(y, preds["p_lgbm"], preds["p_lr"])
        # 数据效率：TabPFN 达到 LGBM 在 8,000 时的 AUROC 所需最小 n
        if has_tab:
            lgb_8000 = roc_auc_score(y, preds["p_lgbm"])
            tp_curve = m[m.model == "tabpfn"].sort_values("n_train")
            reach = tp_curve[tp_curve.auroc >= lgb_8000 - 0.002]
            rec["tabpfn_n_to_match_lgbm_at_8000"] = int(reach.n_train.min()) if len(reach) else None
        summary[f"{f}_{t}"] = rec

    metrics = pd.concat(all_metrics, ignore_index=True)
    metrics.to_csv(OUT / "combined_metrics.csv", index=False)
    (OUT / "bench_summary.json").write_text(
        json.dumps({"summary": summary, "skipped": skipped}, ensure_ascii=False, indent=2),
        encoding="utf-8")

    # ---- 图：每个特征集一张学习曲线（full 实线 + dedup 虚线）----
    import matplotlib
    matplotlib.use("Agg")
    import matplotlib.pyplot as plt

    for f in ["A", "B", "C"]:
        fig, ax = plt.subplots(figsize=(5.6, 4.2))
        for t, ls in [("full", "-"), ("dedup", "--")]:
            m, p, err = load_combo(f, t)
            if err:
                continue
            for model in ["lr", "lgbm", "tabpfn"]:
                s = m[m.model == model].sort_values("n_train")
                if len(s) == 0:
                    continue
                ax.plot(s.n_train, s.auroc, ls, marker="o", ms=3.5,
                        color=COLORS[model], alpha=1.0 if t == "full" else 0.55,
                        label=f"{model.upper()} ({t})")
        ax.set_xscale("log")
        ax.set_xlabel("Training set size (n, log scale)")
        ax.set_ylabel("Test AUROC")
        ax.set_title(f"Feature set {f}: TabPFN vs conventional models")
        ax.legend(frameon=False, fontsize=6.5, ncol=2)
        ax.grid(alpha=0.25, lw=0.5)
        fig.tight_layout()
        fig.savefig(FIG / f"fig_learning_{f}.png", dpi=300)
        plt.close(fig)
```

```
    # ---- 图：n=8000 的 ROC / PR / 校准 / DCA ----
    for f, t in [(f, "full") for f in ["A", "B", "C"]]:
        m, p, err = load_combo(f, t)
        if err:
            continue
        idx = p.n_train == 8000
        y = p.loc[idx, "y"].to_numpy(int)
        preds = {k: p.loc[idx, k].to_numpy(float) for k in ["p_lr", "p_lgbm", "p_tabpfn"]
                 if k in p.columns and p.loc[idx, k].notna().all()}
        fig, axes = plt.subplots(1, 2, figsize=(9, 4))
        for k, v in preds.items():
            fpr, tpr, _ = roc_curve(y, v)
            axes[0].plot(fpr, tpr, color=COLORS[k.replace("p_", "")],
                         label=f"{k[2:].upper()} AUROC {roc_auc_score(y, v):.3f}")
        axes[0].plot([0, 1], [0, 1], "k--", lw=0.7)
        axes[0].set_xlabel("1 - Specificity"); axes[0].set_ylabel("Sensitivity")
        axes[0].legend(frameon=False, fontsize=7)
        for k, v in preds.items():
            order = np.argsort(-v)
            tp = np.cumsum(y[order]) / y.sum()
            pr = tp / (np.arange(len(y)) + 1)
            axes[1].plot(np.arange(len(y)) / len(y), pr,
                         color=COLORS[k.replace("p_", "")],
                         label=f"{k[2:].upper()} AUPRC {average_precision_score(y, v):.3f}")
        axes[1].set_xlabel("Recall"); axes[1].set_ylabel("Precision")
        axes[1].legend(frameon=False, fontsize=7)
        fig.suptitle(f"Feature set {f}, n_train = 8,000 (full cohort)")
        fig.tight_layout()
        fig.savefig(FIG / f"fig_roc_pr_{f}.png", dpi=300)
        plt.close(fig)

        fig, ax = plt.subplots(figsize=(4.6, 4.2))
        for k, v in preds.items():
            bins = np.quantile(v, np.linspace(0, 1, 6))
            bi = np.clip(np.searchsorted(bins, v, side="right") - 1, 0, 4)
            xs = [v[bi == b].mean() for b in range(5) if (bi == b).sum() > 5]
            ys = [y[bi == b].mean() for b in range(5) if (bi == b).sum() > 5]
            ax.plot(xs, ys, "o-", ms=4, color=COLORS[k.replace("p_", "")], label=k[2:].upper())
        ax.plot([0, 0.9], [0, 0.9], "k--", lw=0.7)
        ax.set_xlabel("Predicted risk (quintiles)"); ax.set_ylabel("Observed rate")
        ax.legend(frameon=False, fontsize=7)
        ax.set_title(f"Calibration, feature set {f} (n=8,000)")
        fig.tight_layout(); fig.savefig(FIG / f"fig_calib_{f}.png", dpi=300); plt.close(fig)

        fig, ax = plt.subplots(figsize=(4.8, 4.2))
        th = np.arange(0.05, 0.61, 0.01)
        prev = y.mean()
        for k, v in preds.items():
            nb = [((v >= x) & (y == 1)).sum() / len(y)
                  - ((v >= x) & (y == 0)).sum() / len(y) * x / (1 - x) for x in th]
            ax.plot(th, nb, color=COLORS[k.replace("p_", "")], label=k[2:].upper())
        ax.plot(th, [prev - (1 - prev) * x / (1 - x) for x in th], "k--", lw=0.7, label="Treat all")
        ax.axhline(0, color="gray", lw=0.6)
        ax.set_ylim(-0.05, max(0.5, prev * 1.1))
        ax.set_xlabel("Threshold probability"); ax.set_ylabel("Net benefit")
        ax.legend(frameon=False, fontsize=7)
        ax.set_title(f"Decision curve, feature set {f} (n=8,000)")
        fig.tight_layout(); fig.savefig(FIG / f"fig_dca_{f}.png", dpi=300); plt.close(fig)

    print(json.dumps({"summary": summary, "skipped": skipped}, ensure_ascii=False, indent=1))
    print("FIGURES", sorted(str(p) for p in FIG.glob("*.png")))
    print("DONE")


if __name__ == "__main__":
    main()
```

2.5 table1_baseline.py（41 行）

用途

计算 A 算法架构表 1：36 个入院特征按一年内癫痫结局分组，full 与 dedup 两套；连续变量给中位数与四分位，二值变量给百分比。结果以 JSON 打到标准输出，重定向保存为 table1.json。

- **对应算法架构：**A
- **输入：**data.csv
- **输出：**标准输出 → results/table1.json

运行命令

```
py table1_baseline.py data.csv > table1.json
```

口径提醒

- **注意：**只做描述统计，不建模、不调参。
- **注意：**dedup 用 df.drop_duplicates()，与主基准完全一致。

代码全文

```
# -*- coding: utf-8 -*-
"""计算 A 论文 Table 1：36 特征按结局分组的基线描述（full 与 dedup）。"""
import json
import sys
from pathlib import Path

import numpy as np
import pandas as pd

FEATURES_A = ["age", "gender", "nihss",
              "uremia", "fatty_liver", "diabetes", "hypertension", "coronary_disease",
              "atrial_fibrillation", "hyperuricemia", "hyperlipidaemia",
              "frontal_lobe", "parietal_lobe", "temporal_lobe", "occipital_lobe",
              "insular_lobe", "range_lobe", "basal_ganglia", "capsula_interna",
              "brainstem", "epencephalon", "paraventricular", "centrum_semiovale",
              "thalamus", "subcortex_lobe",
              "aca", "mca", "pca", "va", "ba", "cca_plaque", "ica_plaque", "eca_plaque",
              "ant_circle", "post_circle", "large_ves_as"]

df = pd.read_csv(sys.argv[1])
y = df["second_epilepsy"].to_numpy(int)

def med_iqr(a):
    return [float(np.median(a)), float(np.percentile(a, 25)), float(np.percentile(a, 75))]

out = {}
for tag, d in [("full", df), ("dedup", df.drop_duplicates())]:
    t = {}
    t["n_total"] = int(len(d)); t["n_pos"] = int(d.second_epilepsy.sum())
    t["n_neg"] = int(len(d) - d.second_epilepsy.sum())
    for c in FEATURES_A:
        v0 = d.loc[d.second_epilepsy == 0, c].to_numpy(float)
        v1 = d.loc[d.second_epilepsy == 1, c].to_numpy(float)
        if c in ["age", "nihss", "range_lobe"]:
            t[c] = {"neg_med_iqr": med_iqr(v0), "pos_med_iqr": med_iqr(v1)}
        else:
            t[c] = {"neg_pct": round(float(v0.mean()) * 100, 2),
                    "pos_pct": round(float(v1.mean()) * 100, 2)}
    out[tag] = t

print(json.dumps(out, ensure_ascii=False, indent=1))
```

2.6 restore_analyses.py（276 行）

用途

重建三套算法架构的描述性与可解释性结果：A 的表 1（36 个入院特征）、B 的表 1（70 个特征）、C 的表 1（4 个卒中后并发症）按结局分组的总体/无 PSE/PSE 与 P 值；两模型的特征重要性（LightGBM 总增益占比、标准化 logistic 回归系数绝对值）；以及 A 的训练集五折交叉验证与汇总袋外性能。全部在最终口径下重算，输出一个 JSON 供算法架构装配脚本读取。

- **对应算法架构：**A / B / C
- **输入：**data.csv（21,459 例公共队列）
- **输出：**results/restore/restore.json

运行命令

```
py -3.12 restore_analyses.py
```

口径提醒

- **注意：**连续变量用 Mann–Whitney U、二值变量用 χ^2；实验室指标必须按连续变量处理，否则卡方会报负值。
- **注意：**特征重要性在 full 训练集（n = 17,167）上拟合后计算；LR 系数对应协议预处理尺度。
- **注意：**C 的表 1 只含四个并发症变量；A 的表 1 含 range_lobe（任一脑叶受累）。

代码全文

```
# -*- coding: utf-8 -*-
"""重建 A/B/C 三篇论文中被精简掉的实证内容：
- 表 1：总体 / 无 PSE / PSE / P 值（连续变量 Mann-Whitney，二值卡方）
- 特征重要性：全训练集 LightGBM 增益占比 + logistic 回归 |系数|
- A 论文的五折交叉验证与 OOF 指标
输出 results/restore/restore.json
"""
import json
import os
import warnings

import numpy as np
import pandas as pd
from scipy import stats
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import average_precision_score, brier_score_loss, roc_auc_score
from sklearn.model_selection import StratifiedKFold, train_test_split

warnings.filterwarnings("ignore")

BASE = os.path.dirname(os.path.abspath(__file__))
DATA = os.path.join(BASE, "data", "data.csv")
OUT = os.path.join(BASE, "results", "restore")
os.makedirs(OUT, exist_ok=True)
SEED = 42

A_DEMO = ["age", "gender", "nihss"]
A_COMORB = ["uremia", "fatty_liver", "diabetes", "hypertension", "coronary_disease",
            "atrial_fibrillation", "hyperuricemia", "hyperlipidaemia"]
A_BRAIN = ["frontal_lobe", "parietal_lobe", "temporal_lobe", "occipital_lobe",
           "insular_lobe", "range_lobe", "basal_ganglia", "capsula_interna",
           "brainstem", "epencephalon", "paraventricular", "centrum_semiovale",
           "thalamus", "subcortex_lobe"]
A_VESSEL = ["aca", "mca", "pca", "va", "ba", "cca_plaque", "ica_plaque", "eca_plaque",
            "ant_circle", "post_circle", "large_ves_as"]
FEATURES_A = A_DEMO + A_COMORB + A_BRAIN + A_VESSEL
LABS = ["plt", "wbc", "rbc", "hba1c", "crp", "tg", "ldl", "hdl", "ast", "alt",
        "bilirubin", "albumin", "urea", "creatinine", "bua", "pt", "aptt", "tt",
        "inr", "d_dimer", "fibrinogen", "ck", "ck_mb", "ldh", "hbdh", "ima",
        "na", "k", "cl", "ca", "p", "lactate", "anion_gap", "tco2"]
POST = ["dvt", "cerebral_hernia", "hydrocephalus", "hypoproteinemia"]
FEATURE_SETS = {"A": FEATURES_A, "B": FEATURES_A + LABS, "C": FEATURES_A + LABS + POST}
LOG1P_COLS = ["crp", "ast", "alt", "creatinine", "d_dimer", "ck", "ck_mb",
              "ldh", "hbdh", "ima", "anion_gap", "tg", "bua"]
ORD = ["age", "nihss", "range_lobe"]

LABELS = {
    "age": ("Age, years", "年龄（岁）"),
    "gender": ("Male", "男性"),
    "nihss": ("NIHSS score", "NIHSS 评分"),
```

```
    "uremia": ("Uremia", "尿毒症"),
    "fatty_liver": ("Fatty liver", "脂肪肝"),
    "diabetes": ("Diabetes mellitus", "糖尿病"),
    "hypertension": ("Hypertension", "高血压"),
    "coronary_disease": ("Coronary heart disease", "冠心病"),
    "atrial_fibrillation": ("Atrial fibrillation", "心房颤动"),
    "hyperuricemia": ("Hyperuricemia", "高尿酸血症"),
    "hyperlipidaemia": ("Hyperlipidemia", "高脂血症"),
    "frontal_lobe": ("Frontal lobe", "额叶"),
    "parietal_lobe": ("Parietal lobe", "顶叶"),
    "temporal_lobe": ("Temporal lobe", "颞叶"),
    "occipital_lobe": ("Occipital lobe", "枕叶"),
    "insular_lobe": ("Insular lobe", "岛叶"),
    "range_lobe": ("Any lobe involved (>=1)", "脑叶受累（任一，≥1）"),
    "basal_ganglia": ("Basal ganglia", "基底节"),
    "capsula_interna": ("Internal capsule", "内囊"),
    "brainstem": ("Brainstem", "脑干"),
    "epencephalon": ("Cerebellum", "小脑"),
    "paraventricular": ("Paraventricular", "脑室旁"),
    "centrum_semiovale": ("Centrum semiovale", "半卵圆中心"),
    "thalamus": ("Thalamus", "丘脑"),
    "subcortex_lobe": ("Subcortical", "皮层下"),
    "aca": ("ACA territory", "ACA 供血区"),
    "mca": ("MCA territory", "MCA 供血区"),
    "pca": ("PCA territory", "PCA 供血区"),
    "va": ("VA territory", "VA 供血区"),
    "ba": ("BA territory", "BA 供血区"),
    "cca_plaque": ("Common carotid plaque", "颈总动脉斑块"),
    "ica_plaque": ("Internal carotid plaque", "颈内动脉斑块"),
    "eca_plaque": ("External carotid plaque", "颈外动脉斑块"),
    "ant_circle": ("Anterior circulation", "前循环"),
    "post_circle": ("Posterior circulation", "后循环"),
    "large_ves_as": ("Large-vessel atherosclerosis", "大血管动脉粥样硬化"),
    "plt": ("Platelet count (×10⁹/L)", "血小板计数（×10⁹/L）"),
    "wbc": ("White blood cell count (×10⁹/L)", "白细胞计数（×10⁹/L）"),
    "rbc": ("Red blood cell count (×10¹²/L)", "红细胞计数（×10¹²/L）"),
    "hba1c": ("HbA1c (%)", "糖化血红蛋白（%）"),
    "crp": ("C-reactive protein (mg/L)", "C 反应蛋白（mg/L）"),
    "tg": ("Triglycerides (mmol/L)", "甘油三酯（mmol/L）"),
    "ldl": ("LDL cholesterol (mmol/L)", "低密度脂蛋白胆固醇（mmol/L）"),
    "hdl": ("HDL cholesterol (mmol/L)", "高密度脂蛋白胆固醇（mmol/L）"),
    "ast": ("Aspartate aminotransferase (U/L)", "谷草转氨酶（U/L）"),
    "alt": ("Alanine aminotransferase (U/L)", "谷丙转氨酶（U/L）"),
    "bilirubin": ("Total bilirubin (μmol/L)", "总胆红素（μmol/L）"),
    "albumin": ("Albumin (g/L)", "白蛋白（g/L）"),
    "urea": ("Urea (mmol/L)", "尿素（mmol/L）"),
    "creatinine": ("Creatinine (μmol/L)", "肌酐（μmol/L）"),
    "bua": ("Uric acid (μmol/L)", "血尿酸（μmol/L）"),
    "pt": ("Prothrombin time (s)", "凝血酶原时间（s）"),
    "aptt": ("Activated partial thromboplastin time (s)", "活化部分凝血活酶时间（s）"),
    "tt": ("Thrombin time (s)", "凝血酶时间（s）"),
    "inr": ("International normalized ratio", "国际标准化比值"),
    "d_dimer": ("D-dimer (mg/L)", "D-二聚体（mg/L）"),
    "fibrinogen": ("Fibrinogen (g/L)", "纤维蛋白原（g/L）"),
    "ck": ("Creatine kinase (U/L)", "肌酸激酶（U/L）"),
    "ck_mb": ("Creatine kinase-MB (U/L)", "肌酸激酶同工酶 MB（U/L）"),
    "ldh": ("Lactate dehydrogenase (U/L)", "乳酸脱氢酶（U/L）"),
    "hbdh": ("α-Hydroxybutyrate dehydrogenase (U/L)", "α-羟丁酸脱氢酶（U/L）"),
    "ima": ("Ischemia-modified albumin (AU)", "缺血修饰白蛋白（AU）"),
    "na": ("Sodium (mmol/L)", "钠（mmol/L）"),
    "k": ("Potassium (mmol/L)", "钾（mmol/L）"),
    "cl": ("Chloride (mmol/L)", "氯（mmol/L）"),
    "ca": ("Calcium (mmol/L)", "钙（mmol/L）"),
    "p": ("Phosphorus (mmol/L)", "磷（mmol/L）"),
    "lactate": ("Lactate (mmol/L)", "乳酸（mmol/L）"),
    "anion_gap": ("Anion gap (mmol/L)", "阴离子间隙（mmol/L）"),
    "tco2": ("Total CO2 (mmol/L)", "总二氧化碳（mmol/L）"),
    "dvt": ("Deep vein thrombosis", "深静脉血栓"),
    "cerebral_hernia": ("Cerebral hernia", "脑疝"),
    "hydrocephalus": ("Hydrocephalus", "脑积水"),
    "hypoproteinemia": ("Hypoproteinemia", "低蛋白血症"),
}

SECTIONS = {
    "A": [("__SEC__", "Demographics and comorbidities", "人口学与合并症", 0),
          ("__SEC__", "Initial imaging: lesion location", "初始影像：病灶位置", 11),
          ("__SEC__", "Vascular features", "血管特征", 25)],
}
```

```
def lr_transform(Xtr, Xte, cols):
    Xa, Xb = Xtr.copy(), Xte.copy()
    for j, c in enumerate(cols):
        if c in ORD or c in LABS:
            mu, sd = Xa[:, j].mean(), Xa[:, j].std()
            sd = sd if sd > 1e-12 else 1.0
        if c in LABS:
            lo = np.percentile(Xtr[:, j], 1)
            hi = np.percentile(Xtr[:, j], 99)
            Xa[:, j] = np.clip(Xa[:, j], lo, hi)
            Xb[:, j] = np.clip(Xb[:, j], lo, hi)
            if c in LOG1P_COLS:
                Xa[:, j] = np.log1p(np.maximum(Xa[:, j], 0))
                Xb[:, j] = np.log1p(np.maximum(Xb[:, j], 0))
        if c in ORD or c in LABS:
            Xa[:, j] = (Xa[:, j] - mu) / sd
            Xb[:, j] = (Xb[:, j] - mu) / sd
    return Xa, Xb


def fmt_med(v, one=False):
    if one:
        return f"{v[0]:.1f} ({v[1]:.1f}–{v[2]:.1f})"
    return f"{v[0]:g} ({v[1]:g}–{v[2]:g})"


def pfmt(p):
    return "<0.001" if p < 0.001 else f"{p:.3f}"


def table1(df, cols):
    y = df["second_epilepsy"].to_numpy(int)
    d0, d1 = df[y == 0], df[y == 1]
    rows = []
    for c in cols:
        v = df[c].to_numpy(float)
        v0, v1 = v[y == 0], v[y == 1]
        cont = c in ("age", "nihss", "range_lobe") or c in LABS
        if cont:
            med = lambda a: [float(np.median(a)), float(np.percentile(a, 25)), float(np.percentile(a, 75))]
            p = stats.mannwhitneyu(v0, v1, alternative="two-sided").pvalue
            rows.append({"var": c, "overall": fmt_med(med(v)),
                         "neg": fmt_med(med(v0), one=True), "pos": fmt_med(med(v1), one=True),
                         "p": pfmt(p)})
        else:
            tab = np.array([[int(v0.sum()), int(len(v0) - v0.sum())],
                            [int(v1.sum()), int(len(v1) - v1.sum())]])
            p = stats.chi2_contingency(tab, correction=False)[1]
            rows.append({"var": c, "overall": f"{int(v.sum())} ({v.mean()*100:.1f}%)",
                         "neg": f"{int(v0.sum())} ({v0.mean()*100:.1f}%)",
                         "pos": f"{int(v1.sum())} ({v1.mean()*100:.1f}%)", "p": pfmt(p)})
    return rows


def main():
    df = pd.read_csv(DATA)
    y = df["second_epilepsy"].to_numpy(int)
    Xall = {k: df[v].to_numpy(float) for k, v in FEATURE_SETS.items()}
    Xtr_A, Xte_A, ytr, yte = train_test_split(Xall["A"], y, test_size=0.2, stratify=y, random_state=SEED)

    out = {"table1": {}, "importance": {}, "cv_A": {}, "meta": {}}
    for fs in ["A", "B", "C"]:
        cols = FEATURES_A if fs == "A" else FEATURE_SETS[fs]
        out["table1"][fs] = table1(df, cols)
    # C 表 1 只需四个并发症
    out["table1"]["C"] = [r for r in out["table1"]["C"] if r["var"] in POST]

    # ---------- 全训练集拟合：特征重要性 ----------
    from lightgbm import LGBMClassifier
    for fs in ["A", "B", "C"]:
        cols = FEATURE_SETS[fs]
        X = df[cols].to_numpy(float)
        Xtr, Xte, ytr_, yte_ = train_test_split(X, y, test_size=0.2, stratify=y, random_state=SEED)
        spw = float((ytr_ == 0).sum()) / max(1, int((ytr_ == 1).sum()))
        gb = LGBMClassifier(n_estimators=300, learning_rate=0.05, num_leaves=31,
                            min_child_samples=50, subsample=0.8, colsample_bytree=0.8,
                            reg_alpha=0.1, reg_lambda=0.1, scale_pos_weight=spw,
                            random_state=SEED, n_jobs=8, verbosity=-1)
```

```
        gb.fit(Xtr, ytr_)
        gain = gb.booster_.feature_importance(importance_type="gain")
        share = gain / gain.sum() if gain.sum() > 0 else gain
        lr = LogisticRegression(C=1.0, max_iter=3000, random_state=SEED)
        Xa, Xb = lr_transform(Xtr.copy(), Xte.copy(), cols)
        lr.fit(Xa, ytr_)
        coef = np.abs(lr.coef_[0])
        order_g = np.argsort(-gain)
        order_l = np.argsort(-coef)
        ranks_l = {cols[i]: int(np.where(order_l == i)[0][0]) + 1 for i in range(len(cols))}
        ranks_g = {cols[i]: int(np.where(order_g == i)[0][0]) + 1 for i in range(len(cols))}
        out["importance"][fs] = {
            "n_features": len(cols),
            "lgbm": [{"var": cols[i], "gain_share": float(share[i]), "rank": int(r + 1)}
                     for r, i in enumerate(order_g)],
            "lr": [{"var": cols[i], "abs_coef": float(coef[i]), "rank": int(r + 1)}
                   for r, i in enumerate(order_l)],
            "ranks_lgbm": ranks_g, "ranks_lr": ranks_l,
            "test_auroc_lgbm": float(roc_auc_score(yte_, gb.predict_proba(Xte)[:, 1])),
            "test_auroc_lr": float(roc_auc_score(yte_, lr.predict_proba(Xb)[:, 1])),
        }
        print(fs, "top lgbm", [(cols[i], round(float(share[i]) * 100, 2)) for i in order_g[:5]])
        print(fs, "top lr", [(cols[i], round(float(coef[i]), 3)) for i in order_l[:5]])

    # ---------- A 五折 CV + OOF ----------
    X = Xall["A"]
    Xtr, Xte, ytr_, yte_ = train_test_split(X, y, test_size=0.2, stratify=y, random_state=SEED)
    skf = StratifiedKFold(n_splits=5, shuffle=True, random_state=SEED)
    rec = {"lr": [], "lgbm": []}
    oof = {m: np.zeros(len(ytr_)) for m in ("lr", "lgbm")}
    for k, (ti, vi) in enumerate(skf.split(Xtr, ytr_)):
        Xa, Xb = lr_transform(Xtr[ti].copy(), Xtr[vi].copy(), FEATURES_A)
        lr = LogisticRegression(C=1.0, max_iter=3000, random_state=SEED)
        lr.fit(Xa, ytr_[ti])
        p = lr.predict_proba(Xb)[:, 1]
        oof["lr"][vi] = p
        rec["lr"].append([float(roc_auc_score(ytr_[vi], p)), float(average_precision_score(ytr_[vi], p)),
                          float(brier_score_loss(ytr_[vi], p))])
        spw = float((ytr_[ti] == 0).sum()) / max(1, int((ytr_[ti] == 1).sum()))
        gb = LGBMClassifier(n_estimators=300, learning_rate=0.05, num_leaves=31,
                            min_child_samples=50, subsample=0.8, colsample_bytree=0.8,
                            reg_alpha=0.1, reg_lambda=0.1, scale_pos_weight=spw,
                            random_state=SEED, n_jobs=8, verbosity=-1)
        gb.fit(Xtr[ti], ytr_[ti])
        p = gb.predict_proba(Xtr[vi])[:, 1]
        oof["lgbm"][vi] = p
        rec["lgbm"].append([float(roc_auc_score(ytr_[vi], p)), float(average_precision_score(ytr_[vi], p)),
                            float(brier_score_loss(ytr_[vi], p))])
    for m in ("lr", "lgbm"):
        arr = np.array(rec[m])
        out["cv_A"][m] = {
            "auroc_mean": float(arr[:, 0].mean()), "auroc_sd": float(arr[:, 0].std(ddof=1)),
            "auprc_mean": float(arr[:, 1].mean()), "auprc_sd": float(arr[:, 1].std(ddof=1)),
            "brier_mean": float(arr[:, 2].mean()), "brier_sd": float(arr[:, 2].std(ddof=1)),
            "oof_auroc": float(roc_auc_score(ytr_, oof[m])),
            "oof_auprc": float(average_precision_score(ytr_, oof[m])),
            "oof_brier": float(brier_score_loss(ytr_, oof[m])),
        }
    out["meta"] = {"n_total": int(len(df)), "n_pos": int(y.sum()),
                   "n_train": int(len(ytr_)), "n_test": int(len(yte_)),
                   "data_sha_note": "public Dryad v8, used as provided"}
    with open(os.path.join(OUT, "restore.json"), "w", encoding="utf-8") as f:
        json.dump(out, f, ensure_ascii=False, indent=1)
    print("saved", os.path.join(OUT, "restore.json"))


if __name__ == "__main__":
    main()
```

2.7 bootstrap_ci.py（65 行）

用途

为 A/B/C 的表 2 补 95% bootstrap 置信区间：三个模型 × full/dedup 两个队列 × n = 8,000 与最大训练规模（17,167 或 10,915）× AUROC/AUPRC/Brier 三项指标，每项 2,000 次对测试行的重抽样，种子 42。结果写入 ci.json，算法架构表 2 的括号区间即来自这里。

- **对应算法架构：**A / B / C
- **输入：**results/preds_full_*_final.csv、results/preds_dedup_*_final.csv
- **输出：**results/restore/ci.json

运行命令

```
py -3.12 bootstrap_ci.py
```

口径提醒

- **注意：**公开表无患者标识，只能按行重抽样；这一局限已写进三套算法架构的讨论。
- **注意：**TabPFN 不在 17,167 上评估，对应键不存在，表中以“—”呈现。

代码全文

```
# -*- coding: utf-8 -*-
"""为 A/B/C 的 n=8,000 与全训练集补算每个模型的 95% bootstrap 置信区间。"""
import json
import os

import numpy as np
import pandas as pd
from sklearn.metrics import average_precision_score, brier_score_loss, roc_auc_score

BASE = os.path.dirname(os.path.abspath(__file__))
RES = os.path.join(BASE, "results")
OUT = os.path.join(RES, "restore")
os.makedirs(OUT, exist_ok=True)
SEED = 42
B = 2000


def ci(y, p, rng):
    vals = {"auroc": [], "auprc": [], "brier": []}
    n = len(y)
    for _ in range(B):
        idx = rng.integers(0, n, n)
        yy, pp = y[idx], p[idx]
        if yy.min() == yy.max():
            continue
        vals["auroc"].append(roc_auc_score(yy, pp))
        vals["auprc"].append(average_precision_score(yy, pp))
        vals["brier"].append(brier_score_loss(yy, pp))
    out = {}
    for k, v in vals.items():
        out[k] = [float(np.percentile(v, 2.5)), float(np.percentile(v, 97.5))]
    return out


def main():
    rng = np.random.default_rng(SEED)
    out = {}
    for fs in ["A", "B", "C"]:
        for tag in ["full", "dedup"]:
            d = pd.read_csv(os.path.join(RES, f"preds_{tag}_{fs}_final.csv"))
            nmax = int(d.n_train.max())
            for n in [8000, nmax]:
                sub = d[d.n_train == n]
                y = sub.y.to_numpy(int)
                key = f"{fs}_{tag}_{n}"
                out[key] = {}
                for m in ["lr", "lgbm", "tabpfn"]:
                    col = "p_" + m
                    if col not in sub or sub[col].isna().all():
                        continue
                    p = sub[col].to_numpy(float)
                    out[key][m] = {
                        "auroc": float(roc_auc_score(y, p)),
```

```
                        "auprc": float(average_precision_score(y, p)),
                        "brier": float(brier_score_loss(y, p)),
                        "ci": ci(y, p, rng),
                    }
            print(fs, tag, "done")
    with open(os.path.join(OUT, "ci.json"), "w", encoding="utf-8") as f:
        json.dump(out, f, ensure_ascii=False, indent=1)
    print("saved", os.path.join(OUT, "ci.json"))


if __name__ == "__main__":
    main()
```

2.8 make_figures_restore.py（161 行）

用途

生成算法架构图 2（n = 8,000 的 ROC 与精确率–召回率曲线，full 队列）、图 7（同一曲线，dedup 队列）与图 5/6（特征重要性：LightGBM 总增益与 logistic 回归系数绝对值）。ROC/PR 一律调用 scikit-learn 的 roc_curve 与 precision_recall_curve，避免手写累积量出错。

- **对应算法架构：**A / B / C
- **输入：**results/preds_{full,dedup}_{A,B,C}_final.csv；results/restore/restore.json
- **输出：**results/figs/fig_roc_pr_{A,B,C}.png、fig_roc_pr_{A,B,C}_dedup.png、fig_importance_{A,B,C}.png

运行命令

```
py -3.12 make_figures_restore.py
```

口径提醒

- **注意：**三个模型固定配色：LR 蓝、LightGBM 橙、TabPFN 绿。
- **注意：**PR 面板必须用 sklearn 的 precision_recall_curve；早期手写版本把 recall 当 precision 除，纵轴会压到 0–0.005。
- **注意：**生成后把 PNG 复制到 papers/01_A|02_B|03_C/figs/，再由 build_docx5.py 嵌入。

代码全文

```
# -*- coding: utf-8 -*-
"""补画：去重队列 ROC/PR（n=8,000）与 A/B/C 特征重要性图。"""
import json
import os

import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
from sklearn.metrics import precision_recall_curve, roc_curve, roc_auc_score, average_precision_score

BASE = os.path.dirname(os.path.abspath(__file__))
RES = os.path.join(BASE, "results")
OUT = os.path.join(RES, "figs")
os.makedirs(OUT, exist_ok=True)

COL = {"lr": "#4C72B0", "lgbm": "#DD8452", "tabpfn": "#55A868"}
NAME = {"lr": "LR", "lgbm": "LGBM", "tabpfn": "TABPFN"}

SHORT = {
    "age": "Age", "gender": "Sex (value 1)", "nihss": "NIHSS",
    "uremia": "Uremia", "fatty_liver": "Fatty liver", "diabetes": "Diabetes",
    "hypertension": "Hypertension", "coronary_disease": "Coronary disease",
    "atrial_fibrillation": "Atrial fibrillation", "hyperuricemia": "Hyperuricemia",
    "hyperlipidaemia": "Hyperlipidemia", "frontal_lobe": "Frontal lobe",
    "parietal_lobe": "Parietal lobe", "temporal_lobe": "Temporal lobe",
    "occipital_lobe": "Occipital lobe", "insular_lobe": "Insular lobe",
    "range_lobe": "Any lobe", "basal_ganglia": "Basal ganglia",
    "capsula_interna": "Internal capsule", "brainstem": "Brainstem",
    "epencephalon": "Cerebellum", "paraventricular": "Paraventricular",
    "centrum_semiovale": "Centrum semiovale", "thalamus": "Thalamus",
    "subcortex_lobe": "Subcortical", "aca": "ACA territory", "mca": "MCA territory",
    "pca": "PCA territory", "va": "VA territory", "ba": "BA territory",
    "cca_plaque": "Common carotid plaque", "ica_plaque": "Internal carotid plaque",
    "eca_plaque": "External carotid plaque", "ant_circle": "Anterior circulation",
    "post_circle": "Posterior circulation", "large_ves_as": "Large-vessel disease",
    "plt": "Platelet count", "wbc": "WBC count", "rbc": "RBC count", "hba1c": "HbA1c",
    "crp": "CRP", "tg": "Triglycerides", "ldl": "LDL cholesterol", "hdl": "HDL cholesterol",
    "ast": "AST", "alt": "ALT", "bilirubin": "Total bilirubin", "albumin": "Albumin",
    "urea": "Urea", "creatinine": "Creatinine", "bua": "Uric acid",
    "pt": "Prothrombin time", "aptt": "APTT", "tt": "Thrombin time", "inr": "INR",
    "d_dimer": "D-dimer", "fibrinogen": "Fibrinogen", "ck": "Creatine kinase",
    "ck_mb": "CK-MB", "ldh": "LDH", "hbdh": "α-HBDH", "ima": "Ischemia-modified albumin",
    "na": "Sodium", "k": "Potassium", "cl": "Chloride", "ca": "Calcium",
    "p": "Phosphorus", "lactate": "Lactate", "anion_gap": "Anion gap", "tco2": "Total CO2",
    "dvt": "Deep vein thrombosis", "cerebral_hernia": "Cerebral hernia",
    "hydrocephalus": "Hydrocephalus", "hypoproteinemia": "Hypoproteinemia",
}
```

```
def roc_pr(fs, cohort):
    """cohort: 'full' or 'dedup'; draws sklearn-exact ROC and PR panels at n = 8,000."""
    d = pd.read_csv(os.path.join(RES, f"preds_{cohort}_{fs}_final.csv"))
    d = d[d.n_train == 8000]
    y = d.y.to_numpy(int)
    fig, ax = plt.subplots(1, 2, figsize=(13, 5.2), dpi=160)
    for m in ["lr", "lgbm", "tabpfn"]:
        p = d["p_" + m].to_numpy(float)
        fpr, tpr, _ = roc_curve(y, p)
        ax[0].plot(fpr, tpr, color=COL[m], lw=2.2,
                   label=f"{NAME[m]} AUROC {roc_auc_score(y, p):.3f}")
        pr, rc, _ = precision_recall_curve(y, p)
        ax[1].plot(rc, pr, color=COL[m], lw=2.2,
                   label=f"{NAME[m]} AUPRC {average_precision_score(y, p):.3f}")
    ax[0].plot([0, 1], [0, 1], "k--", lw=1)
    ax[0].set_xlabel("1 - Specificity", fontsize=12)
    ax[0].set_ylabel("Sensitivity", fontsize=12)
    ax[0].legend(loc="lower right", fontsize=9, frameon=False)
    ax[1].set_xlabel("Recall", fontsize=12)
    ax[1].set_ylabel("Precision", fontsize=12)
    ax[1].legend(loc="lower left", fontsize=9, frameon=False)
    ax[1].set_xlim(-0.02, 1.02)
    ax[1].set_ylim(-0.02, 1.02)
    tag = "full cohort" if cohort == "full" else "deduplicated cohort"
    fig.suptitle(f"Feature set {fs}, n_train = 8,000 ({tag})", fontsize=15)
    for a in ax:
        a.tick_params(labelsize=10)
    fig.tight_layout(rect=[0, 0, 1, 0.95])
    suffix = "" if cohort == "full" else "_dedup"
    p = os.path.join(OUT, f"fig_roc_pr_{fs}{suffix}.png")
    fig.savefig(p, bbox_inches="tight")
    plt.close(fig)
    print("saved", p)


def dedup_roc(fs):
    d = pd.read_csv(os.path.join(RES, f"preds_dedup_{fs}_final.csv"))
    d = d[d.n_train == 8000]
    y = d.y.to_numpy(int)
    fig, ax = plt.subplots(1, 2, figsize=(13, 5.2), dpi=160)
    for m in ["lr", "lgbm", "tabpfn"]:
        p = d["p_" + m].to_numpy(float)
        fpr, tpr, _ = roc_curve(y, p)
        ax[0].plot(fpr, tpr, color=COL[m], lw=2.2,
                   label=f"{NAME[m]} AUROC {roc_auc_score(y, p):.3f}")
        pr, rc, _ = precision_recall_curve(y, p)
        ax[1].plot(rc, pr, color=COL[m], lw=2.2,
                   label=f"{NAME[m]} AUPRC {average_precision_score(y, p):.3f}")
    ax[0].plot([0, 1], [0, 1], "k--", lw=1)
    ax[0].set_xlabel("1 - Specificity", fontsize=12)
    ax[0].set_ylabel("Sensitivity", fontsize=12)
    ax[0].legend(loc="lower right", fontsize=9, frameon=False)
    ax[1].set_xlabel("Recall", fontsize=12)
    ax[1].set_ylabel("Precision", fontsize=12)
    ax[1].legend(loc="upper right", fontsize=9, frameon=False)
    ax[1].set_xlim(-0.02, 1.02)
    fig.suptitle(f"Feature set {fs}, n_train = 8,000 (deduplicated cohort)", fontsize=15)
    for a in ax:
        a.tick_params(labelsize=10)
    fig.tight_layout(rect=[0, 0, 1, 0.95])
    p = os.path.join(OUT, f"fig_roc_pr_{fs}_dedup.png")
    fig.savefig(p, bbox_inches="tight")
    plt.close(fig)
    print("saved", p)


def importance(fs, data):
    lgbm = data["importance"][fs]["lgbm"][:12]
    lr = data["importance"][fs]["lr"][:12]
    fig, ax = plt.subplots(1, 2, figsize=(13.5, 6.4), dpi=160)
    # LGBM
    names = [SHORT.get(x["var"], x["var"]) for x in lgbm][::-1]
    vals = [x["gain_share"] * 100 for x in lgbm][::-1]
    ax[0].barh(names, vals, color=COL["lgbm"])
    ax[0].set_xlabel("Gain share (%)", fontsize=12)
    ax[0].set_title("LightGBM: total gain", fontsize=13)
    ax[0].tick_params(labelsize=9)
    for i, v in enumerate(vals):
        ax[0].text(v + max(vals) * 0.01, i, f"{v:.1f}", va="center", fontsize=8)
```

```
    ax[0].set_xlim(0, max(vals) * 1.15)
    # LR
    names = [SHORT.get(x["var"], x["var"]) for x in lr][::-1]
    vals = [x["abs_coef"] for x in lr][::-1]
    ax[1].barh(names, vals, color=COL["lr"])
    ax[1].set_xlabel("|Coefficient|", fontsize=12)
    ax[1].set_title("Logistic regression: absolute coefficient", fontsize=13)
    ax[1].tick_params(labelsize=9)
    for i, v in enumerate(vals):
        ax[1].text(v + max(vals) * 0.01, i, f"{v:.2f}", va="center", fontsize=8)
    ax[1].set_xlim(0, max(vals) * 1.15)
    n = data["importance"][fs]["n_features"]
    fig.suptitle(f"Feature set {fs}: feature importance (full training set, n = 17,167; {n} features)",
                 fontsize=14)
    fig.tight_layout(rect=[0, 0, 1, 0.95])
    p = os.path.join(OUT, f"fig_importance_{fs}.png")
    fig.savefig(p, bbox_inches="tight")
    plt.close(fig)
    print("saved", p)


def main():
    data = json.load(open(os.path.join(RES, "restore", "restore.json"), encoding="utf-8"))
    for fs in ["A", "B", "C"]:
        roc_pr(fs, "full")
        dedup_roc(fs)
        importance(fs, data)


if __name__ == "__main__":
    main()
```

3. D / E：温州 167 例真实队列

D/E 使用同一个单中心去标识队列（167 例，PSE 49 例，中位随访 592 天）。D 研究首次发作时点的预测，E 研究抗癫痫药物与晚期 PSE 的关联。所有分析在一个脚本内按 --task 分流，表 1 由独立脚本生成。

3.1 wenzhou_de.py（698 行）

用途

温州 167 例的 D、E 两个任务都在这一个脚本里。D：核心 8 变量与全集 24 变量，重复分层 5 折 × 10 次（种子 42），LR/LightGBM/TabPFN 三模型，输出 CV 指标、学习曲线（种子 42/43/44，n = 30–134）、校准、决策曲线、风险三分位 KM 与 log-rank、配对 bootstrap。E：ASM 与 PSE 的粗 OR、多变量校正 OR（bootstrap CI）、倾向评分作为协变量的 OR、IPTW/AIPW 稳定性检查、SMD 平衡、E 值、KM/RMST（730 天）、90 天界标、分期事件表、卒中类型分层。

- **对应算法架构：**D / E
- **输入：**Wenzhou_167_cases_deidentified.csv（167 例，SHA-256 见附录 B）
- **输出：**summary_D.json、summary_E.json；cv_summary_core/full.csv、cv_fold_metrics_*.csv、learning_curve_*.csv、dca.csv、E/smd_balance.csv；D/E 的 KM、ROC、校准、DCA、学习曲线图

运行命令

```
$PY wenzhou_de.py --data $DATA --out out/D --task D
$PY wenzhou_de.py --data $DATA --out out/E --task E
```

口径提醒

- **注意：**IPTW/AIPW 只作稳定性检查，不进入算法架构主估计。
- **注意：**E 算法架构不报告单一 Cox 风险比：比例风险假设不成立；脚本中的诊断函数不作为结论。
- **注意：**ASM 增量预测在核心 8 变量上做 50 次重复，逐次比较加/不加 ASM 的 AUROC。

代码全文

```
# -*- coding: utf-8 -*-
"""温州 167 例真实队列：D（首次发作→晚期非诱发性发作预测）与 E（服药/未服药分析）。

数据：Wenzhou_167_cases_deidentified.csv（167 例，全部为卒中后 7 天内急性症状性发作，
结局 PSE 49 例，随访中位 592 天）。

D 任务（预测）：两套特征集（core 8 变量，EPV≈6；full 25 变量）→ PSE；
  重复分层 5 折 CV（10 次重复），LR / LightGBM / TabPFN 三模型 + 学习曲线 +
  校准 + 决策曲线 + KM 风险分层。
E 任务（治疗）：ASM 服药(134) vs 未服药(33) 与 PSE 的关联；
  未校正 OR、多变量校正 OR、倾向评分 IPTW（交叉拟合）、SMD 平衡、E 值、分层分析。

用法:
  python wenzhou_de.py --data .../Wenzhou_167_cases_deidentified.csv --out out/D --task D
  python wenzhou_de.py --data ... --out out/E --task E
"""
import argparse
import json
import warnings
from pathlib import Path

import numpy as np
import pandas as pd
from scipy import stats
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import average_precision_score, brier_score_loss, roc_auc_score, roc_curve
from sklearn.model_selection import RepeatedStratifiedKFold, StratifiedKFold, train_test_split

warnings.filterwarnings("ignore")

SEED = 42
WEIGHTS = Path.home() / "TABPFN_BENCH_20261107/weights/tabpfn-v2-classifier-finetuned-zk73skhh.ckpt"

COLS = ["row", "stroke_type", "sex", "age", "age_lt65", "alcohol", "smoking",
        "hypertension", "diabetes", "hyperlipidemia", "hyperuricemia", "seizure_type",
        "interval_days", "interval_gt3", "multiple_seizures", "status_epilepticus",
        "eeg", "asm", "nihss", "nihss_gt14", "infection", "lesion_vol", "lesion_gt10",
        "cortical", "multilobar", "deep", "ventricular_ext", "surgery", "toast",
        "thrombolysis", "lipid_lowering", "antiplatelet", "anticoag", "pse",
        "followup_days", "late_seizure_days", "censor_days"]

D1 = ["age", "sex", "stroke_type", "smoking", "alcohol", "hypertension", "diabetes",
      "hyperlipidemia", "hyperuricemia", "sz_btc", "sz_focal", "interval_days",
      "interval_gt3", "multiple_seizures", "status_epilepticus", "eeg_epileptic",
      "eeg_nonspecific", "eeg_absent", "nihss", "infection", "log_lesion_vol",
      "cortical", "multilobar", "deep"]
# 精简核心集：按 EPV（49 事件 / 8 变量 ≈ 6）预先设定的临床变量
CORE = ["age", "stroke_type", "interval_gt3", "multiple_seizures",
```

```
        "status_epilepticus", "nihss", "cortical", "eeg_epileptic"]
FEATURE_SETS = {"core": CORE, "full": D1}
E_COV = ["age", "sex", "stroke_type", "nihss", "status_epilepticus",
         "multiple_seizures", "log_lesion_vol", "cortical", "eeg_epileptic"]


def load(path):
    df = pd.read_csv(path)
    assert df.shape[1] == 37, df.shape
    df.columns = COLS
    df["sz_btc"] = (df.seizure_type == 1).astype(float)
    df["sz_focal"] = (df.seizure_type == 2).astype(float)
    df["eeg_epileptic"] = (df.eeg == 2).astype(float)
    df["eeg_nonspecific"] = (df.eeg == 1).astype(float)
    df["eeg_absent"] = (df.eeg == 3).astype(float)
    df["log_lesion_vol"] = np.log1p(df.lesion_vol)
    # 事件时间：PSE 用首次晚期发作天数，删失用删失天数（缺失时退回随访天数）
    df["time"] = np.where(df.pse == 1, df.late_seizure_days,
                          df.censor_days.fillna(df.followup_days))
    return df


CONT = ["age", "nihss", "interval_days", "log_lesion_vol", "lesion_vol"]


def prep_lr(Xtr, Xte, cols):
    """与 D13 场景口径一致：连续列 winsorize+标准化，二值列保持原值。"""
    Xa, Xb = Xtr.copy(), Xte.copy()
    for j, c in enumerate(cols):
        if c in CONT:
            lo = np.percentile(Xtr[:, j], 1)
            hi = np.percentile(Xtr[:, j], 99)
            Xa[:, j] = np.clip(Xa[:, j], lo, hi)
            Xb[:, j] = np.clip(Xb[:, j], lo, hi)
            mu, sd = Xa[:, j].mean(), Xa[:, j].std()
            sd = sd if sd > 1e-12 else 1.0
            Xa[:, j] = (Xa[:, j] - mu) / sd
            Xb[:, j] = (Xb[:, j] - mu) / sd
    return Xa, Xb


def make_models(seed=SEED, tabpfn=True):
    from lightgbm import LGBMClassifier
    ms = {
        "lr": LogisticRegression(C=1.0, max_iter=5000, random_state=seed),
        # 小样本配置：浅树（3 叶）+ 较少树数，避免在 n~150 上过拟合
        "lgbm": LGBMClassifier(n_estimators=100, learning_rate=0.05, num_leaves=3,
                               min_child_samples=10, random_state=seed,
                               n_jobs=8, verbosity=-1),
    }
    if tabpfn:
        from tabpfn import TabPFNClassifier
        ms["tabpfn"] = TabPFNClassifier(model_path=str(WEIGHTS), n_estimators="auto",
                                        random_state=seed, ignore_pretraining_limits=True,
                                        device="cpu")
    return ms


def fit_predict(name, model, Xtr, ytr, Xte, cols=None):
    if name == "lr":
        a, b = prep_lr(Xtr, Xte, cols)
        model.fit(a, ytr)
        return model.predict_proba(b)[:, 1]
    if name == "lgbm":
        spw = (ytr == 0).sum() / max(1, (ytr == 1).sum())
        model.set_params(scale_pos_weight=float(spw))
    model.fit(Xtr, ytr)
    return model.predict_proba(Xte)[:, 1]


def cv_oof(df, cols, n_splits=5, n_repeats=10, tabpfn=True):
    X = df[cols].to_numpy(float)
    y = df.pse.to_numpy(int)
    rskf = RepeatedStratifiedKFold(n_splits=n_splits, n_repeats=n_repeats, random_state=SEED)
    names = ["lr", "lgbm", "tabpfn"] if tabpfn else ["lr", "lgbm"]
    oof = {m: np.zeros((n_repeats, len(y))) for m in names}
    for i, (tr, te) in enumerate(rskf.split(X, y)):
        rep = i // n_splits
        models = make_models(SEED + i, tabpfn=tabpfn)
```

```
        for name, model in models.items():
            oof[name][rep, te] = fit_predict(name, model, X[tr], y[tr], X[te], cols)
    return y, oof


def cv_metrics(y, oof):
    rows = []
    for name, P in oof.items():
        for rep in range(P.shape[0]):
            p = P[rep]
            row = {"model": name, "repeat": rep,
                   "auroc": roc_auc_score(y, p),
                   "auprc": average_precision_score(y, p),
                   "brier": brier_score_loss(y, p)}
            lp = np.log(np.clip(p, 1e-6, 1 - 1e-6) / (1 - np.clip(p, 1e-6, 1 - 1e-6)))
            cs = LogisticRegression(C=1e6, max_iter=1000).fit(lp.reshape(-1, 1), y)
            row["cal_slope"] = float(cs.coef_[0, 0])
            rows.append(row)
    m = pd.DataFrame(rows)
    summ = m.groupby("model").agg(["mean", "std"]).round(4)
    return m, summ


def paired_delta(y, p1, p2, n_boot=2000, seed=SEED):
    """配对 bootstrap：ΔAUROC 的 95% CI（同一批患者）。"""
    rng = np.random.default_rng(seed)
    d = []
    for _ in range(n_boot):
        idx = rng.integers(0, len(y), len(y))
        if y[idx].min() == y[idx].max():
            continue
        d.append(roc_auc_score(y[idx], p1[idx]) - roc_auc_score(y[idx], p2[idx]))
    d = np.array(d)
    return float(np.mean(d)), float(np.percentile(d, 2.5)), float(np.percentile(d, 97.5))


def km(t, e):
    t = np.asarray(t, float)
    e = np.asarray(e, int)
    times, surv = [0.0], [1.0]
    s = 1.0
    for u in np.unique(t[e == 1]):
        at_risk = (t >= u).sum()
        d = ((t == u) & (e == 1)).sum()
        if at_risk > 0:
            s *= 1 - d / at_risk
            times.append(float(u))
            surv.append(float(s))
    return np.array(times), np.array(surv)


def km_at(times, surv, q):
    idx = np.searchsorted(times, q, side="right") - 1
    return float(surv[max(idx, 0)])


def logrank_k(t, e, g):
    """k 组 log-rank 检验，返回 (chi2, p, df)。"""
    t, e, g = np.asarray(t, float), np.asarray(e, int), np.asarray(g)
    groups = np.unique(g)
    if len(groups) < 2:
        return float("nan"), float("nan"), 0
    O = np.zeros(len(groups)); E = np.zeros(len(groups))
    V = np.zeros((len(groups), len(groups)))
    for u in np.unique(t[e == 1]):
        at_risk = t >= u
        n = at_risk.sum()
        if n <= 1:
            continue
        d = ((t == u) & (e == 1)).sum()
        nk = np.array([(at_risk & (g == k)).sum() for k in groups], float)
        dk = np.array([((t == u) & (e == 1) & (g == k)).sum() for k in groups], float)
        O += dk
        E += d * nk / n
        pp = nk / n
        V += d * (n - d) / (n - 1) * (np.diag(pp) - np.outer(pp, pp))
    O_, E_, V_ = O[:-1], E[:-1], V[:-1, :-1]
    chi2 = float((O_ - E_) @ np.linalg.pinv(V_) @ (O_ - E_))
    df = len(groups) - 1
```

```
    return chi2, float(stats.chi2.sf(chi2, df)), df


def cox_binary(t, e, g):
    """单变量 Cox（二分类/连续协变量均可），返回 HR。"""
    t, e, g = np.asarray(t, float), np.asarray(e, int), np.asarray(g, float)

    def negll(b):
        b = float(b)
        ll = 0.0
        for u in np.unique(t[e == 1]):
            risk = t >= u
            xb = b * g[risk]
            mx = xb.max()
            ll += b * g[(t == u) & (e == 1)].sum() - (np.log(np.exp(xb - mx).sum()) + mx)
        return -ll

    from scipy.optimize import minimize_scalar
    r = minimize_scalar(negll, bounds=(-4, 4), method="bounded")
    return float(np.exp(r.x))


def rmst(t, e, tau):
    """限制平均生存时间（0..tau 天内 PSE-free 天数），由 KM 曲线积分。"""
    tt, ss = km(t, e)
    area = 0.0
    for i in range(1, len(tt)):
        a, b = tt[i - 1], min(tt[i], tau)
        if b > a:
            area += ss[i - 1] * (b - a)
        if tt[i] >= tau:
            break
    if tt[-1] < tau:
        area += ss[-1] * (tau - tt[-1])
    return float(area)


def e_value(orv):
    """E 值（二分类结局 OR），衡量未测量混杂需多强才能解释掉该关联。"""
    if orv < 1:
        orv = 1 / orv
    return float(orv + np.sqrt(orv * (orv - 1)))


def plot_all(out, D=None, E=None):
    import matplotlib
    matplotlib.use("Agg")
    import matplotlib.pyplot as plt

    if D:
        y, oof, df = D["y"], D["oof"], D["df"]
        fig, ax = plt.subplots(figsize=(5, 5))
        for name, color in [("lr", "#4C72B0"), ("lgbm", "#DD8452"), ("tabpfn", "#55A868")]:
            if name not in oof:
                continue
            p = oof[name].mean(0)
            fpr, tpr, _ = roc_curve(y, p)
            ax.plot(fpr, tpr, color=color, label=f"{name.upper()} (AUROC {roc_auc_score(y, p):.3f})")
        ax.plot([0, 1], [0, 1], "k--", lw=0.8)
        ax.set_xlabel("1 - Specificity"); ax.set_ylabel("Sensitivity")
        ax.legend(frameon=False, fontsize=8); ax.set_title("ROC (mean of 10 repeats, core model)")
        fig.tight_layout(); fig.savefig(out / "fig_roc.png", dpi=200); plt.close(fig)

        fig, ax = plt.subplots(figsize=(5, 5))
        for name, color in [("lr", "#4C72B0"), ("lgbm", "#DD8452"), ("tabpfn", "#55A868")]:
            if name not in oof:
                continue
            p = oof[name].mean(0)
            bins = np.quantile(p, np.linspace(0, 1, 6))
            idx = np.clip(np.searchsorted(bins, p, side="right") - 1, 0, 4)
            xs, ys = [], []
            for b in range(5):
                m = idx == b
                if m.sum() > 3:
                    xs.append(p[m].mean()); ys.append(y[m].mean())
            ax.plot(xs, ys, "o-", color=color, label=name.upper(), ms=4)
        ax.plot([0, 0.8], [0, 0.8], "k--", lw=0.8)
        ax.set_xlabel("Predicted risk (quintiles)"); ax.set_ylabel("Observed rate")
        ax.legend(frameon=False, fontsize=8); ax.set_title("Calibration (core model)")
```

```
        fig.tight_layout(); fig.savefig(out / "fig_calibration.png", dpi=200); plt.close(fig)

        fig, ax = plt.subplots(figsize=(5, 5))
        th = np.arange(0.05, 0.61, 0.01)
        prev = y.mean()
        for name, color in [("lr", "#4C72B0"), ("lgbm", "#DD8452"), ("tabpfn", "#55A868")]:
            if name not in oof:
                continue
            p = oof[name].mean(0)
            nb = [((p >= t) & (y == 1)).sum() / len(y) - ((p >= t) & (y == 0)).sum() / len(y) * t / (1 - t) for t in th]
            ax.plot(th, nb, color=color, label=name.upper())
        ax.plot(th, [prev - (1 - prev) * t / (1 - t) for t in th], "k--", lw=0.8, label="Treat all")
        ax.axhline(0, color="gray", lw=0.6)
        ax.set_ylim(-0.05, max(0.45, prev * 1.1))
        ax.set_xlabel("Threshold"); ax.set_ylabel("Net benefit")
        ax.legend(frameon=False, fontsize=8); ax.set_title("Decision curve (core model)")
        fig.tight_layout(); fig.savefig(out / "fig_dca.png", dpi=200); plt.close(fig)

        lc = D.get("learning_curve")
        if lc is not None and len(lc):
            fig, ax = plt.subplots(figsize=(5.5, 4))
            for name, color in [("lr", "#4C72B0"), ("lgbm", "#DD8452"), ("tabpfn", "#55A868")]:
                sub = lc[lc.model == name]
                if len(sub) == 0:
                    continue
                ax.errorbar(sub.n_train, sub.auroc_mean, yerr=sub.auroc_sd, marker="o",
                            color=color, capsize=3, label=name.upper(), ms=4)
            ax.axhline(0.5, color="gray", lw=0.6, ls=":")
            ax.set_xlabel("Training size (n)"); ax.set_ylabel("AUROC (fixed split)")
            ax.legend(frameon=False, fontsize=8); ax.set_title("Learning curve (core model)")
            fig.tight_layout(); fig.savefig(out / "fig_learning_curve.png", dpi=200); plt.close(fig)

        for nm, fn in [("lr", "fig_km_lr.png"), ("tabpfn", "fig_km_tabpfn.png")]:
            if nm not in oof:
                continue
            p = oof[nm].mean(0)
            q = np.quantile(p, [1 / 3, 2 / 3])
            grp = np.digitize(p, q)
            fig, ax = plt.subplots(figsize=(5.5, 4))
            for gi, lab, color in [(0, "Low risk", "#4C72B0"), (1, "Intermediate", "#DD8452"), (2, "High risk",
"#C44E52")]:
                m = grp == gi
                tt, ss = km(df.time[m], df.pse[m])
                ax.step(tt, ss, where="post", color=color, label=f"{lab} (n={m.sum()})")
            ax.set_xlabel("Days since stroke"); ax.set_ylabel("PSE-free probability")
            ax.set_xlim(0, 1500); ax.legend(frameon=False, fontsize=8)
            ax.set_title(f"Kaplan-Meier by {nm.upper()} risk tertile")
            fig.tight_layout(); fig.savefig(out / fn, dpi=200); plt.close(fig)

    if E:
        smd = E["smd"]
        if len(smd):
            fig, ax = plt.subplots(figsize=(5, 4))
            yy = np.arange(len(smd))
            ax.scatter(smd.smd_before, yy, color="#C44E52", label="Before weighting", s=22)
            ax.scatter(smd.smd_after, yy, color="#55A868", label="After IPTW", s=22)
            ax.axvline(0.1, color="gray", ls="--", lw=0.8)
            ax.axvline(-0.1, color="gray", ls="--", lw=0.8)
            ax.set_yticks(yy); ax.set_yticklabels(smd.variable, fontsize=8)
            ax.set_xlabel("Standardized mean difference")
            ax.legend(frameon=False, fontsize=8); ax.set_title("Covariate balance")
            fig.tight_layout(); fig.savefig(out / "fig_love.png", dpi=200); plt.close(fig)
        df = E["df"]
        fig, ax = plt.subplots(figsize=(5.5, 4))
        for g, lab, color in [(0, "No ASM (n=%d)" % (df.asm == 0).sum(), "#C44E52"),
                              (1, "ASM (n=%d)" % (df.asm == 1).sum(), "#4C72B0")]:
            m = df.asm == g
            tt, ss = km(df.time[m], df.pse[m])
            ax.step(tt, ss, where="post", color=color, label=lab)
        ax.set_xlabel("Days since stroke"); ax.set_ylabel("PSE-free probability")
        ax.set_xlim(0, 1500); ax.legend(frameon=False, fontsize=8)
        ax.set_title("Kaplan-Meier by ASM status (unadjusted)")
        fig.tight_layout(); fig.savefig(out / "fig_km_asm.png", dpi=200); plt.close(fig)


def learning_curve(df, cols, tabpfn=True, seeds=(42, 43, 44)):
    X = df[cols].to_numpy(float); yv = df.pse.to_numpy(int)
    rows = []
    sizes = [30, 50, 80, 110, int(round(0.8 * len(yv)))]
```

```
    for seed in seeds:
        Xtr, Xte, ytr, yte = train_test_split(X, yv, test_size=0.2, stratify=yv, random_state=seed)
        rng = np.random.default_rng(seed)
        pos = rng.permutation(np.where(ytr == 1)[0]); neg = rng.permutation(np.where(ytr == 0)[0])
        for n in sizes:
            p_n = min(max(int(round(n * len(pos) / len(ytr))), 1), len(pos))
            q_n = min(max(n - p_n, 1), len(neg))
            idx = np.concatenate([pos[:p_n], neg[:q_n]])
            models = make_models(SEED + n + seed, tabpfn=tabpfn)
            for name, model in models.items():
                p = fit_predict(name, model, Xtr[idx], ytr[idx], Xte, cols)
                rows.append({"seed": seed, "n_train": n, "model": name,
                             "auroc": roc_auc_score(yte, p)})
    lc = pd.DataFrame(rows)
    lc_sum = lc.groupby(["model", "n_train"]).auroc.agg(["mean", "std"]).reset_index()
    lc_sum.columns = ["model", "n_train", "auroc_mean", "auroc_sd"]
    return lc, lc_sum


def task_d(df, out, tabpfn=True):
    out.mkdir(parents=True, exist_ok=True)
    res, stores = {}, {}
    for key, cols in FEATURE_SETS.items():
        y, oof = cv_oof(df, cols, tabpfn=tabpfn)
        m, summ = cv_metrics(y, oof)
        m.to_csv(out / f"cv_fold_metrics_{key}.csv", index=False)
        summ.to_csv(out / f"cv_summary_{key}.csv")
        stores[key] = (y, oof, cols)
        res[key] = {"n_features": len(cols), "features": cols,
                    "metrics_mean_std": json.loads(summ.round(4).to_json())}
        lc, lc_sum = learning_curve(df, cols, tabpfn=tabpfn)
        lc.to_csv(out / f"learning_curve_raw_{key}.csv", index=False)
        lc_sum.to_csv(out / f"learning_curve_{key}.csv", index=False)
        res[key]["learning_curve"] = json.loads(lc_sum.round(4).to_json(orient="records"))

    # 主分析 = core（EPV 预先设定）
    y, oof, cols = stores["core"]
    deltas = {}
    if "tabpfn" in oof:
        deltas["tabpfn_minus_lgbm"] = paired_delta(y, oof["tabpfn"][0], oof["lgbm"][0])
        deltas["tabpfn_minus_lr"] = paired_delta(y, oof["tabpfn"][0], oof["lr"][0])
        deltas["lr_minus_lgbm"] = paired_delta(y, oof["lr"][0], oof["lgbm"][0])

    # 增量值：core vs core+ASM
    y2, oof2 = cv_oof(df, CORE + ["asm"], tabpfn=tabpfn)
    inc = {}
    for name in oof2:
        a1 = np.array([roc_auc_score(y2, oof2[name][r]) for r in range(oof2[name].shape[0])])
        b1 = np.array([roc_auc_score(y, oof[name][r]) for r in range(oof[name].shape[0])])
        inc[name] = {"auroc_without_asm": float(b1.mean()), "auroc_with_asm": float(a1.mean()),
                     "delta_mean": float((a1 - b1).mean()), "n_repeats_delta_gt0": int((a1 > b1).sum())}

    # KM 风险三分位 + log-rank（2 组自由度），对 LR 与 TabPFN 各做一次
    kmres = {}
    for strat_model in ["lr", "tabpfn"]:
        if strat_model not in oof:
            continue
        p = oof[strat_model].mean(0)
        q = np.quantile(p, [1 / 3, 2 / 3])
        grp = np.digitize(p, q)
        entry = {}
        for g in [0, 1, 2]:
            tt, ss = km(df.time[grp == g], df.pse[grp == g])
            entry[f"tertile{g}"] = {"n": int((grp == g).sum()),
                                    "events": int(df.pse[grp == g].sum()),
                                    "pse_free_2y": round(km_at(tt, ss, 730), 3)}
        chi2, pval, dfree = logrank_k(df.time, df.pse, grp)
        entry.update({"logrank_chi2": chi2, "logrank_p": pval, "logrank_df": dfree})
        kmres[strat_model] = entry

    th = np.arange(0.05, 0.61, 0.01)
    dca = []
    for name in oof:
        p = oof[name].mean(0)
        for t in th:
            nb = ((p >= t) & (y == 1)).sum() / len(y) - ((p >= t) & (y == 0)).sum() / len(y) * t / (1 - t)
            dca.append({"model": name, "threshold": round(float(t), 2), "net_benefit": float(nb)})
    pd.DataFrame(dca).to_csv(out / "dca.csv", index=False)
    pd.DataFrame({"y": y, **{f"p_{k}": v.mean(0) for k, v in oof.items()}}).to_csv(
```

```
        out / "oof_predictions_core.csv", index=False)

    summary = {
        "cohort": {"n": int(len(df)), "events": int(df.pse.sum()),
                   "prevalence": float(df.pse.mean()),
                   "median_followup_days": float(df.followup_days.median()),
                   "median_days_to_pse": float(df.loc[df.pse == 1, "late_seizure_days"].median())},
        "cv": {"folds": 5, "repeats": 10},
        "primary_feature_set": "core",
        "feature_sets": res,
        "paired_bootstrap_delta_auroc_core": deltas,
        "asm_incremental_core": inc,
        "km_risk_tertiles": kmres,
    }
    (out / "summary_D.json").write_text(json.dumps(summary, ensure_ascii=False, indent=2), encoding="utf-8")
    lc_core = pd.read_csv(out / "learning_curve_core.csv")
    plot_all(out, {"y": y, "oof": oof, "df": df, "learning_curve": lc_core})
    return summary


def task_e(df, out, tabpfn=True):
    out.mkdir(parents=True, exist_ok=True)
    y = df.pse.to_numpy(int)
    a = df.asm.to_numpy(int)
    rng = np.random.default_rng(SEED)

    def or_ci(yy, aa, n_boot=3000):
        def orr(y_, a_):
            t = ((a_ == 1) & (y_ == 1)).sum(); tn = ((a_ == 1) & (y_ == 0)).sum()
            c = ((a_ == 0) & (y_ == 1)).sum(); cn = ((a_ == 0) & (y_ == 0)).sum()
            if min(t, tn, c, cn) == 0:
                return np.nan
            return (t * cn) / (tn * c)
        est = orr(yy, aa)
        bs = []
        for _ in range(n_boot):
            i = rng.integers(0, len(yy), len(yy))
            bs.append(orr(yy[i], aa[i]))
        bs = np.array([b for b in bs if np.isfinite(b)])
        return float(est), float(np.percentile(bs, 2.5)), float(np.percentile(bs, 97.5))

    crude = {
        "n_asm": int((a == 1).sum()), "events_asm": int(((a == 1) & (y == 1)).sum()),
        "n_noasm": int((a == 0).sum()), "events_noasm": int(((a == 0) & (y == 1)).sum()),
        "risk_asm": float(y[a == 1].mean()), "risk_noasm": float(y[a == 0].mean()),
    }
    crude["risk_diff"] = crude["risk_asm"] - crude["risk_noasm"]
    crude["or"], crude["or_lo"], crude["or_hi"] = or_ci(y, a)
    crude["e_value"] = e_value(crude["or"])

    # 多变量校正（bootstrap 系数 CI；ASM 保持 0/1 原尺度）
    X = df[E_COV + ["asm"]].to_numpy(float)

    def fit_adj(idx):
        Xa, _ = prep_lr(X[idx][:, :-1], X[idx][:, :-1], E_COV)
        Xall = np.column_stack([Xa, X[idx][:, -1]])
        lr = LogisticRegression(C=1.0, max_iter=5000).fit(Xall, y[idx])
        return lr.coef_[0, -1]

    beta = fit_adj(np.arange(len(y)))
    bs = []
    for _ in range(2000):
        i = rng.integers(0, len(y), len(y))
        if y[i].sum() < 5 or len(np.unique(y[i])) < 2:
            continue
        try:
            bs.append(fit_adj(i))
        except Exception:
            pass
    adj_or = float(np.exp(beta))
    adj = {"or": adj_or,
           "or_lo": float(np.exp(np.percentile(bs, 2.5))),
           "or_hi": float(np.exp(np.percentile(bs, 97.5))),
           "n_boot_ok": len(bs), "e_value": e_value(adj_or),
           "covariates": E_COV}

    # 倾向评分 IPTW（交叉拟合：每折用训练折拟合，避免同数据过拟合）
    psX = df[E_COV].to_numpy(float)
    ps = np.zeros(len(y))
```

```
for tr, te in StratifiedKFold(n_splits=5, shuffle=True, random_state=SEED).split(psX, a):
    Xa, Xb = prep_lr(psX[tr], psX[te], E_COV)
    m = LogisticRegression(C=1.0, max_iter=5000).fit(Xa, a[tr])
    ps[te] = m.predict_proba(Xb)[:, 1]
ps_auc = float(roc_auc_score(a, ps))
lo, hi = np.percentile(ps, [1, 99])
psc = np.clip(ps, lo, hi)
w = np.where(a == 1, 1 / psc, 1 / (1 - psc))
w = np.where(a == 1, w * a.mean(), w * (1 - a.mean()))  # 稳定权重
w = w / w.mean()

def smd(x, aa, ww=None):
    if ww is None:
        m1, m0 = x[aa == 1].mean(), x[aa == 0].mean()
        v1, v0 = x[aa == 1].var(ddof=1), x[aa == 0].var(ddof=1)
    else:
        m1 = np.average(x[aa == 1], weights=ww[aa == 1])
        m0 = np.average(x[aa == 0], weights=ww[aa == 0])
        v1 = np.average((x[aa == 1] - m1) ** 2, weights=ww[aa == 1])
        v0 = np.average((x[aa == 0] - m0) ** 2, weights=ww[aa == 0])
    sp = np.sqrt((v1 + v0) / 2)
    return float((m1 - m0) / sp) if sp > 0 else 0.0

smd_rows = [{"variable": c, "smd_before": smd(df[c].to_numpy(float), a),
             "smd_after": smd(df[c].to_numpy(float), a, w)} for c in E_COV]
smd = pd.DataFrame(smd_rows)
smd.to_csv(out / "smd_balance.csv", index=False)

r1 = np.average(y[a == 1], weights=w[a == 1])
r0 = np.average(y[a == 0], weights=w[a == 0])
rd = r1 - r0
bs_rd = []
for _ in range(3000):
    i = rng.integers(0, len(y), len(y))
    aa, yy, ww = a[i], y[i], w[i]
    if (aa == 1).sum() < 3 or (aa == 0).sum() < 3:
        continue
    bs_rd.append(np.average(yy[aa == 1], weights=ww[aa == 1])
                 - np.average(yy[aa == 0], weights=ww[aa == 0]))
iptw = {"risk_asm_weighted": float(r1), "risk_noasm_weighted": float(r0),
        "risk_diff": float(rd), "rd_lo": float(np.percentile(bs_rd, 2.5)),
        "rd_hi": float(np.percentile(bs_rd, 97.5)),
        "ps_auc": ps_auc, "ps_min": float(ps.min()), "ps_max": float(ps.max()),
        "ps_median_asm": float(np.median(ps[a == 1])),
        "ps_median_noasm": float(np.median(ps[a == 0])),
        "max_abs_smd_before": float(smd.smd_before.abs().max()),
        "max_abs_smd_after": float(smd.smd_after.abs().max())}

# PS 作为协变量校正（比 IPTW 稳定，尤其当 PS 判别力弱时）
logit_ps = np.log(np.clip(ps, 1e-6, 1 - 1e-6) / (1 - np.clip(ps, 1e-6, 1 - 1e-6)))

def fit_ps_adj(idx):
    Z = np.column_stack([a[idx], logit_ps[idx]])
    lr = LogisticRegression(C=1.0, max_iter=5000).fit(Z, y[idx])
    return lr.coef_[0, 0]

b_ps = fit_ps_adj(np.arange(len(y)))
bs_ps = []
for _ in range(2000):
    i = rng.integers(0, len(y), len(y))
    if y[i].sum() < 5 or len(np.unique(a[i])) < 2:
        continue
    try:
        bs_ps.append(fit_ps_adj(i))
    except Exception:
        pass
ps_adj = {"or": float(np.exp(b_ps)),
          "or_lo": float(np.exp(np.percentile(bs_ps, 2.5))),
          "or_hi": float(np.exp(np.percentile(bs_ps, 97.5))),
          "n_boot_ok": len(bs_ps)}

# AIPW（双重稳健估计）：交叉拟合倾向评分 + 结局模型
mu1 = np.zeros(len(y)); mu0 = np.zeros(len(y)); ps2 = np.zeros(len(y))
for tr, te in StratifiedKFold(n_splits=5, shuffle=True, random_state=SEED).split(psX, a):
    Xa, Xb = prep_lr(psX[tr], psX[te], E_COV)
    om = LogisticRegression(C=1.0, max_iter=5000).fit(np.column_stack([Xa, a[tr]]), y[tr])
    mu1[te] = om.predict_proba(np.column_stack([Xb, np.ones(len(te))]))[:, 1]
    mu0[te] = om.predict_proba(np.column_stack([Xb, np.zeros(len(te))]))[:, 1]
    ps2[te] = LogisticRegression(C=1.0, max_iter=5000).fit(Xa, a[tr]).predict_proba(Xb)[:, 1]
```

```
    p2 = np.clip(ps2, 0.01, 0.99)
    aipw_i = (a / p2) * (y - mu1) + mu1 - ((1 - a) / (1 - p2)) * (y - mu0) - mu0
    ate = float(aipw_i.mean())
    bs_ate = [float(aipw_i[rng.integers(0, len(y), len(y))].mean()) for _ in range(3000)]
    aipw = {"ate_risk_diff": ate, "rd_lo": float(np.percentile(bs_ate, 2.5)),
            "rd_hi": float(np.percentile(bs_ate, 97.5)),
            "ps_auc": float(roc_auc_score(a, ps2))}

    # 生存分析：log-rank + RMST（比例风险假设不成立，不做单一 Cox HR）
    chi2, pval, dfree = logrank_k(df.time, df.pse, a)
    tt1, ss1 = km(df.time[a == 1], df.pse[a == 1])
    tt0, ss0 = km(df.time[a == 0], df.pse[a == 0])
    rm1, rm0 = rmst(df.time[a == 1], df.pse[a == 1], 730), rmst(df.time[a == 0], df.pse[a == 0], 730)
    bs_rm = []
    for _ in range(2000):
        i = rng.integers(0, len(y), len(y))
        try:
            bs_rm.append(rmst(df.time[i][a[i] == 1], df.pse[i][a[i] == 1], 730)
                         - rmst(df.time[i][a[i] == 0], df.pse[i][a[i] == 0], 730))
        except Exception:
            pass
    # 分期事件数（检验比例风险：早期事件是否集中在治疗组）
    epoch = {}
    for lab, m0_, m1_ in [("0-90d", 0, 90), ("91-365d", 90, 365), (">365d", 365, 10 ** 6)]:
        epoch[lab] = {g: int(((a == g) & (y == 1) & (df.time > m0_) & (df.time <= m1_)).sum())
                      for g in [0, 1]}
    # 90 天 landmark 分析：排除 90 天内事件后重新比较
    lm = df.time > 90
    yy_l, aa_l = y[lm.to_numpy()], a[lm.to_numpy()]
    lm_or = or_ci(yy_l, aa_l, n_boot=1500)
    landmark = {"n_at_risk": int(lm.sum()), "n_asm": int((aa_l == 1).sum()),
                "n_noasm": int((aa_l == 0).sum()),
                "events_asm": int(((aa_l == 1) & (yy_l == 1)).sum()),
                "events_noasm": int(((aa_l == 0) & (yy_l == 1)).sum()),
                "or": lm_or[0], "or_lo": lm_or[1], "or_hi": lm_or[2]}
    surv = {"logrank_chi2": chi2, "logrank_p": pval, "logrank_df": dfree,
            "pse_free_2y_asm": km_at(tt1, ss1, 730),
            "pse_free_2y_noasm": km_at(tt0, ss0, 730),
            "rmst_730_asm": rm1, "rmst_730_noasm": rm0,
            "rmst_diff": rm1 - rm0,
            "rmst_diff_lo": float(np.percentile(bs_rm, 2.5)),
            "rmst_diff_hi": float(np.percentile(bs_rm, 97.5)),
            "events_by_epoch": epoch, "landmark_90d": landmark,
            "ph_note": "比例风险假设不成立（早期事件集中于治疗组），不报告单一 Cox HR"}

    strata = {}
    for st, lab in [(0, "ICH"), (1, "ischemic")]:
        m = (df.stroke_type == st).to_numpy()
        yy, aa = y[m], a[m]
        o, lo_, hi_ = or_ci(yy, aa, n_boot=1500)
        strata[lab] = {"n": int(m.sum()), "n_asm": int((aa == 1).sum()),
                       "n_noasm": int((aa == 0).sum()),
                       "risk_asm": float(yy[aa == 1].mean()),
                       "risk_noasm": float(yy[aa == 0].mean()),
                       "or": o, "or_lo": lo_, "or_hi": hi_}

    ycv, oof = cv_oof(df, CORE, tabpfn=tabpfn)
    ycv2, oof2 = cv_oof(df, CORE + ["asm"], tabpfn=tabpfn)
    inc = {}
    for name in oof:
        a1 = np.array([roc_auc_score(ycv2, oof2[name][r]) for r in range(oof2[name].shape[0])])
        b1 = np.array([roc_auc_score(ycv, oof[name][r]) for r in range(oof[name].shape[0])])
        inc[name] = {"auroc_without_asm": float(b1.mean()), "auroc_with_asm": float(a1.mean()),
                     "delta_mean": float((a1 - b1).mean()),
                     "n_repeats_delta_gt0": int((a1 > b1).sum())}

    summary = {"crude": crude, "adjusted": adj, "ps_adjusted": ps_adj,
               "iptw": iptw, "aipw": aipw,
               "survival": surv, "stroke_type_strata": strata,
               "asm_incremental_prediction": inc, "covariates": E_COV,
               "iptw_note": "PS 判别力弱（c≈0.53），IPTW 估计不稳定，主报告为多变量校正 OR 与 AIPW"}
    (out / "summary_E.json").write_text(json.dumps(summary, ensure_ascii=False, indent=2), encoding="utf-8")
    pd.DataFrame({"asm": a, "pse": y, "time": df.time, "ps": ps, "w": w}).to_csv(
        out / "iptw_patient_level.csv", index=False)
    plot_all(out, None, {"smd": smd, "df": df})
    return summary


def main():
```

```
    ap = argparse.ArgumentParser()
    ap.add_argument("--data", required=True)
    ap.add_argument("--out", required=True)
    ap.add_argument("--task", required=True, choices=["D", "E"])
    ap.add_argument("--no-tabpfn", action="store_true")
    args = ap.parse_args()
    out = Path(args.out)
    df = load(args.data)
    print(f"loaded n={len(df)} pse={int(df.pse.sum())} asm={int(df.asm.sum())}", flush=True)
    if args.task == "D":
        s = task_d(df, out, tabpfn=not args.no_tabpfn)
    else:
        s = task_e(df, out, tabpfn=not args.no_tabpfn)
    print(json.dumps(s, ensure_ascii=False, indent=1)[:5000], flush=True)
    print("DONE", out, flush=True)


if __name__ == "__main__":
    main()
```

3.2 table1_wenzhou.py（72 行）

用途

计算 D、E 算法架构表 1：同一批 167 例，分别按 PSE 结局（D）和 ASM 用药（E）分组，输出总体、by_pse、by_asm 三套描述统计与末次发作时间分布。结果 JSON 打到标准输出。

- **对应算法架构：**D / E
- **输入：**Wenzhou_167_cases_deidentified.csv
- **输出：**标准输出 → results/table1_wenzhou.json

运行命令

```
py table1_wenzhou.py Wenzhou_167_cases_deidentified.csv > table1_wenzhou.json
```

口径提醒

- **注意：**连续变量给中位数与四分位，二值变量给百分比。
- **注意：**列名在脚本开头按固定顺序重命名，防止错列。

代码全文

```
# -*- coding: utf-8 -*-
"""温州 167 例队列基线表：按 PSE 结局分组（D 论文）与按 ASM 分组（E 论文）。"""
import json
import sys

import numpy as np
import pandas as pd

COLS = ["row", "stroke_type", "sex", "age", "age_lt65", "alcohol", "smoking",
        "hypertension", "diabetes", "hyperlipidemia", "hyperuricemia",
        "seizure_type", "interval_days", "interval_gt3", "multiple_seizures",
        "status_epilepticus", "eeg", "asm", "nihss", "nihss_gt14", "infection",
        "lesion_vol", "lesion_gt10", "cortical", "multilobar", "deep",
        "ventricular_ext", "surgery", "toast", "thrombolysis", "lipid_lowering",
        "antiplatelet", "anticoag", "pse", "followup_days", "late_seizure_days",
        "censor_days"]

df = pd.read_csv(sys.argv[1])
df.columns = COLS

def med_iqr(a):
    return [round(float(np.median(a)), 1), round(float(np.percentile(a, 25)), 1),
            round(float(np.percentile(a, 75)), 1)]

def block(d):
    t = {}
    t["n"] = int(len(d))
    t["age_med_iqr"] = med_iqr(d.age)
    t["sex_male_pct"] = round(float(d.sex.mean()) * 100, 1)
    t["stroke_ischemic_pct"] = round(float((d.stroke_type == 1).mean()) * 100, 1)
    t["nihss_med_iqr"] = med_iqr(d.nihss)
    t["interval_days_med_iqr"] = med_iqr(d.interval_days)
    t["interval_gt3_pct"] = round(float(d.interval_gt3.mean()) * 100, 1)
    t["multiple_seizures_pct"] = round(float(d.multiple_seizures.mean()) * 100, 1)
    t["status_epilepticus_pct"] = round(float(d.status_epilepticus.mean()) * 100, 1)
    t["cortical_pct"] = round(float(d.cortical.mean()) * 100, 1)
    t["eeg_epileptic_pct"] = round(float((d.eeg == 1).mean()) * 100, 1)
    t["eeg_nonspecific_pct"] = round(float((d.eeg == 2).mean()) * 100, 1)
    t["eeg_normal_pct"] = round(float((d.eeg == 3).mean()) * 100, 1)
    t["sz_focal_pct"] = round(float((d.seizure_type == 1).mean()) * 100, 1)
    t["sz_btc_pct"] = round(float((d.seizure_type == 2).mean()) * 100, 1)
    t["sz_other_pct"] = round(float((d.seizure_type == 3).mean()) * 100, 1)
    t["hypertension_pct"] = round(float(d.hypertension.mean()) * 100, 1)
    t["diabetes_pct"] = round(float(d.diabetes.mean()) * 100, 1)
    t["hyperlipidemia_pct"] = round(float(d.hyperlipidemia.mean()) * 100, 1)
    t["hyperuricemia_pct"] = round(float(d.hyperuricemia.mean()) * 100, 1)
    t["smoking_pct"] = round(float(d.smoking.mean()) * 100, 1)
    t["alcohol_pct"] = round(float(d.alcohol.mean()) * 100, 1)
    t["infection_pct"] = round(float(d.infection.mean()) * 100, 1)
    t["lesion_vol_med_iqr"] = med_iqr(d.lesion_vol)
    t["multilobar_pct"] = round(float(d.multilobar.mean()) * 100, 1)
    t["deep_pct"] = round(float(d.deep.mean()) * 100, 1)
    t["asm_pct"] = round(float(d.asm.mean()) * 100, 1)
    t["followup_days_med_iqr"] = med_iqr(d.followup_days)
```

```
    t["nihss_gt14_pct"] = round(float(d.nihss_gt14.mean()) * 100, 1)
    t["thrombolysis_pct"] = round(float(d.thrombolysis.mean()) * 100, 1)
    t["surgery_pct"] = round(float(d.surgery.mean()) * 100, 1)
    return t

out = {
    "overall": block(df),
    "by_pse": {"pse_1": block(df[df.pse == 1]), "pse_0": block(df[df.pse == 0])},
    "by_asm": {"asm_1": block(df[df.asm == 1]), "asm_0": block(df[df.asm == 0])},
    "late_seizure_days": {
        "median": float(np.median(df.loc[df.pse == 1, "late_seizure_days"])),
        "min": float(df.loc[df.pse == 1, "late_seizure_days"].min()),
        "max": float(df.loc[df.pse == 1, "late_seizure_days"].max()),
        "n_le30d": int((df.loc[df.pse == 1, "late_seizure_days"] <= 30).sum()),
        "n_le90d": int((df.loc[df.pse == 1, "late_seizure_days"] <= 90).sum()),
    },
}
print(json.dumps(out, ensure_ascii=False, indent=1))
```

附录 A 环境自检脚本

A.1 tabpfn_smoke.py（51 行）

用途

加载固定权重，检查 TabPFNClassifier 是否支持 ignore_pretraining_limits/device/random_state，并在 n = 1,000/4,000/10,000 上各拟合一次，打印耗时与 AUROC。用于确认权重路径、版本和 CPU 推理正常，不产生算法架构数字。

- **对应算法架构：**环境自检
- **输入：**data.csv + 权重文件
- **输出：**标准输出

运行命令

```
$PY tabpfn_smoke.py
```

口径提醒

- **注意：**第一次在新机器上跑数时先执行它，冒烟通过再跑主基准。

代码全文

```
# -*- coding: utf-8 -*-
"""TabPFN 环境冒烟测试：分类器权重 + CPU 推理速度。"""
import inspect
import sys
import time
from pathlib import Path

import numpy as np
import pandas as pd
from sklearn.metrics import roc_auc_score
from sklearn.model_selection import train_test_split

from tabpfn import TabPFNClassifier

WEIGHTS = Path.home() / "TABPFN_BENCH_20261107/weights/tabpfn-v2-classifier-finetuned-zk73skhh.ckpt"
DATA = Path.home() / "D13_20261107/data.csv"

A_DEMO = ["age", "gender", "nihss"]
A_COMORB = ["uremia", "fatty_liver", "diabetes", "hypertension", "coronary_disease",
            "atrial_fibrillation", "hyperuricemia", "hyperlipidaemia"]
A_BRAIN = ["frontal_lobe", "parietal_lobe", "temporal_lobe", "occipital_lobe",
           "insular_lobe", "range_lobe", "basal_ganglia", "capsula_interna",
           "brainstem", "epencephalon", "paraventricular", "centrum_semiovale",
           "thalamus", "subcortex_lobe"]
A_VESSEL = ["aca", "mca", "pca", "va", "ba", "cca_plaque", "ica_plaque", "eca_plaque",
            "ant_circle", "post_circle", "large_ves_as"]
FEATURES_A = A_DEMO + A_COMORB + A_BRAIN + A_VESSEL

df = pd.read_csv(DATA)
X = df[FEATURES_A].to_numpy(dtype=float)
y = df["second_epilepsy"].to_numpy(dtype=int)
Xtr, Xte, ytr, yte = train_test_split(X, y, test_size=0.2, stratify=y, random_state=42)
print("full train", Xtr.shape, "test", Xte.shape)

sig = str(inspect.signature(TabPFNClassifier.__init__))
print("HAS ignore_pretraining_limits:", "ignore_pretraining_limits" in sig)
print("HAS device:", "device" in sig)
print("HAS random_state:", "random_state" in sig)

rng = np.random.default_rng(42)
for n in (1000, 4000, 10000):
    idx = rng.choice(len(Xtr), size=min(n, len(Xtr)), replace=False)
    t0 = time.time()
    clf = TabPFNClassifier(model_path=str(WEIGHTS), n_estimators="auto", random_state=42,
                           ignore_pretraining_limits=True, device="cpu")
    clf.fit(Xtr[idx], ytr[idx])
    t1 = time.time()
    p = clf.predict_proba(Xte)[:, 1]
    t2 = time.time()
    print(f"n_train={len(idx)} fit={t1-t0:.1f}s predict={t2-t1:.1f}s "
          f"auroc={roc_auc_score(yte, p):.4f} n_estimators_used={getattr(clf, 'n_estimators_', '?')}")
```

附录 B 脚本指纹

下表给出本汇编收录的每个文件的 SHA-256 与行数。改动任何一个字符，指纹都会变；交接时先核对指纹，再谈复现。

文件	行数	SHA-256（前 32 位）	完整 SHA-256
tabpfn_bench.py	186	af20a2c74b36b41d57c2be27c974545e	af20a2c74b36b41d57c2be27c974545ef215d6e06924461115593480eee8a149
lr_rerun.py	134	8bb5cc54cc8578d5fbf105053f8574e3	8bb5cc54cc8578d5fbf105053f8574e374de0fa708fad6ce717559a05fd8df10
verify_lr_172.py	71	13edc1e5e0cf5211bb20cca88d904e29	13edc1e5e0cf5211bb20cca88d904e29e5356bf559e0cb9da0198efcb88580ef
bench_analysis.py	196	428c2c3b9e6fd4ef1a6f3cbdda11bdf8	428c2c3b9e6fd4ef1a6f3cbdda11bdf8d38e5181c71266605aedb8565e546035
table1_baseline.py	41	7caf7d36894695f4873539f492d9a031	7caf7d36894695f4873539f492d9a0317f3b22229e980e5ef11dcc994980cf6d
restore_analyses.py	276	31bd3af5f67b853a578dab37a6353780	31bd3af5f67b853a578dab37a6353780764c0f996fe0235d6cf74d8b7c42cb90
bootstrap_ci.py	65	295501f2f33cf22aee8bd6e4ebc1d013	295501f2f33cf22aee8bd6e4ebc1d01394bbec0c2e38950754125dfde0d6e357
make_figures_restore.py	161	3c1b29400d043cdd6d6f008aaed65dab	3c1b29400d043cdd6d6f008aaed65dab22ba268c7d55cff9fd3667a095455d85
wenzhou_de.py	698	6f9924bbc0ba3bb25520560252f2e851	6f9924bbc0ba3bb25520560252f2e851dc95330ac6804f273ec01093e078a3dd
table1_wenzhou.py	72	ba6304ede96fc9b54feda39dd7d2c14b	ba6304ede96fc9b54feda39dd7d2c14b71e59e52fcc8e4a06ee5a64e573e80fe
tabpfn_smoke.py	51	95768e5828bc7c1bbab78bf56eb942f1	95768e5828bc7c1bbab78bf56eb942f1a020adfe002f0202c3d790240ec4077b
build_docx5.py	870	708b713d48a331094efcf3077b75c280	708b713d48a331094efcf3077b75c28094bd882c022b251a661a73622b755a12
to_pdf5.ps1	16	13dad6acabc36de92131056a0c8345e6	13dad6acabc36de92131056a0c8345e6c6e9cd19f37e9cfe93e4ef07c31018c1
build_resultsbook.py	1183	fb37fa0228b475e031cbf0cfbefcdf05	fb37fa0228b475e031cbf0cfbefcdf051d2e7075ff3837a99ed28703dbe8d7b5
build_codebook.py	587	4fc61cf3f61d02ea5681ddc6fe067630	4fc61cf3f61d02ea5681ddc6fe067630ce81a3539e7f66f664a3459657f08f37
external_transfer.py	153	4896df27a33ac4c20b3fdeff5c4fad87	4896df27a33ac4c20b3fdeff5c4fad87796d32bde7a00aeadf4c8b390e9ad0d3
debug_lr.py	73	64016e844df59da59409002a9658843b	64016e844df59da59409002a9658843bb5441558b56efafdf6830c875b55a532
debug_loop.py	79	9c405de97ea077d4e1623c7f32560539	9c405de97ea077d4e1623c7f32560539a4e3fd6f5be5d8cf3fe961fee80db720
setup_venv_server.sh	24	c88563058de61094cb5493e9a6cb4279	c88563058de61094cb5493e9a6cb427944ee90a863c0d83d6182e1387e3dea12
requirements.txt	7	7fcee7d59dc715b2ea98410dde9415d2	7fcee7d59dc715b2ea98410dde9415d24d7f7d9a13943627fa36974abc7048af
model_d13_server.py	564	1b31510635703876eaebcdc56f1f8812	1b31510635703876eaebcdc56f1f8812e2331d479ce3d1e0328daecfb50a1f8f
audit_d13.py	279	e65bcc9795426595d6a76c25d0c21f05	e65bcc9795426595d6a76c25d0c21f0585a3741ca71bf2e9224377a146582c09
slurm_d13.sh	27	21d4a27d07d3ec1e7c09eca18dae52b4	21d4a27d07d3ec1e7c09eca18dae52b42c21999e2b56c4a57af17b95af033a40

附录 C 未收录脚本说明

这三类脚本不改变算法架构任何数字。若需要审计“所有跑过的代码”，它们仍在 server/ 目录中，与本汇编收录版本同源。

附录 D 算法架构装配脚本

以下两个脚本用于把结果装配成 docx/PDF，不属于统计分析。放在附录是为了在需要重新生成算法架构文件时不必另找。

D.1 build_docx5.py（870 行）

用途

算法架构装配脚本（本机 Windows 运行，不参与科学计算）。读取 papers/ 下的中英文 Markdown 源稿、results/ 下的 JSON/CSV 指标、figs/ 图片，生成 10 份算法架构 docx 和《配置要求与复现说明》docx。表格中的数字全部来自结果文件，不在脚本里手写。

- **对应算法架构：**算法架构装配
- **输入：**papers/*/*.md、papers/*/figs/*.png、results/*.json、results/*.csv
- **输出：**docx/paperA–E_{en,cn}.docx；docx/配置要求与复现说明.docx

运行命令

```
py -3.12 build_docx5.py
```

口径提醒

- **注意：**表格列宽、跨页重复表头、图注同页等排版规则都写在脚本里。
- **注意：**修改算法架构数字时改 results/ 的源文件，不要改 docx。

代码全文

```
# -*- coding: utf-8 -*-
"""五篇论文（A-E, EN/CN）+ 配置要求与复现说明 的 docx 生成器。
真实表格：A/B/C 的 Table 1/2 来自 results/ 下的最终指标与 table_numbers.json；
D/E 的 Table 1 来自 table1_wenzhou.json，Table 2 来自 cv_summary_*.csv。
"""
import json
import os
import re

import pandas as pd
from docx import Document
from docx.enum.section import WD_SECTION
from docx.enum.text import WD_ALIGN_PARAGRAPH
from docx.enum.table import WD_TABLE_ALIGNMENT, WD_CELL_VERTICAL_ALIGNMENT
from docx.oxml import OxmlElement
from docx.oxml.ns import qn
from docx.shared import Cm, Pt

BASE = os.path.dirname(os.path.abspath(__file__))
PAPERS = os.path.join(BASE, "papers")
RES = os.path.join(BASE, "results")
DOCS = os.path.join(BASE, "docs")
OUT = os.path.join(BASE, "docx")
os.makedirs(OUT, exist_ok=True)

SUMMARY = json.load(open(os.path.join(RES, "bench_summary_final.json"), encoding="utf-8"))["summary"]
TN = json.load(open(os.path.join(RES, "table_numbers.json"), encoding="utf-8"))
T1 = json.load(open(os.path.join(RES, "table1.json"), encoding="utf-8"))
REST = json.load(open(os.path.join(RES, "restore", "restore.json"), encoding="utf-8"))
CI = json.load(open(os.path.join(RES, "restore", "ci.json"), encoding="utf-8"))
TW = json.load(open(os.path.join(RES, "table1_wenzhou.json"), encoding="utf-8"))

CVCOLS = ["model", "repeat_mean", "repeat_sd", "auroc", "auroc_sd",
          "auprc", "auprc_sd", "brier", "brier_sd", "cal_slope", "cal_slope_sd"]
CVC = pd.read_csv(os.path.join(RES, "cv_summary_core.csv"), skiprows=2, names=CVCOLS)
CVF = pd.read_csv(os.path.join(RES, "cv_summary_full.csv"), skiprows=2, names=CVCOLS)


# ---------------------------------------------------------------- 通用排版
def set_font(run, size, lang, bold=False, italic=False, east_font=None):
    run.font.name = "Times New Roman"
    run.font.size = Pt(size)
    run.font.bold = bold
    run.font.italic = italic
    rPr = run._element.get_or_add_rPr()
    rFonts = rPr.find(qn("w:rFonts"))
    if rFonts is None:
        rFonts = OxmlElement("w:rFonts")
        rPr.append(rFonts)
    rFonts.set(qn("w:eastAsia"), east_font or ("宋体" if lang == "cn" else "Times New Roman"))


def add_rich(p, text, size, lang, bold_all=False, italic_all=False, mono=False, east_font=None):
```

```
    parts = re.split(r"(\*\*.*?\*\*|`[^`]*`)", text)
    for part in parts:
        if not part:
            continue
        if part.startswith("**") and part.endswith("**") and len(part) > 4:
            r = p.add_run(part[2:-2])
            set_font(r, size, lang, bold=True, italic=italic_all, east_font=east_font)
        elif part.startswith("`") and part.endswith("`") and len(part) > 2:
            r = p.add_run(part[1:-1])
            set_font(r, size, lang, bold=bold_all, italic=italic_all, east_font=east_font)
            r.font.name = "Consolas"
        else:
            r = p.add_run(part)
            set_font(r, size, lang, bold=bold_all, italic=italic_all, east_font=east_font)
            if mono:
                r.font.name = "Consolas"


def add_caption(doc, text, lang, size=9, keep_next=True):
    p = doc.add_paragraph()
    p.alignment = WD_ALIGN_PARAGRAPH.CENTER
    p.paragraph_format.keep_with_next = keep_next
    m = re.match(r"^((?:Table|表)\s*\d+\.?)\s*(.*)$", text)
    if m:
        r = p.add_run(m.group(1) + " ")
        set_font(r, size, lang, bold=True)
        r2 = p.add_run(m.group(2))
        set_font(r2, size, lang)
    else:
        add_rich(p, text, size, lang)
    p.paragraph_format.space_after = Pt(3)


def _tr_prop(row, tag):
    trPr = row._tr.get_or_add_trPr()
    el = OxmlElement(tag)
    trPr.append(el)


def add_table(doc, hdr, rows, foot, lang, size=8, widths=None):
    t = doc.add_table(rows=1, cols=len(hdr))
    t.style = "Table Grid"
    t.alignment = WD_TABLE_ALIGNMENT.CENTER
    _tr_prop(t.rows[0], "w:tblHeader")
    if widths:
        t.autofit = False
        for j, w in enumerate(widths):
            t.columns[j].width = Cm(w)
            t.rows[0].cells[j].width = Cm(w)
    for j, h in enumerate(hdr):
        c = t.rows[0].cells[j]
        c.vertical_alignment = WD_CELL_VERTICAL_ALIGNMENT.CENTER
        c.paragraphs[0].alignment = WD_ALIGN_PARAGRAPH.CENTER
        r = c.paragraphs[0].add_run(str(h))
        set_font(r, size, lang, bold=True)
    for row in rows:
        tr = t.add_row()
        _tr_prop(tr, "w:cantSplit")
        cells = tr.cells
        if widths:
            for j, w in enumerate(widths):
                cells[j].width = Cm(w)
        if isinstance(row, tuple) and row and row[0] == "__SECTION__":
            merged = cells[0].merge(cells[-1])
            r = merged.paragraphs[0].add_run(row[1])
            set_font(r, size, lang, bold=True)
            continue
        for j, v in enumerate(row):
            cells[j].vertical_alignment = WD_CELL_VERTICAL_ALIGNMENT.CENTER
            cells[j].paragraphs[0].alignment = (WD_ALIGN_PARAGRAPH.LEFT if j == 0 else WD_ALIGN_PARAGRAPH.CENTER)
            add_rich(cells[j].paragraphs[0], str(v), size, lang)
    if len(rows) <= 20:  # 短表整表不跨页
        for tr in t.rows[:-1]:
            for c in tr.cells:
                for para in c.paragraphs:
                    para.paragraph_format.keep_with_next = True
    if foot:
        p = doc.add_paragraph()
        r = p.add_run(foot)
```

```
        set_font(r, 7.5, lang, italic=True)
        p.paragraph_format.space_after = Pt(8)


def new_section(doc, cols):
    s = doc.add_section(WD_SECTION.CONTINUOUS)
    sectPr = s._sectPr
    c = sectPr.find(qn("w:cols"))
    if c is None:
        c = OxmlElement("w:cols")
        sectPr.append(c)
    c.set(qn("w:num"), str(cols))
    c.set(qn("w:space"), "720")
    return s


# ---------------------------------------------------------------- 数字格式
def f4(x):
    return f"{x:.4f}"


def sgn(x, nd=4):
    return f"{x:+.{nd}f}"


def ci(v, nd=4):
    return f"{v[0]:+.{nd}f} ({v[1]:+.{nd}f}, {v[2]:+.{nd}f})"


def med_iqr(v):
    return f"{v[0]:g} ({v[1]:g}–{v[2]:g})"


def pct(v):
    return f"{v:.1f}%"


def get(combo, key):
    return SUMMARY[combo][f"p_{ {'lr':'lr','lgbm':'lgbm','tabpfn':'tabpfn'}[key] }"]


def met(combo, model, field):
    return SUMMARY[combo]["p_" + model][field]


# ---------------------------------------------------------------- A 表 1
A_T1_ROWS = [
    ("age", "Age, years", "年龄（岁）", "med"),
    ("gender", "Sex (value 1)", "性别（取 1 值）", "pct"),
    ("nihss", "NIHSS score", "NIHSS 评分", "med"),
    ("__SEC__", "Comorbidities", "合并症"),
    ("uremia", "Uremia", "尿毒症", "pct"),
    ("fatty_liver", "Fatty liver", "脂肪肝", "pct"),
    ("diabetes", "Diabetes mellitus", "糖尿病", "pct"),
    ("hypertension", "Hypertension", "高血压", "pct"),
    ("coronary_disease", "Coronary heart disease", "冠心病", "pct"),
    ("atrial_fibrillation", "Atrial fibrillation", "心房颤动", "pct"),
    ("hyperuricemia", "Hyperuricemia", "高尿酸血症", "pct"),
    ("hyperlipidaemia", "Hyperlipidemia", "高脂血症", "pct"),
    ("__SEC__", "Initial imaging: lesion location", "初始影像：病灶位置"),
    ("frontal_lobe", "Frontal lobe", "额叶", "pct"),
    ("parietal_lobe", "Parietal lobe", "顶叶", "pct"),
    ("temporal_lobe", "Temporal lobe", "颞叶", "pct"),
    ("occipital_lobe", "Occipital lobe", "枕叶", "pct"),
    ("insular_lobe", "Insular lobe", "岛叶", "pct"),
    ("range_lobe", "Any lobe involved (≥1)", "脑叶受累（任一，≥1）", "pct"),
    ("basal_ganglia", "Basal ganglia", "基底节", "pct"),
    ("capsula_interna", "Internal capsule", "内囊", "pct"),
    ("brainstem", "Brainstem", "脑干", "pct"),
    ("epencephalon", "Cerebellum", "小脑", "pct"),
    ("paraventricular", "Paraventricular", "脑室旁", "pct"),
    ("centrum_semiovale", "Centrum semiovale", "半卵圆中心", "pct"),
    ("thalamus", "Thalamus", "丘脑", "pct"),
    ("subcortex_lobe", "Subcortical", "皮层下", "pct"),
    ("__SEC__", "Vascular features", "血管特征"),
    ("aca", "ACA territory", "ACA 供血区", "pct"),
    ("mca", "MCA territory", "MCA 供血区", "pct"),
    ("pca", "PCA territory", "PCA 供血区", "pct"),
    ("va", "VA territory", "VA 供血区", "pct"),
```

```
    ("ba", "BA territory", "BA 供血区", "pct"),
    ("cca_plaque", "Common carotid plaque", "颈总动脉斑块", "pct"),
    ("ica_plaque", "Internal carotid plaque", "颈内动脉斑块", "pct"),
    ("eca_plaque", "External carotid plaque", "颈外动脉斑块", "pct"),
    ("ant_circle", "Anterior circulation", "前循环", "pct"),
    ("post_circle", "Posterior circulation", "后循环", "pct"),
    ("large_ves_as", "Large-vessel atherosclerosis", "大血管动脉粥样硬化", "pct"),
]

LAB_ROWS = [
    ("plt", "Platelet count (×10⁹/L)", "血小板计数（×10⁹/L）"),
    ("wbc", "White blood cell count (×10⁹/L)", "白细胞计数（×10⁹/L）"),
    ("rbc", "Red blood cell count (×10¹²/L)", "红细胞计数（×10¹²/L）"),
    ("hba1c", "HbA1c (%)", "糖化血红蛋白（%）"),
    ("crp", "C-reactive protein (mg/L)", "C 反应蛋白（mg/L）"),
    ("tg", "Triglycerides (mmol/L)", "甘油三酯（mmol/L）"),
    ("ldl", "LDL cholesterol (mmol/L)", "低密度脂蛋白胆固醇（mmol/L）"),
    ("hdl", "HDL cholesterol (mmol/L)", "高密度脂蛋白胆固醇（mmol/L）"),
    ("ast", "Aspartate aminotransferase (U/L)", "谷草转氨酶（U/L）"),
    ("alt", "Alanine aminotransferase (U/L)", "谷丙转氨酶（U/L）"),
    ("bilirubin", "Total bilirubin (μmol/L)", "总胆红素（μmol/L）"),
    ("albumin", "Albumin (g/L)", "白蛋白（g/L）"),
    ("urea", "Urea (mmol/L)", "尿素（mmol/L）"),
    ("creatinine", "Creatinine (μmol/L)", "肌酐（μmol/L）"),
    ("bua", "Uric acid (μmol/L)", "血尿酸（μmol/L）"),
    ("pt", "Prothrombin time (s)", "凝血酶原时间（s）"),
    ("aptt", "Activated partial thromboplastin time (s)", "活化部分凝血活酶时间（s）"),
    ("tt", "Thrombin time (s)", "凝血酶时间（s）"),
    ("inr", "International normalized ratio", "国际标准化比值"),
    ("d_dimer", "D-dimer (mg/L)", "D-二聚体（mg/L）"),
    ("fibrinogen", "Fibrinogen (g/L)", "纤维蛋白原（g/L）"),
    ("ck", "Creatine kinase (U/L)", "肌酸激酶（U/L）"),
    ("ck_mb", "Creatine kinase-MB (U/L)", "肌酸激酶同工酶 MB（U/L）"),
    ("ldh", "Lactate dehydrogenase (U/L)", "乳酸脱氢酶（U/L）"),
    ("hbdh", "α-Hydroxybutyrate dehydrogenase (U/L)", "α-羟丁酸脱氢酶（U/L）"),
    ("ima", "Ischemia-modified albumin (AU)", "缺血修饰白蛋白（AU）"),
    ("na", "Sodium (mmol/L)", "钠（mmol/L）"),
    ("k", "Potassium (mmol/L)", "钾（mmol/L）"),
    ("cl", "Chloride (mmol/L)", "氯（mmol/L）"),
    ("ca", "Calcium (mmol/L)", "钙（mmol/L）"),
    ("p", "Phosphorus (mmol/L)", "磷（mmol/L）"),
    ("lactate", "Lactate (mmol/L)", "乳酸（mmol/L）"),
    ("anion_gap", "Anion gap (mmol/L)", "阴离子间隙（mmol/L）"),
    ("tco2", "Total CO2 (mmol/L)", "总二氧化碳（mmol/L）"),
]

POST_ROWS = [
    ("dvt", "Deep vein thrombosis", "深静脉血栓"),
    ("cerebral_hernia", "Cerebral hernia", "脑疝"),
    ("hydrocephalus", "Hydrocephalus", "脑积水"),
    ("hypoproteinemia", "Hypoproteinemia", "低蛋白血症"),
]

T1P = {fs: {r["var"]: r for r in REST["table1"][fs]} for fs in ("A", "B", "C")}


def t1_header(lang, d):
    if lang == "en":
        return ["Characteristic", f"Overall (n = {d['n_total']:,})",
                f"No PSE (n = {d['n_neg']:,})", f"PSE (n = {d['n_pos']:,})", "P value"]
    return ["特征", f"总体（n = {d['n_total']:,}）",
            f"无 PSE（n = {d['n_neg']:,}）", f"PSE（n = {d['n_pos']:,}）", "P 值"]


def t1_row(lang, key, en, cn):
    r = T1P["A"].get(key) or T1P["B"].get(key) or T1P["C"].get(key)
    return [en if lang == "en" else cn, r["overall"], r["neg"], r["pos"], r["p"]]


def tbl_A1(lang):
    hdr = t1_header(lang, T1["full"])
    rows = []
    for item in A_T1_ROWS:
        if len(item) == 3:
            rows.append(("__SECTION__", item[1] if lang == "en" else item[2]))
            continue
        key, en, cn, _ = item
        rows.append(t1_row(lang, key, en, cn))
    cap = ("Table 1. Baseline characteristics of the 21,459 patients by one-year epilepsy status (full cohort)."
```

```
           if lang == "en" else "表 1. 21,459 例患者按一年内癫痫状态的基线特征（全队列）。")
    foot = ("Continuous variables: median (IQR); binary variables: n (%). P values: χ² test for binary variables, "
            "Mann–Whitney U test for continuous variables. PSE, post-stroke epilepsy."
            if lang == "en" else
            "连续变量：中位数（IQR）；二值变量：n（%）。P 值：二值变量用 χ² 检验，连续变量用 Mann–Whitney U 检验。"
            "PSE，卒中后癫痫。")
    return cap, hdr, rows, foot


# ---------------------------------------------------------------- A 表 2
def f4ci(key, model, metric):
    d = CI[key][model]
    lo, hi = d["ci"][metric]
    return f"{d[metric]:.4f} ({lo:.4f}–{hi:.4f})"


def tbl_A2(lang):
    hdr = (["Model", "Training cohort / n", "AUROC (95% CI)", "AUPRC (95% CI)", "Brier (95% CI)", "Calibration slope"]
           if lang == "en" else ["模型", "训练队列 / n", "AUROC（95% CI）", "AUPRC（95% CI）", "Brier（95% CI）", "校准斜率"])
    rows = []
    labels = {"lr": "Logistic regression", "lgbm": "LightGBM", "tabpfn": "TabPFN v2"}
    labels_cn = {"lr": "logistic 回归", "lgbm": "LightGBM", "tabpfn": "TabPFN v2"}
    cohorts = [("A_full", "A_full_8000", "Full", "全队列", "8,000"),
               ("A_dedup", "A_dedup_8000", "Dedup", "去重队列", "8,000"),
               ("A_full", "A_full_17167", "Full", "全队列", "17,167")]
    for m in ["lr", "lgbm", "tabpfn"]:
        for combo, key, coh_en, coh_cn, n in cohorts:
            label = f"{coh_en} / {n}" if lang == "en" else f"{coh_cn} / {n}"
            if n.startswith("17") and m == "tabpfn":
                rows.append([labels[m] if lang == "en" else labels_cn[m], label, "–", "–", "–", "–"])
                continue
            if n.startswith("17"):
                sl = TN[f"slope_full_A_{m}_fullsize"]
                rows.append([labels[m] if lang == "en" else labels_cn[m], label,
                             f4ci(key, m, "auroc"), f4ci(key, m, "auprc"), f4ci(key, m, "brier"),
                             f"{sl:.4f}"])
            else:
                rows.append([labels[m] if lang == "en" else labels_cn[m], label,
                             f4ci(key, m, "auroc"), f4ci(key, m, "auprc"), f4ci(key, m, "brier"),
                             f4(met(combo, m, "cal_slope"))])
    cap = ("Table 2. Test-set discrimination and calibration at n = 8,000 and at the full training set, in the full and
deduplicated cohorts."
           if lang == "en" else "表 2. n = 8,000 与全训练集下的测试集判别力与校准（全队列与去重队列）。")
    foot = ("Values are point estimates with 95% bootstrap confidence intervals (2,000 resamples of test rows). "
            "At n = 8,000 (full cohort), ΔAUROC TabPFN − LightGBM = +0.0067 (95% CI −0.0021 to +0.0159); "
            "TabPFN − logistic regression = +0.0892 (+0.0718 to +0.1090). TabPFN was not evaluated above 8,000 "
            "because of its native context limit."
            if lang == "en" else
            "数值为点估计与 95% bootstrap 置信区间（对测试行重抽样 2,000 次）。n = 8,000（全队列）时，"
            "ΔAUROC TabPFN − LightGBM = +0.0067（95% CI −0.0021 至 +0.0159）；TabPFN − logistic 回归 = +0.0892（+0.0718 至
+0.1090）。"
            "受原生上下文上限约束，TabPFN 未在 8,000 以上评估。")
    return cap, hdr, rows, foot


# ---------------------------------------------------------------- A 表 3 / 表 4
def feat_label(var, lang):
    for key, en, cn, *_ in A_T1_ROWS:
        if key == var:
            return en if lang == "en" else cn
    for key, en, cn in LAB_ROWS + POST_ROWS:
        if key == var:
            return en if lang == "en" else cn
    return var


def tbl_A3(lang):
    hdr = (["Rank", "LightGBM feature (gain share)", "Rank", "Logistic regression feature (|coefficient|)"]
           if lang == "en" else ["排名", "LightGBM 特征（增益占比）", "排名", "logistic 回归特征（|系数|）"])
    g = REST["importance"]["A"]["lgbm"][:10]
    l = REST["importance"]["A"]["lr"][:10]
    rows = []
    for i in range(10):
        rows.append([str(i + 1), f"{feat_label(g[i]['var'], lang)} ({g[i]['gain_share']*100:.2f}%)",
                     str(i + 1), f"{feat_label(l[i]['var'], lang)} ({l[i]['abs_coef']:.2f})"])
    cap = ("Table 3. Top ten predictors in the two admission-only models (full training set, n = 17,167)."
           if lang == "en" else "表 3. 仅入院特征两模型的前十预测因子（全训练集，n = 17,167）。")
    foot = ("LightGBM importance is total gain share; logistic-regression importance is the absolute coefficient "
            "on the protocol preprocessing scale."
```

```
           if lang == "en" else "LightGBM 为总增益占比；logistic 回归为协议预处理尺度上的系数绝对值。")
    return cap, hdr, rows, foot


def tbl_A4(lang):
    hdr = (["Model", "CV AUROC", "CV AUPRC", "CV Brier", "OOF AUROC", "OOF AUPRC", "OOF Brier"]
           if lang == "en" else ["模型", "CV AUROC", "CV AUPRC", "CV Brier", "OOF AUROC", "OOF AUPRC", "OOF Brier"])
    rows = []
    labels = {"lr": ("Logistic regression", "logistic 回归"), "lgbm": ("LightGBM", "LightGBM")}
    for m in ["lr", "lgbm"]:
        d = REST["cv_A"][m]
        rows.append([labels[m][0] if lang == "en" else labels[m][1],
                     f"{d['auroc_mean']:.4f} ± {d['auroc_sd']:.4f}",
                     f"{d['auprc_mean']:.4f} ± {d['auprc_sd']:.4f}",
                     f"{d['brier_mean']:.4f} ± {d['brier_sd']:.4f}",
                     f"{d['oof_auroc']:.4f}", f"{d['oof_auprc']:.4f}", f"{d['oof_brier']:.4f}"])
    cap = ("Table 4. Five-fold cross-validation and pooled out-of-fold performance on the training set (full cohort)."
           if lang == "en" else "表 4. 训练集五折交叉验证与汇总袋外性能（全队列）。")
    foot = ("CV, five-fold stratified cross-validation (mean ± SD across folds); OOF, pooled out-of-fold predictions. "
            "The hold-out test set is never used here."
            if lang == "en" else "CV，五折分层交叉验证（各折均值 ± 标准差）；OOF，汇总袋外预测。此处不使用留出测试集。")
    return cap, hdr, rows, foot


# ---------------------------------------------------------------- B 表 1-3
def tbl_B1(lang):
    hdr = t1_header(lang, T1["full"])
    rows = [("__SECTION__", "Admission characteristics (36)" if lang == "en" else "入院特征（36 项）")]
    for item in A_T1_ROWS:
        if len(item) == 3:
            rows.append(("__SECTION__", item[1] if lang == "en" else item[2]))
            continue
        key, en, cn, _ = item
        rows.append(t1_row(lang, key, en, cn))
    rows.append(("__SECTION__", "Laboratory tests (34)" if lang == "en" else "实验室检查（34 项）"))
    for key, en, cn in LAB_ROWS:
        r = T1P["B"][key]
        rows.append([en if lang == "en" else cn, r["overall"], r["neg"], r["pos"], r["p"]])
    cap = ("Table 1. Baseline and laboratory characteristics of the 21,459 patients by one-year epilepsy status (full
cohort)."
           if lang == "en" else "表 1. 21,459 例患者按一年内癫痫结局分组的基线与实验室特征（全队列）。")
    foot = ("Continuous variables: median (IQR); binary variables: n (%). P values: χ² test for binary variables, "
            "Mann–Whitney U test for continuous variables. PSE, post-stroke epilepsy."
            if lang == "en" else
            "连续变量：中位数（IQR）；二值变量：n（%）。P 值：二值变量用 χ² 检验，连续变量用 Mann–Whitney U 检验。"
            "PSE，卒中后癫痫。")
    return cap, hdr, rows, foot


def tbl_B2(lang):
    hdr = (["Model", "Feature set", "AUROC", "AUPRC", "Brier", "Slope", "ΔAUROC vs 36 (95% CI)"]
           if lang == "en" else ["模型", "特征集", "AUROC", "AUPRC", "Brier", "斜率", "ΔAUROC vs 36（95% CI）"])
    labels = {"lr": ("Logistic regression", "logistic 回归"), "lgbm": ("LightGBM", "LightGBM"), "tabpfn": ("TabPFN v2",
"TabPFN v2")}
    rows = []
    for m in ["lr", "lgbm", "tabpfn"]:
        lab = labels[m][0] if lang == "en" else labels[m][1]
        for combo, fs in [("A_full", "36 admission"), ("B_full", "70 admission + lab")]:
            rows.append([lab if fs.startswith("36") else "",
                         fs if lang == "en" else ("36 入院特征" if fs.startswith("36") else "70 入院＋实验室"),
                         f4(met(combo, m, "auroc")), f4(met(combo, m, "auprc")),
                         f4(met(combo, m, "brier")), f4(met(combo, m, "cal_slope")),
                         "—" if fs.startswith("36") else ci(TN[f"delta_AB_full_{m}"])])
    cap = ("Table 2. Incremental value of the 34-feature laboratory block at n = 8,000 (full cohort)."
           if lang == "en" else "表 2. n = 8,000 时 34 项实验室特征的增量价值（全队列）。")
    foot = ("ΔAUROC from 2,000 paired bootstrap replications on identical test patients. The laboratory block added "
            "+0.1072, +0.0544 and +0.0535 AUROC to logistic regression, LightGBM and TabPFN, respectively."
            if lang == "en" else
            "ΔAUROC 为同一批测试患者上的 2,000 次配对 bootstrap。实验室块分别使 logistic 回归、LightGBM 与 TabPFN 的 AUROC "
            "提高 +0.1072、+0.0544 与 +0.0535。")
    return cap, hdr, rows, foot


def tbl_B3(lang):
    hdr = (["Rank", "LightGBM feature (gain share)", "Rank", "Logistic regression feature (|coefficient|)"]
           if lang == "en" else ["排名", "LightGBM 特征（增益占比）", "排名", "logistic 回归特征（|系数|）"])
    g = REST["importance"]["B"]["lgbm"][:15]
    l = REST["importance"]["B"]["lr"][:15]
    rows = []
```

```
    for i in range(15):
        rows.append([str(i + 1), f"{feat_label(g[i]['var'], lang)} ({g[i]['gain_share']*100:.2f}%)",
                     str(i + 1), f"{feat_label(l[i]['var'], lang)} ({l[i]['abs_coef']:.2f})"])
    cap = ("Table 3. Top fifteen predictors in the 70-feature models (full training set, n = 17,167)."
           if lang == "en" else "表 3. 70 特征模型的前十五预测因子（全训练集，n = 17,167）。")
    foot = ("LightGBM importance is total gain share; logistic-regression importance is the absolute coefficient "
            "on the protocol preprocessing scale."
            if lang == "en" else "LightGBM 为总增益占比；logistic 回归为协议预处理尺度上的系数绝对值。")
    return cap, hdr, rows, foot


# ---------------------------------------------------------------- C 表 1-3
def tbl_C1(lang):
    hdr = t1_header(lang, T1["full"])
    rows = []
    for key, en, cn in POST_ROWS:
        r = T1P["C"][key]
        rows.append([en if lang == "en" else cn, r["overall"], r["neg"], r["pos"], r["p"]])
    cap = ("Table 1. Prevalence of the four post-stroke complications by one-year epilepsy status (full cohort)."
           if lang == "en" else "表 1. 四项卒中后并发症按一年内癫痫结局分组的患病率（全队列）。")
    foot = ("Values are n (%). P values from the χ² test. PSE, post-stroke epilepsy."
            if lang == "en" else "数值为 n（%）。P 值来自 χ² 检验。PSE，卒中后癫痫。")
    return cap, hdr, rows, foot


def tbl_C2(lang):
    hdr = (["Model", "Feature set", "AUROC", "AUPRC", "Brier", "Slope", "ΔAUROC vs 70 (95% CI)"]
           if lang == "en" else ["模型", "特征集", "AUROC", "AUPRC", "Brier", "斜率", "ΔAUROC vs 70（95% CI）"])
    labels = {"lr": ("Logistic regression", "logistic 回归"), "lgbm": ("LightGBM", "LightGBM"), "tabpfn": ("TabPFN v2",
"TabPFN v2")}
    rows = []
    for m in ["lr", "lgbm", "tabpfn"]:
        lab = labels[m][0] if lang == "en" else labels[m][1]
        for combo, fs in [("B_full", "70 admission + lab"), ("C_full", "74 + complications")]:
            rows.append([lab if fs.startswith("70") else "",
                         fs if lang == "en" else ("70 入院+实验室" if fs.startswith("70") else "74 +卒中后并发症"),
                         f4(met(combo, m, "auroc")), f4(met(combo, m, "auprc")),
                         f4(met(combo, m, "brier")), f4(met(combo, m, "cal_slope")),
                         "—" if fs.startswith("70") else ci(TN[f"delta_CB_full_{m}"])])
    cap = ("Table 2. Incremental value of the four-variable post-stroke complication block at n = 8,000 (full cohort)."
           if lang == "en" else "表 2. n = 8,000 时四变量卒中后并发症块的增量价值（全队列）。")
    foot = ("ΔAUROC from 2,000 paired bootstrap replications on identical test patients. Only the logistic-regression "
            "interval excludes zero (+0.0043, 95% CI +0.0008 to +0.0084), a gain 25-fold smaller than the laboratory
increment (+0.107)."
            if lang == "en" else
            "ΔAUROC 为同一批测试患者上的 2,000 次配对 bootstrap。仅 logistic 回归的区间排除零（+0.0043，95% CI +0.0008 至
+0.0084），"
            "其增益仅为实验室增量（+0.107）的二十五分之一。")
    return cap, hdr, rows, foot


def tbl_C3(lang):
    hdr = (["Variable", "Prevalence, n (%)", "Univariate P", "LGBM rank (/74)", "LR rank (/74)"]
           if lang == "en" else ["变量", "患病率，n（%）", "单变量 P", "LGBM 排名（/74）", "LR 排名（/74）"])
    rows = []
    for key, en, cn in POST_ROWS:
        r = T1P["C"][key]
        rg = REST["importance"]["C"]["ranks_lgbm"][key]
        rl = REST["importance"]["C"]["ranks_lr"][key]
        rows.append([en if lang == "en" else cn, r["overall"], r["p"], str(rg), str(rl)])
    cap = ("Table 3. Prevalence, univariate association and importance rank of the four complication variables in the 74-
feature models."
           if lang == "en" else "表 3. 四项并发症变量的患病率、单变量关联与在 74 特征模型中的重要性排名。")
    foot = ("Ranks are among all 74 features; 1 = most important. Importance is total gain for LightGBM and absolute "
            "coefficient for logistic regression (full training set)."
            if lang == "en" else "排名范围为全部 74 个特征，1 = 最重要。重要性在全训练集上计算：LightGBM 为总增益，"
            "logistic 回归为系数绝对值。")
    return cap, hdr, rows, foot


# ---------------------------------------------------------------- D/E 表 1
WZ_ROWS = [
    ("age_med_iqr", "Age, years", "年龄（岁）", "med"),
    ("sex_male_pct", "Male", "男性", "pct"),
    ("stroke_ischemic_pct", "Ischemic stroke", "缺血性卒中", "pct"),
    ("nihss_med_iqr", "NIHSS score", "NIHSS 评分", "med"),
    ("interval_days_med_iqr", "Days from stroke to first seizure", "卒中至首次发作天数", "med"),
    ("interval_gt3_pct", "First seizure >3 days after stroke", "首次发作在卒中 3 天后", "pct"),
    ("multiple_seizures_pct", "Multiple early seizures", "早期多次发作", "pct"),
```

```
    ("status_epilepticus_pct", "Status epilepticus", "癫痫持续状态", "pct"),
    ("cortical_pct", "Cortical involvement", "皮层受累", "pct"),
    ("eeg_epileptic_pct", "Epileptiform EEG", "癫痫样脑电", "pct"),
    ("hypertension_pct", "Hypertension", "高血压", "pct"),
    ("diabetes_pct", "Diabetes mellitus", "糖尿病", "pct"),
    ("hyperlipidemia_pct", "Hyperlipidemia", "高脂血症", "pct"),
    ("hyperuricemia_pct", "Hyperuricemia", "高尿酸血症", "pct"),
    ("smoking_pct", "Smoking", "吸烟", "pct"),
    ("alcohol_pct", "Alcohol use", "饮酒", "pct"),
    ("infection_pct", "Infection", "感染", "pct"),
    ("lesion_vol_med_iqr", "Lesion volume (mL)", "病灶体积（mL）", "med"),
    ("multilobar_pct", "Multilobar involvement", "多脑叶受累", "pct"),
    ("deep_pct", "Deep structure involvement", "深部结构受累", "pct"),
    ("nihss_gt14_pct", "NIHSS >14", "NIHSS >14", "pct"),
    ("thrombolysis_pct", "Thrombolysis", "溶栓", "pct"),
    ("surgery_pct", "Neurosurgery", "神经外科手术", "pct"),
]


def wz_block(d, lang, include_asm=True):
    rows = []
    for key, en, cn, typ in WZ_ROWS:
        v = d[key]
        val = med_iqr(v) if typ == "med" else pct(v)
        rows.append([en if lang == "en" else cn, val])
    if include_asm:
        rows.insert(2, ["ASM (antiseizure medication)" if lang == "en" else "抗癫痫药物（ASM）", pct(d["asm_pct"])])
    rows.append(["Follow-up, days" if lang == "en" else "随访天数", med_iqr(d["followup_days_med_iqr"])])
    return rows


def tbl_D1(lang):
    a, b = TW["by_pse"]["pse_0"], TW["by_pse"]["pse_1"]
    hdr = (["Characteristic", f"No PSE (n = {a['n']})", f"PSE (n = {b['n']})"]
           if lang == "en" else ["特征", f"无 PSE（n = {a['n']}）", f"PSE（n = {b['n']}）"])
    ra, rb = wz_block(a, lang, include_asm=True), wz_block(b, lang, include_asm=True)
    rows = [[ra[i][0], ra[i][1], rb[i][1]] for i in range(len(ra))]
    cap = ("Table 1. Baseline characteristics of the 167 patients by PSE status."
           if lang == "en" else "表 1. 167 例患者按 PSE 状态的基线特征。")
    foot = ("Continuous variables: median (IQR); binary variables: %. ASM, antiseizure medication; EEG,
electroencephalogram."
            if lang == "en" else "连续变量：中位数（IQR）；二值变量：百分比（%）。ASM，抗癫痫药物；EEG，脑电图。")
    return cap, hdr, rows, foot


def tbl_E1(lang):
    a, b = TW["by_asm"]["asm_1"], TW["by_asm"]["asm_0"]
    hdr = (["Characteristic", f"ASM (n = {a['n']})", f"No ASM (n = {b['n']})"]
           if lang == "en" else ["特征", f"ASM 组（n = {a['n']}）", f"未用 ASM（n = {b['n']}）"])
    ra, rb = wz_block(a, lang, include_asm=False), wz_block(b, lang, include_asm=False)
    rows = [[ra[i][0], ra[i][1], rb[i][1]] for i in range(len(ra))]
    cap = ("Table 1. Baseline characteristics of the 167 patients by ASM status."
           if lang == "en" else "表 1. 167 例患者按 ASM 状态的基线特征。")
    foot = ("Continuous variables: median (IQR); binary variables: %. ASM, antiseizure medication."
            if lang == "en" else "连续变量：中位数（IQR）；二值变量：百分比（%）。ASM，抗癫痫药物。")
    return cap, hdr, rows, foot


# ---------------------------------------------------------------- D 表 2
def tbl_D2(lang):
    hdr = (["Model", "Feature set", "AUROC", "AUPRC", "Brier", "Calibration slope"]
           if lang == "en" else ["模型", "特征集", "AUROC", "AUPRC", "Brier", "校准斜率"])
    labels = {"lr": ("Logistic regression", "logistic 回归"), "lgbm": ("LightGBM", "LightGBM"), "tabpfn": ("TabPFN v2",
"TabPFN v2")}
    order = ["lr", "lgbm", "tabpfn"]
    rows = []
    for m in order:
        for df_, fs in [(CVC, "core (8 variables)"), (CVF, "full (24 variables)")]:
            r = df_[df_["model"] == m].iloc[0]
            rows.append([labels[m][0] if lang == "en" else labels[m][1],
                         fs if lang == "en" else ("核心（8 变量）" if fs.startswith("core") else "全集（24 变量）"),
                         f"{float(r['auroc']):.4f} ± {float(r['auroc_sd']):.4f}",
                         f"{float(r['auprc']):.4f} ± {float(r['auprc_sd']):.4f}",
                         f"{float(r['brier']):.4f} ± {float(r['brier_sd']):.4f}",
                         f"{float(r['cal_slope']):.4f} ± {float(r['cal_slope_sd']):.4f}"])
    cap = ("Table 2. Cross-validated performance (mean ± SD over 50 folds) of the three models on the core and full
feature sets."
           if lang == "en" else "表 2. 三模型在核心与全集特征集上的交叉验证表现（50 折均值 ± 标准差）。")
    foot = ("50 folds = 5-fold stratified cross-validation × 10 repeats, seed 42. Paired bootstrap ΔAUROC (core set): "
```

```
            "TabPFN − logistic regression = −0.0565 (95% CI −0.1090 to −0.0058); TabPFN − LightGBM = −0.0383 (−0.0943 to
+0.0145)."
            if lang == "en" else
            "50 折 = 5 折分层交叉验证 × 10 次重复，种子 42。配对 bootstrap ΔAUROC（核心集）："
            "TabPFN − logistic 回归 = −0.0565（95% CI −0.1090 至 −0.0058）；TabPFN − LightGBM = −0.0383（−0.0943 至
+0.0145）。")
    return cap, hdr, rows, foot


# ---------------------------------------------------------------- 解析 markdown
def parse_md(path):
    blocks = []
    with open(path, encoding="utf-8") as f:
        for raw in f:
            s = raw.rstrip("\n").strip()
            if not s or s == "---":
                continue
            if s.startswith("!["):
                m = re.match(r"^!\[(.*?)\]\((.*?)\)$", s)
                blocks.append(("fig", m.group(1), m.group(2)))
            elif s.startswith("*[") and s.endswith("]*"):
                blocks.append(("tbl", s))
            elif s.startswith("# "):
                blocks.append(("title", s[2:]))
            elif s.startswith("### "):
                blocks.append(("h2", s[4:]))
            elif s.startswith("## "):
                blocks.append(("h1", s[3:]))
            else:
                blocks.append(("p", s))
    return blocks


PAPER_DIRS = {"A": "01_A", "B": "02_B", "C": "03_C", "D": "04_D", "E": "05_E"}
WIDTHS = {
    ("A", 0): [4.2, 3.4, 3.5, 3.4, 1.9],
    ("A", 1): [2.8, 2.7, 3.2, 3.2, 3.2, 1.9],
    ("A", 2): [1.2, 7.6, 1.2, 6.8],
    ("A", 3): [2.5, 2.4, 2.4, 2.4, 2.4, 2.4, 2.4],
    ("B", 0): [4.2, 3.4, 3.5, 3.4, 1.9],
    ("B", 1): [2.8, 3.0, 1.8, 1.8, 1.8, 1.8, 4.0],
    ("B", 2): [1.2, 7.6, 1.2, 6.8],
    ("C", 0): [4.2, 3.4, 3.5, 3.4, 1.9],
    ("C", 1): [2.8, 3.0, 1.8, 1.8, 1.8, 1.8, 4.0],
    ("C", 2): [4.2, 3.4, 2.4, 3.4, 3.4],
    ("D", 0): [6.2, 5.4, 5.4],
    ("D", 1): [3.4, 3.6, 2.5, 2.5, 2.5, 2.5],
    ("E", 0): [6.2, 5.4, 5.4],
}


def build_paper(paper, lang):
    md = os.path.join(PAPERS, PAPER_DIRS[paper], f"paper{paper}_{'en' if lang == 'en' else 'cn'}.md")
    doc_dir = os.path.dirname(md)
    blocks = parse_md(md)
    spec = {"A": [tbl_A1, tbl_A2, tbl_A3, tbl_A4],
            "B": [tbl_B1, tbl_B2, tbl_B3],
            "C": [tbl_C1, tbl_C2, tbl_C3],
            "D": [tbl_D1, tbl_D2], "E": [tbl_E1]}[paper]
    ti = 0

    doc = Document()
    sec = doc.sections[0]
    sec.page_width, sec.page_height = Cm(21), Cm(29.7)
    sec.top_margin = sec.bottom_margin = Cm(2.54)
    sec.left_margin = sec.right_margin = Cm(2.0)

    refs_h1 = "References" if lang == "en" else "参考文献"
    abstract_h1 = "Abstract" if lang == "en" else "摘要"
    head_east = "黑体" if lang == "cn" else None

    assert blocks[0][0] == "title"
    p = doc.add_paragraph()
    p.alignment = WD_ALIGN_PARAGRAPH.CENTER
    add_rich(p, blocks[0][1], 16, lang, bold_all=True, east_font=head_east)
    p.paragraph_format.space_after = Pt(6)
    i = 1
    # 作者/通讯行（正文开始前的普通段落）
    while i < len(blocks) and blocks[i][0] == "p" and not blocks[i][1].startswith("**"):
```

```
    p = doc.add_paragraph()
    p.alignment = WD_ALIGN_PARAGRAPH.CENTER
    add_rich(p, blocks[i][1], 10.5 if i == 1 else 9, lang, italic_all=(i > 1))
    p.paragraph_format.space_after = Pt(2)
    i += 1

body_started = False
while i < len(blocks):
    kind, *rest = blocks[i]
    if kind == "h1":
        txt = rest[0]
        p = doc.add_paragraph()
        if txt == refs_h1:
            add_rich(p, txt, 11, lang, bold_all=True, east_font=head_east)
            p.paragraph_format.space_before = Pt(10)
        elif txt == abstract_h1:
            add_rich(p, txt, 11, lang, bold_all=True, east_font=head_east)
        else:
            body_started = True
            add_rich(p, txt, 11, lang, bold_all=True, east_font=head_east)
            p.paragraph_format.space_before = Pt(8)
        p.paragraph_format.space_after = Pt(3)
        p.paragraph_format.keep_with_next = True
        i += 1
        continue
    if kind == "h2":
        p = doc.add_paragraph()
        add_rich(p, rest[0], 10, lang, bold_all=True, italic_all=(lang == "en"), east_font=head_east)
        p.paragraph_format.space_before = Pt(6)
        p.paragraph_format.space_after = Pt(2)
        p.paragraph_format.keep_with_next = True
        i += 1
        continue
    if kind == "p":
        txt = rest[0]
        in_abstract = (not body_started)
        is_kw = txt.startswith("**关键词") or txt.startswith("**Keywords")
        is_ref = bool(re.match(r"^\[\d+\]", txt))
        p = doc.add_paragraph()
        p.alignment = WD_ALIGN_PARAGRAPH.JUSTIFY
        p.paragraph_format.space_after = Pt(4)
        if in_abstract and not is_kw:
            p.paragraph_format.left_indent = Cm(1)
            p.paragraph_format.right_indent = Cm(1)
            add_rich(p, txt, 9, lang)
        elif is_kw:
            add_rich(p, txt, 9, lang)
        elif is_ref:
            p.paragraph_format.left_indent = Cm(0.75)
            p.paragraph_format.first_line_indent = Cm(-0.75)
            add_rich(p, txt, 8, lang)
        else:
            add_rich(p, txt, 10, lang)
        i += 1
        continue
    if kind == "fig":
        cap, rel = rest
        img = os.path.join(doc_dir, rel)
        p = doc.add_paragraph()
        p.alignment = WD_ALIGN_PARAGRAPH.CENTER
        p.add_run().add_picture(img, width=Cm(9.5))
        p.paragraph_format.space_after = Pt(2)
        p.paragraph_format.keep_with_next = True
        add_caption(doc, cap, lang, keep_next=False)
        i += 1
        continue
    if kind == "tbl":
        cap, hdr, rows, foot = spec[ti](lang)
        add_caption(doc, cap, lang)
        add_table(doc, hdr, rows, foot, lang, widths=WIDTHS.get((paper, ti)))
        ti += 1
        i += 1
        continue
    i += 1

out = os.path.join(OUT, f"paper{paper}_{'en' if lang == 'en' else 'cn'}.docx")
doc.save(out)
print("saved", out, "| tables:", ti)
```

```
# ---------------------------------------------------------------- 复现说明生成
def build_doc():
    md = os.path.join(DOCS, "配置要求与复现说明.md")
    lines = open(md, encoding="utf-8").read().splitlines()
    doc = Document()
    sec = doc.sections[0]
    sec.page_width, sec.page_height = Cm(21), Cm(29.7)
    sec.top_margin = sec.bottom_margin = Cm(2.2)
    sec.left_margin = sec.right_margin = Cm(2.2)

    i = 0
    first_title = True
    while i < len(lines):
        s = lines[i].rstrip()
        st = s.strip()
        if not st or st == "---":
            i += 1
            continue
        if st.startswith("```"):
            # 代码块
            i += 1
            code = []
            while i < len(lines) and not lines[i].strip().startswith("```"):
                code.append(lines[i])
                i += 1
            i += 1
            p = doc.add_paragraph()
            p.paragraph_format.left_indent = Cm(0.4)
            p.paragraph_format.space_after = Pt(6)
            r = p.add_run("\n".join(code))
            r.font.name = "Consolas"
            r.font.size = Pt(8)
            r._element.get_or_add_rPr()
            continue
        if st.startswith("|") and st.endswith("|"):
            # 管道表格
            rows = []
            while i < len(lines) and lines[i].strip().startswith("|"):
                cells = [c.strip() for c in lines[i].strip().strip("|").split("|")]
                if not all(re.fullmatch(r":?-{3,}:?", c) for c in cells):
                    rows.append(cells)
                i += 1
            if rows:
                hdr, data = rows[0], rows[1:]
                w = None
                if len(hdr) == 3 and hdr[-1] == "说明":
                    w = [3.2, 3.2, 10.2]
                elif len(hdr) == 4 and "SHA-256" in hdr[-1]:
                    w = [2.2, 5.8, 2.6, 6.0]
                elif len(hdr) == 3 and hdr[-1] == "95% CI":
                    w = [4.6, 5.0, 7.0]
                elif len(hdr) == 2:
                    w = [4.8, 11.8]
                add_table(doc, hdr, data, None, "cn", size=8, widths=w)
            continue
        if st.startswith("# "):
            p = doc.add_paragraph()
            p.alignment = WD_ALIGN_PARAGRAPH.CENTER
            add_rich(p, st[2:], 16 if first_title else 13, "cn", bold_all=True, east_font="黑体")
            p.paragraph_format.space_after = Pt(6)
            first_title = False
            i += 1
            continue
        if st.startswith("## "):
            p = doc.add_paragraph()
            add_rich(p, st[3:], 12, "cn", bold_all=True, east_font="黑体")
            p.paragraph_format.space_before = Pt(10)
            p.paragraph_format.space_after = Pt(3)
            i += 1
            continue
        if st.startswith("### "):
            p = doc.add_paragraph()
            add_rich(p, st[4:], 10.5, "cn", bold_all=True, east_font="黑体")
            p.paragraph_format.space_before = Pt(6)
            p.paragraph_format.space_after = Pt(2)
            i += 1
            continue
        if st.startswith("> "):
```

```
            p = doc.add_paragraph()
            p.paragraph_format.left_indent = Cm(0.8)
            add_rich(p, st[2:], 9, "cn", italic_all=True)
            p.paragraph_format.space_after = Pt(4)
            i += 1
            continue
        if st.startswith("- [ ] "):
            p = doc.add_paragraph(style="List Bullet")
            add_rich(p, "□ " + st[6:], 10, "cn")
            p.paragraph_format.space_after = Pt(1)
            i += 1
            continue
        if st.startswith("- "):
            p = doc.add_paragraph(style="List Bullet")
            add_rich(p, st[2:], 10, "cn")
            p.paragraph_format.space_after = Pt(1)
            i += 1
            continue
        if re.match(r"^\d+\.\s", st):
            p = doc.add_paragraph(style="List Number")
            add_rich(p, re.sub(r"^\d+\.\s", "", st), 10, "cn")
            p.paragraph_format.space_after = Pt(1)
            i += 1
            continue
        p = doc.add_paragraph()
        p.alignment = WD_ALIGN_PARAGRAPH.JUSTIFY
        add_rich(p, st, 10, "cn")
        p.paragraph_format.space_after = Pt(3)
        i += 1

    out = os.path.join(OUT, "配置要求与复现说明.docx")
    doc.save(out)
    print("saved", out)


if __name__ == "__main__":
    for p_ in ["A", "B", "C", "D", "E"]:
        for l_ in ["en", "cn"]:
            build_paper(p_, l_)
    build_doc()
```

D.2 to_pdf5.ps1（16 行）

用途

用本机 Word 的 COM 接口把 docx/ 下所有 docx 批量另存为 PDF（格式码 17）。只做格式转换，不改内容。

- **对应算法架构：**PDF 导出
- **输入：**docx/*.docx
- **输出：**docx/*.pdf

运行命令

```
powershell -ExecutionPolicy Bypass -File to_pdf5.ps1
```

口径提醒

- **注意：**需要本机安装 Microsoft Word。
- **注意：**转换前先关闭已打开的旧 PDF，避免文件占用。

代码全文

```
$ErrorActionPreference = 'Stop'
$src = 'C:\Users\SMYqy\D13_论文写作\07_五篇 SCI\docx'
$word = New-Object -ComObject Word.Application
$word.Visible = $false
$word.DisplayAlerts = 0
$ok = 0
Get-ChildItem $src -Filter *.docx | ForEach-Object {
    $doc = $word.Documents.Open($_.FullName, $false, $true)
    $pdf = [System.IO.Path]::ChangeExtension($_.FullName, '.pdf')
    $doc.SaveAs([ref]$pdf, [ref]17)
    $doc.Close($false)
    $ok++
    Write-Output ("OK " + $_.Name)
}
$word.Quit()
Write-Output ("DONE " + $ok)
```

D.3 build_resultsbook.py（1183 行）

用途

生成《卒中后癫痫五套算法架构_跑数结果汇编》docx + pdf。全部数字从 results/ 的 JSON/CSV 读取（144 次 A/B/C 拟合明细、n = 8,000 主结果与 CI、校准、增量、重要性、D/E 全部结果），不在脚本里手写任何指标值。

- **对应算法架构：**结果汇编
- **输入：**results/*.json、results/*_final.csv、results/figs/*.png、results/E/*
- **输出：**books/卒中后癫痫五套算法架构_跑数结果汇编.docx/.pdf

运行命令

```
py -3.12 build_resultsbook.py
```

口径提醒

- **注意：**本书是“账本”：每个表下有源文件脚注，可逐位核对。
- **注意：**修订跑数时先改 results/ 源文件再重跑本脚本，禁止手改表格。

代码全文

```
# -*- coding: utf-8 -*-
"""生成《卒中后癫痫五套算法架构_跑数结果汇编》docx + pdf。

全部数字从 results/ 的 JSON / CSV 读取，不在正文手写。
"""
import json
import os
import re

import pandas as pd
from docx import Document
from docx.enum.table import WD_CELL_VERTICAL_ALIGNMENT, WD_TABLE_ALIGNMENT
from docx.enum.text import WD_ALIGN_PARAGRAPH
from docx.oxml import OxmlElement
from docx.oxml.ns import qn
from docx.shared import Cm, Pt

BASE = os.path.dirname(os.path.abspath(__file__))
RES = os.path.join(BASE, "results")
FIG = os.path.join(RES, "figs")
OUT = os.path.join(BASE, "books")
os.makedirs(OUT, exist_ok=True)

SUMMARY = json.load(open(os.path.join(RES, "bench_summary_final.json"), encoding="utf-8"))["summary"]
TN = json.load(open(os.path.join(RES, "table_numbers.json"), encoding="utf-8"))
REST = json.load(open(os.path.join(RES, "restore", "restore.json"), encoding="utf-8"))
CI = json.load(open(os.path.join(RES, "restore", "ci.json"), encoding="utf-8"))
SD = json.load(open(os.path.join(RES, "summary_D.json"), encoding="utf-8"))
SE = json.load(open(os.path.join(RES, "E", "summary_E.json"), encoding="utf-8"))
TW = json.load(open(os.path.join(RES, "table1_wenzhou.json"), encoding="utf-8"))
SMD = pd.read_csv(os.path.join(RES, "E", "smd_balance.csv"))

MET = {}
for _fs in "ABC":
    for _tag in ("full", "dedup"):
        MET[(_fs, _tag)] = pd.read_csv(os.path.join(RES, f"metrics_{_tag}_{_fs}_final.csv"))

MODEL = {"lr": "LR", "lgbm": "LGBM", "tabpfn": "TabPFN"}
CN = {
    "age": "年龄", "gender": "性别（男=1）", "nihss": "NIHSS 评分", "uremia": "尿毒症",
    "fatty_liver": "脂肪肝", "diabetes": "糖尿病", "hypertension": "高血压",
    "coronary_disease": "冠心病", "atrial_fibrillation": "心房颤动",
    "hyperuricemia": "高尿酸血症", "hyperlipidaemia": "高脂血症",
    "frontal_lobe": "额叶", "parietal_lobe": "顶叶", "temporal_lobe": "颞叶",
    "occipital_lobe": "枕叶", "insular_lobe": "岛叶", "range_lobe": "多叶",
    "basal_ganglia": "基底节", "capsula_interna": "内囊", "brainstem": "脑干",
    "epencephalon": "小脑", "paraventricular": "脑室旁", "centrum_semiovale": "半卵圆中心",
    "thalamus": "丘脑", "subcortex_lobe": "皮层下", "aca": "大脑前动脉供血区",
    "mca": "大脑中动脉供血区", "pca": "大脑后动脉供血区", "va": "椎动脉供血区",
    "ba": "基底动脉供血区", "cca_plaque": "颈总动脉斑块", "ica_plaque": "颈内动脉斑块",
    "eca_plaque": "颈外动脉斑块", "ant_circle": "前循环", "post_circle": "后循环",
    "large_ves_as": "大血管病变", "plt": "血小板", "wbc": "白细胞", "rbc": "红细胞",
    "hba1c": "糖化血红蛋白", "crp": "C 反应蛋白", "tg": "甘油三酯", "ldl": "LDL 胆固醇",
    "hdl": "HDL 胆固醇", "ast": "谷草转氨酶", "alt": "谷丙转氨酶", "bilirubin": "总胆红素",
```

```
    "albumin": "白蛋白", "urea": "尿素", "creatinine": "肌酐", "bua": "血尿酸",
    "pt": "凝血酶原时间", "aptt": "APTT", "tt": "凝血酶时间", "inr": "INR",
    "d_dimer": "D-二聚体", "fibrinogen": "纤维蛋白原", "ck": "肌酸激酶", "ck_mb": "CK-MB",
    "ldh": "乳酸脱氢酶", "hbdh": "α-羟丁酸脱氢酶", "ima": "缺血修饰白蛋白",
    "na": "钠", "k": "钾", "cl": "氯", "ca": "钙", "p": "磷", "lactate": "乳酸",
    "anion_gap": "阴离子间隙", "tco2": "总二氧化碳", "dvt": "深静脉血栓",
    "cerebral_hernia": "脑疝", "hydrocephalus": "脑积水", "hypoproteinemia": "低蛋白血症",
}


# ---------------------------------------------------------------- 排版工具
def set_font(run, size, bold=False, italic=False, east=None, color=None, mono=False):
    run.font.name = "Consolas" if mono else "Times New Roman"
    run.font.size = Pt(size)
    run.font.bold = bold
    run.font.italic = italic
    if color:
        run.font.color.rgb = color
    rPr = run._element.get_or_add_rPr()
    rFonts = rPr.find(qn("w:rFonts"))
    if rFonts is None:
        rFonts = OxmlElement("w:rFonts")
        rPr.append(rFonts)
    rFonts.set(qn("w:eastAsia"), east or ("宋体" if not mono else "宋体"))


def add_rich(p, text, size, bold_all=False, italic_all=False, east=None, color=None):
    parts = re.split(r"(\*\*.*?\*\*|`[^`]*`)", text)
    for part in parts:
        if not part:
            continue
        if part.startswith("**") and part.endswith("**") and len(part) > 4:
            r = p.add_run(part[2:-2])
            set_font(r, size, bold=True, italic=italic_all, east=east, color=color)
        elif part.startswith("`") and part.endswith("`") and len(part) > 2:
            r = p.add_run(part[1:-1])
            set_font(r, size, bold=bold_all, italic=italic_all, color=color, mono=True)
        else:
            r = p.add_run(part)
            set_font(r, size, bold=bold_all, italic=italic_all, east=east, color=color)


def para(doc, text="", size=9.5, bold=False, italic=False, align=WD_ALIGN_PARAGRAPH.JUSTIFY,
         before=0, after=4, indent=None, keep=False, east=None, color=None):
    p = doc.add_paragraph()
    p.alignment = align
    pf = p.paragraph_format
    pf.space_before = Pt(before)
    pf.space_after = Pt(after)
    if indent:
        pf.first_line_indent = Cm(indent)
    if keep:
        pf.keep_with_next = True
    if text:
        add_rich(p, text, size, bold_all=bold, italic_all=italic, east=east, color=color)
    return p


def h1(doc, text, page=True):
    if page:
        doc.add_page_break()
    p = doc.add_paragraph()
    p.paragraph_format.space_before = Pt(0)
    p.paragraph_format.space_after = Pt(6)
    p.paragraph_format.keep_with_next = True
    r = p.add_run(text)
    set_font(r, 15, bold=True, east="黑体")
    return p


def h2(doc, text):
    p = doc.add_paragraph()
    p.paragraph_format.space_before = Pt(9)
    p.paragraph_format.space_after = Pt(3)
    p.paragraph_format.keep_with_next = True
    r = p.add_run(text)
    set_font(r, 11.5, bold=True, east="黑体")
    return p
```

```
def cap(doc, text, before=6, after=3, keep=True):
    p = doc.add_paragraph()
    p.alignment = WD_ALIGN_PARAGRAPH.CENTER
    p.paragraph_format.space_before = Pt(before)
    p.paragraph_format.space_after = Pt(after)
    p.paragraph_format.keep_with_next = keep
    m = re.match(r"^((?:图|表)\s*\d+-\d+)\s*(.*)$", text)
    if m:
        r = p.add_run(m.group(1) + "  ")
        set_font(r, 8.8, bold=True)
        r2 = p.add_run(m.group(2))
        set_font(r2, 8.8, bold=True)
    else:
        r = p.add_run(text)
        set_font(r, 8.8, bold=True)
    return p


def _tr_prop(row, tag):
    trPr = row._tr.get_or_add_trPr()
    el = OxmlElement(tag)
    trPr.append(el)


def shade(cell, fill="F2F2F2"):
    tcPr = cell._tc.get_or_add_tcPr()
    shd = OxmlElement("w:shd")
    shd.set(qn("w:val"), "clear")
    shd.set(qn("w:color"), "auto")
    shd.set(qn("w:fill"), fill)
    tcPr.append(shd)


def table(doc, hdr, rows, widths, size=7.9, foot=None, first_left=True):
    t = doc.add_table(rows=1, cols=len(hdr))
    t.style = "Table Grid"
    t.alignment = WD_TABLE_ALIGNMENT.CENTER
    t.autofit = False
    _tr_prop(t.rows[0], "w:tblHeader")
    for j, w in enumerate(widths):
        t.columns[j].width = Cm(w)
        c = t.rows[0].cells[j]
        c.width = Cm(w)
        c.vertical_alignment = WD_CELL_VERTICAL_ALIGNMENT.CENTER
        shade(c)
        c.paragraphs[0].alignment = WD_ALIGN_PARAGRAPH.CENTER
        r = c.paragraphs[0].add_run(str(hdr[j]))
        set_font(r, size, bold=True)
    for row in rows:
        tr = t.add_row()
        _tr_prop(tr, "w:cantSplit")
        cells = tr.cells
        for j, w in enumerate(widths):
            cells[j].width = Cm(w)
        if isinstance(row, tuple) and row and row[0] == "__SECTION__":
            merged = cells[0].merge(cells[-1])
            merged.paragraphs[0].alignment = WD_ALIGN_PARAGRAPH.LEFT
            r = merged.paragraphs[0].add_run(row[1])
            set_font(r, size, bold=True)
            shade(merged, "FAFAFA")
            continue
        for j, v in enumerate(row):
            c = cells[j]
            c.vertical_alignment = WD_CELL_VERTICAL_ALIGNMENT.CENTER
            c.paragraphs[0].alignment = (
                WD_ALIGN_PARAGRAPH.LEFT if (j == 0 and first_left) else WD_ALIGN_PARAGRAPH.CENTER
            )
            add_rich(c.paragraphs[0], str(v), size)
    if len(rows) <= 26:
        for tr in t.rows[:-1]:
            for c in tr.cells:
                for pp in c.paragraphs:
                    pp.paragraph_format.keep_with_next = True
    if foot:
        p = doc.add_paragraph()
        r = p.add_run(foot)
        set_font(r, 7.4, italic=True)
        p.paragraph_format.space_before = Pt(2)
```

```
        p.paragraph_format.space_after = Pt(8)


def size_table(doc, data, foot=None):
    """两级表头的规模明细表：n × 3 模型 × 3 指标。"""
    widths = [1.55] + [1.74] * 9
    t = doc.add_table(rows=2, cols=10)
    t.style = "Table Grid"
    t.alignment = WD_TABLE_ALIGNMENT.CENTER
    t.autofit = False
    for j, w in enumerate(widths):
        t.columns[j].width = Cm(w)
    for a, b, lab in [(1, 3, "Logistic regression"), (4, 6, "LightGBM"), (7, 9, "TabPFN v2")]:
        c = t.cell(0, a).merge(t.cell(0, b))
        c.paragraphs[0].alignment = WD_ALIGN_PARAGRAPH.CENTER
        shade(c)
        r = c.paragraphs[0].add_run(lab)
        set_font(r, 7.6, bold=True)
    c0 = t.cell(0, 0)
    shade(c0)
    c0.paragraphs[0].add_run("")
    hdr1 = ["n_train", "AUROC", "AUPRC", "Brier", "AUROC", "AUPRC", "Brier",
            "AUROC", "AUPRC", "Brier"]
    for j, h in enumerate(hdr1):
        c = t.cell(1, j)
        c.vertical_alignment = WD_CELL_VERTICAL_ALIGNMENT.CENTER
        c.paragraphs[0].alignment = WD_ALIGN_PARAGRAPH.CENTER
        shade(c)
        r = c.paragraphs[0].add_run(h)
        set_font(r, 7.6, bold=True)
    for j in range(10):
        t.cell(1, j).width = Cm(widths[j])
    for n in sorted(data.n_train.unique()):
        tr = t.add_row()
        _tr_prop(tr, "w:cantSplit")
        cells = tr.cells
        for j, w in enumerate(widths):
            cells[j].width = Cm(w)
        cells[0].paragraphs[0].alignment = WD_ALIGN_PARAGRAPH.CENTER
        add_rich(cells[0].paragraphs[0], f"{int(n):,}", 7.6)
        for k, m in enumerate(["lr", "lgbm", "tabpfn"]):
            r = data[(data.n_train == n) & (data.model == m)]
            vals = ["—", "—", "—"] if len(r) == 0 else [
                f"{r.iloc[0].auroc:.4f}", f"{r.iloc[0].auprc:.4f}", f"{r.iloc[0].brier:.4f}"
            ]
            for q, v in enumerate(vals):
                c = cells[1 + 3 * k + q]
                c.paragraphs[0].alignment = WD_ALIGN_PARAGRAPH.CENTER
                add_rich(c.paragraphs[0], v, 7.6)
    for tr in t.rows[:2]:
        _tr_prop(tr, "w:tblHeader")
    if foot:
        p = doc.add_paragraph()
        r = p.add_run(foot)
        set_font(r, 7.4, italic=True)
        p.paragraph_format.space_before = Pt(2)
        p.paragraph_format.space_after = Pt(8)


def figure(doc, filename, caption, width=12.3):
    path = filename if os.path.isabs(filename) else os.path.join(FIG, filename)
    if not os.path.exists(path):
        path = os.path.join(RES, os.path.basename(filename))
    if not os.path.exists(path):
        path = os.path.join(RES, "E", os.path.basename(filename))
    doc.add_picture(path, width=Cm(width))
    p = doc.paragraphs[-1]
    p.alignment = WD_ALIGN_PARAGRAPH.CENTER
    p.paragraph_format.keep_with_next = True
    p.paragraph_format.space_before = Pt(4)
    p.paragraph_format.space_after = Pt(2)
    cap(doc, caption, before=0, after=7, keep=False)


def add_page_number(section):
    p = section.footer.paragraphs[0]
    p.alignment = WD_ALIGN_PARAGRAPH.CENTER
    r1 = p.add_run("第 ")
    set_font(r1, 8)
```

```
    run = p.add_run()
    set_font(run, 8)
    f1 = OxmlElement("w:fldChar"); f1.set(qn("w:fldCharType"), "begin")
    it = OxmlElement("w:instrText"); it.set(qn("xml:space"), "preserve"); it.text = "PAGE"
    f2 = OxmlElement("w:fldChar"); f2.set(qn("w:fldCharType"), "separate")
    tt = OxmlElement("w:t"); tt.text = "1"
    f3 = OxmlElement("w:fldChar"); f3.set(qn("w:fldCharType"), "end")
    for el in (f1, it, f2, tt, f3):
        run._r.append(el)
    r2 = p.add_run(" 页")
    set_font(r2, 8)


# ---------------------------------------------------------------- 数字 helper
def f4(x):
    return f"{x:.4f}"


def sci(x):
    if x < 1e-3:
        e = int(f"{x:.0e}".split("e")[1])
        m = x / (10 ** e)
        sup = str(-e) if e < 0 else str(e)
        trans = str.maketrans("0123456789-", "⁰¹²³⁴⁵⁶⁷⁸⁹⁻")
        return f"{m:.1f}×10{sup.translate(trans)}"
    return f"{x:.4f}"


def mval(fs, tag, n, model, field="auroc"):
    d = MET[(fs, tag)]
    r = d[(d.n_train == n) & (d.model == model)]
    if len(r) == 0:
        return None
    return float(r.iloc[0][field])


def mrow_best(fs, tag):
    d = MET[(fs, tag)]
    r = d.sort_values("auroc", ascending=False).iloc[0]
    return r.model, int(r.n_train), float(r.auroc)


def dmet(fs, metric, model, stat="mean"):
    return SD["feature_sets"][fs]["metrics_mean_std"][f"('{metric}', '{stat}')"][model]


def med_iqr(v):
    return f"{v[0]:g}（{v[1]:g}–{v[2]:g}）"


def pct(v):
    return f"{v:.1f}%"


def sgn(x, nd=4):
    return f"{x:+.{nd}f}"


def ci_str(v, nd=4, signed=False):
    f = (lambda z: f"{z:+.{nd}f}") if signed else (lambda z: f"{z:.{nd}f}")
    return f"{f(v[0])}（{f(v[1])}, {f(v[2])}）"


def ci95(obj, metric):
    return f"{obj[metric]:.4f}（{obj['ci'][metric][0]:.4f}, {obj['ci'][metric][1]:.4f}）"


# ================================================================== 正文
def cover(doc):
    para(doc, "", 10, before=150)
    para(doc, "卒中后癫痫五套算法架构", 26, bold=True, align=WD_ALIGN_PARAGRAPH.CENTER, after=8, east="黑体")
    para(doc, "第五次大跑数 · 全量结果汇编", 15, align=WD_ALIGN_PARAGRAPH.CENTER, after=22)
    para(doc, "A / B / C：21,459 例公共队列（Dryad v8）", 10.5, align=WD_ALIGN_PARAGRAPH.CENTER, after=2)
    para(doc, "D / E：温州 167 例真实队列", 10.5, align=WD_ALIGN_PARAGRAPH.CENTER, after=26)
    para(doc, "三合医研发团队 · 河南大学", 11, align=WD_ALIGN_PARAGRAPH.CENTER, after=6)
    para(doc, "2026-11-07", 10, align=WD_ALIGN_PARAGRAPH.CENTER, after=40)
    para(doc, "全部数字直接读取 results/ 原始 JSON 与 CSV，未做人工转录或修饰", 9,
         italic=True, align=WD_ALIGN_PARAGRAPH.CENTER, after=2)
    para(doc, "每个表、每张图均注明源文件，可逐位核对", 9,
```

```
          italic=True, align=WD_ALIGN_PARAGRAPH.CENTER)


def toc(doc):
    doc.add_page_break()
    p = doc.add_paragraph()
    p.alignment = WD_ALIGN_PARAGRAPH.CENTER
    r = p.add_run("目　录")
    set_font(r, 16, bold=True, east="黑体")
    p.paragraph_format.space_after = Pt(14)
    items = [
        ("0", "阅读指南：这本书是什么、怎么核对"),
        ("1", "数据与设计：队列、特征集、模型、跑数清单"),
        ("2", "A / B / C 全训练规模明细（144 次拟合）"),
        ("3", "主结果：n = 8,000 与全训练集锚点"),
        ("4", "校准与决策曲线"),
        ("5", "增量与配对比较：哪个特征块、哪个模型真正有用"),
        ("6", "特征重要性：A / B / C 的前十变量"),
        ("7", "特征集 A 的五折交叉验证与 OOF"),
        ("8", "D 研究：温州 167 例的首次发作时点预测"),
        ("9", "E 研究：抗癫痫药物与晚期 PSE 风险"),
        ("10", "图表总索引"),
        ("11", "复现锚点：文件指纹、关键数字、运行步骤"),
    ]
    for no, title in items:
        p = doc.add_paragraph()
        p.paragraph_format.space_before = Pt(2)
        p.paragraph_format.space_after = Pt(2)
        r = p.add_run(f"{no}  ")
        set_font(r, 10.5, bold=True)
        r2 = p.add_run(title)
        set_font(r2, 10.5)


def sec0(doc):
    h1(doc, "0  阅读指南")
    para(doc, "这份汇编是五套算法架构的账本。算法架构正文回答“这些数字说明什么”，本书回答四个更基础的问题："
              "每个数字具体是多少；它在哪一种数据划分、哪一档训练规模、哪一个模型下得到；"
              "换一种划分是否还成立；以及这个数字可以在哪个原始文件里逐位核对。")
    para(doc, "本书覆盖第五次大跑数的全部最终结果：公共队列 A/B/C 三项研究共 **144 次模型拟合**；"
              "温州 167 例队列 D 研究 **300 次交叉验证拟合**；E 研究 **5 类效应估计、2,000 次 bootstrap** "
              "与全部生存分析。所有数字由脚本直接写入 JSON/CSV，本书再由脚本读取生成，中间没有人工转录。")
    para(doc, "一条阅读建议：先看第 3 节主结果表（n = 8,000），再看第 5 节增量表；这两张表足以回答"
              "“哪个模型更好、哪个特征块有用”。第 2 节的全量明细用于核对与复现，不必顺序阅读。")

    h2(doc, "0.1  数据流（一页看懂）")
    for t in [
        "公共队列 21,459 例（事件 936，4.36%）→ 分层 80/20 划分（种子 42）→ 训练 17,167 / 测试 4,292（事件 187）。",
        "去重臂：删除 7,815 条所有字段完全相同的重复行 → 13,644 例（事件 892，6.54%）→ 训练 10,915 / 测试 2,729（事件 178）。",
        "特征集：A = 36 个入院特征；B = 70 个（A + 34 项实验室指标）；C = 74 个（B + 4 项卒中后并发症）。",
        "模型：L2 logistic 回归（LR）、LightGBM（LGBM）、TabPFN v2（预训练检查点，零调参）。",
        "训练规模：100、250、500、1,000、2,000、4,000、8,000、16,000、17,167；去重臂最后一档为 10,915；TabPFN 受上下文上限约束，最大 8,000。",
        "温州队列 167 例（PSE 49 例，29.3%）→ D：核心 8 特征 / 扩展 24 特征，5 折 × 10 重复；E：ASM 暴露（134 vs 33）的观察性比较。",
    ]:
        para(doc, "· " + t, 9.3, after=3, indent=0.0)

    h2(doc, "0.2  记号与缩写")
    cap(doc, "表 0-1  本书反复出现的术语")
    table(doc, ["术语", "含义"], [
        ["AUROC", "受试者工作特征曲线下面积：0.5 = 随机猜测，1 = 完美排序"],
        ["AUPRC", "精确率–召回率曲线下面积（average precision）；事件稀少时比 AUROC 更能区分模型"],
        ["Brier 分数", "预测概率与真实结局（0/1）的均方误差；越低越好，同时惩罚判别力与校准"],
        ["校准斜率", "把预测 logit 对真实结局做 logistic 回归的斜率：1 = 校准良好，< 1 = 概率被压缩/过度自信"],
        ["校准截距", "同一回归的截距：0 = 无系统性高估或低估"],
        ["ΔAUROC", "同一批测试患者上两个模型的 AUROC 之差；配对 bootstrap 2,000 次给出 95% 区间"],
        ["n_train", "训练子集例数（每一档都从训练集重新抽样，测试集始终不变）"],
        ["full / dedup", "原始全队列 / 去除完全重复行后的去重队列"],
        ["LR / LGBM / TabPFN", "logistic 回归 / LightGBM / TabPFN v2 表格基础模型"],
        ["E 值", "未测量混杂需要达到多强的关联，才能把观察到的效应完全解释掉"],
        ["SMD", "标准化均值差：绝对值 < 0.1 通常视为两组平衡"],
        ["IPTW / AIPW", "逆概率加权 / 增广逆概率加权，两类倾向评分方法"],
        ["RMST", "限制平均生存时间：在固定时间窗（此处 730 天）内的平均无事件时间"],
    ], [3.6, 13.6], size=8.2, foot="源文件：术语定义对应 docs/配置要求与复现说明.md 与各算法架构方法节。")

    h2(doc, "0.3  怎么核对")
    for t in [
        "每个表下都有“源文件”脚注；打开 results/ 下对应文件即可逐位核对。",
```

```
        "重跑后，用第 11 节的锚点表比对到小数点后四位；任何一位不同都说明环境或数据版本不一致。",
        "图中曲线由 preds_*.csv 的行级预测重绘；若图与表不一致，以表为准，并检查 preds 文件是否为 final 版本。",
        "本书的“第五次”沿用项目内部计数；不同次跑数的口径可能不同，跨版本比较请只使用本书数字。",
    ]:
        para(doc, "· " + t, 9.3, after=3)


def sec1(doc):
    h1(doc, "1  数据与设计")
    h2(doc, "1.1  队列与划分")
    para(doc, "公共队列来自 Dryad 公开的缺血性卒中数据集（第 8 版），共 21,459 例，一年内继发性癫痫 936 例（4.36%）。"
              "固定分层 80/20 划分（种子 42）产生 17,167 例训练集与 4,292 例测试集；测试集含 187 个事件。"
              "去重臂删除 7,815 条所有字段完全相同的重复记录，剩 13,644 例（事件 892，6.54%），"
              "训练 10,915 / 测试 2,729（事件 178）。")
    para(doc, "一个值得注意的事实：被删除的 7,815 行里只有 44 个事件（0.56%），远低于全队列的 4.36%——"
              "重复记录以阴性随访为主。这正是去重臂要检验的：如果结论依赖这些重复的阴性记录，去重后应当改变。"
              "第 2–5 节会看到，结论没有改变，数值只发生了小幅移动。")
    cap(doc, "表 1-1  两套数据划分")
    table(doc, ["队列", "总例数", "事件数", "事件率", "训练例数", "测试例数", "测试事件"], [
        ["full（原始）", "21,459", "936", "4.36%", "17,167", "4,292", "187"],
        ["dedup（去重）", "13,644", "892", "6.54%", "10,915", "2,729", "178"],
    ], [3.8, 2.6, 2.2, 2.2, 2.6, 2.6, 2.4], size=8.4,
        foot="源文件：results/metrics_{full,dedup}_{A,B,C}_final.csv 的 n_train 网格；事件率 = 事件数 / 总例数。")

    h2(doc, "1.2  三套特征集")
    para(doc, "A/B/C 共用同一套划分、同一批测试患者与同一组模型，唯一变化是特征块。这是本系列能够做"
              "“同一患者上的配对增量比较”的原因：A→B 的差值就是实验室块的贡献，B→C 的差值就是并发症块的贡献。")
    cap(doc, "表 1-2  三套特征集的构成")
    table(doc, ["特征集", "变量数", "构成", "回答的问题"], [
        ["A", "36", "人口学（年龄、性别）、血管危险因素与病史、卒中病灶部位（脑叶/基底节/脑干/小脑等）、供血区与颈动脉斑块、大血管病变", "仅凭入院即可得的信息能预测到什么程度"],
        ["B", "70", "A 的 36 个 + 34 项实验室指标（血常规、凝血、生化、心肌酶、电解质等）", "实验室块带来多少增量"],
        ["C", "74", "B 的 70 个 + 4 项卒中后并发症（深静脉血栓、脑疝、脑积水、低蛋白血症）", "后验变量还能不能带来增量（边界测试）"],
    ], [1.6, 1.8, 10.4, 3.6], size=8.0,
        foot="源文件：server/tabpfn_bench.py 的 FEATURES_A/B/C 定义；D 研究另有核心 8 特征与扩展 24 特征，见 8.1 节。")

    h2(doc, "1.3  三个模型")
    para(doc, "三个模型代表三种归纳偏置：线性加性（LR）、树集成（LGBM）、预训练 Transformer（TabPFN）。"
              "三者使用完全相同的输入列与训练行，不做任何按模型的额外特征工程。")
    cap(doc, "表 1-3  A / B / C 的模型配置")
    table(doc, ["模型", "实现", "关键设置", "说明"], [
        ["LR", "scikit-learn LogisticRegression", "C = 1.0；max_iter = 3,000；random_state = 42", "实验室值按训练集 1/99 百分位缩尾；13 个偏态实验室变量 log1p；连续与有序变量标准化；二值变量原值。所有变换参数只来自训练行。"],
        ["LGBM", "LightGBM LGBMClassifier", "n_estimators = 300；learning_rate = 0.05；num_leaves = 31；min_child_samples = 50；subsample = 0.8；colsample_bytree = 0.8；reg_alpha = 0.1；reg_lambda = 0.1；scale_pos_weight = 阴性/阳性", "沿用源研究超参数，未在本数据上调参；随机种子 42。"],
        ["TabPFN", "TabPFN v2（预训练检查点）", "n_estimators = \"auto\"；ignore_pretraining_limits = True；device = cpu；random_state = 42", "零调参；仅在 n ≤ 8,000 时运行。检查点 SHA-256 见 11.1。"],
    ], [1.6, 4.2, 6.2, 5.2], size=7.8,
        foot="源文件：server/tabpfn_bench.py 第 130–165 行；预处理函数 lr_transform()。")
    para(doc, "D 研究使用不同配置：LR 的 max_iter = 5,000；LGBM 改为浅树（n_estimators = 100、num_leaves = 3、"
              "min_child_samples = 10），因为 n = 167 时深树会直接过拟合；TabPFN 配置不变。E 不是机器学习研究，"
              "而是观察性效应估计，方法见第 9 节。", 9.2, after=4)

    h2(doc, "1.4  跑数清单")
    para(doc, "下表是本次归档包含的全部拟合次数。A/B/C 的每一次拟合都产生一行指标（结果 CSV 共 144 行）；"
              "D 的 300 次拟合汇总为均值±标准差；E 的每次 bootstrap 重抽样都记录估计值，用于构造区间。")
    cap(doc, "表 1-4  本次跑数的拟合次数")
    table(doc, ["研究", "数据", "模型 × 规模", "拟合次数"], [
        ["A/B/C full", "17,167 训练 / 4,292 测试", "LR 9 档 + LGBM 9 档 + TabPFN 7 档 = 25 / 特征集", "75"],
        ["A/B/C dedup", "10,915 训练 / 2,729 测试", "LR 8 档 + LGBM 8 档 + TabPFN 7 档 = 23 / 特征集", "69"],
        ["D", "167 例，5 折 × 10 重复", "3 模型 × 2 特征集 × 50 折次", "300"],
        ["E", "167 例", "5 类效应估计 + 2,000 次 bootstrap + 生存分析", "—"],
    ], [3.2, 4.2, 7.2, 2.4], size=8.2,
        foot="合计 A/B/C 拟合 144 次。TabPFN 的 7 档为 100–8,000；16,000 与 17,167（或 10,915）档不运行。")

    h2(doc, "1.5  统计协议")
    para(doc, "判别力用 AUROC 与 AUPRC；概率质量用 Brier 分数；校准用“预测 logit 对真实结局”的 logistic 回归"
              "（C = 1e6，固定）给出斜率与截距。所有区间来自测试行上的 **2,000 次配对 bootstrap**（种子 42，"
              "百分位法）；配对意味着每次重抽样都同时抽取两个模型的同一批患者，因此区间反映的是“同一批人上的差”，"
              "而不是两个独立样本的比较。决策曲线在 0.05–0.60 的阈值网格上以 0.01 为步长计算净获益。"
              "D 研究另用 5 折 × 10 重复的分层交叉验证、Kaplan–Meier 三分位与 log-rank 检验；E 研究用 "
              "bootstrap 区间、E 值与 90 天界标分析。", 9.3)


def sec2(doc):
    h1(doc, "2  A / B / C 全训练规模明细")
    para(doc, "本节把 144 次拟合的每一条结果原样列出。每一行是一个训练规模；三组列分别是三个模型；"
```

```
            "每组内是 AUROC、AUPRC 与 Brier。“–”表示该模型在该规模没有运行（TabPFN 的预训练上下文上限为 8,000）。"
            "表格只做四位小数取舍，没有其他加工。")
    para(doc, "读表时注意三件事：第一，测试集始终固定，因此同一列内不同 n 的差异只来自训练集大小；第二，"
            "AUPRC 的随机基线约等于事件率（full 4.36%，dedup 6.52%），不是 0.5；第三，Brier 同时受判别力与"
            "校准影响，不能单独当作“准确率”。", 9.2, after=4)

    blocks = [
        ("A", "特征集 A（36 个入院特征）", "这是信息最少的一套特征：只有入院当时可得的人口学、病史、病灶与血管信息。"),
        ("B", "特征集 B（70 个：A + 34 项实验室）", "实验室块把信息量推高一档；这也是本系列的主推配置。"),
        ("C", "特征集 C（74 个：B + 4 项并发症）", "并发症块位于预测时点之后，本应不带来增量；这张表是验证。"),
    ]
    tno = 1
    for fs, title, intro in blocks:
        h2(doc, f"2.{'ABC'.index(fs) + 1}  {title}")
        para(doc, intro, 9.2, after=4)
        for tag, label in [("full", "全队列"), ("dedup", "去重队列")]:
            cap(doc, f"表 2-{tno}  特征集 {fs}，{label}：全部训练规模的指标（n = {MET[(fs, tag)].n_train.min():,}–{MET[(fs,
tag)].n_train.max():,}）")
            size_table(doc, MET[(fs, tag)],
                       foot=("源文件：results/metrics_%s_%s_final.csv。n 为训练例数；测试集固定为 "
                             "4,292 例/187 事件（full）或 2,729 例/178 事件（dedup）。" % (tag, fs)))
            tno += 1

        # 汇总段落（从数据生成，避免手抄）
        d_f = MET[(fs, "full")]
        n0, nmax_f = int(d_f.n_train.min()), int(d_f.n_train.max())
        d_d = MET[(fs, "dedup")]
        nmax_d = int(d_d.n_train.max())
        best_m, best_n, best_v = mrow_best(fs, "full")
        para(doc,
             f"全队列：LR 从 n = {n0:,} 的 {mval(fs,'full',n0,'lr'):.4f} 升到 n = {nmax_f:,} 的 "
             f"{mval(fs,'full',nmax_f,'lr'):.4f}；LGBM 从 {mval(fs,'full',n0,'lgbm'):.4f} 升到 "
             f"{mval(fs,'full',nmax_f,'lgbm'):.4f}；TabPFN 在 n = {n0:,} 已达 {mval(fs,'full',n0,'tabpfn'):.4f}，"
             f"n = 8,000 为 {mval(fs,'full',8000,'tabpfn'):.4f}。全表最高值出现在 {MODEL[best_m]}、"
             f"n = {best_n:,}（{best_v:.4f}）。去重队列（n = {int(d_d.n_train.min()):,}–{nmax_d:,}）的形态相同："
             f"LR {mval(fs,'dedup',int(d_d.n_train.min()),'lr'):.4f} → {mval(fs,'dedup',nmax_d,'lr'):.4f}；"
             f"LGBM {mval(fs,'dedup',int(d_d.n_train.min()),'lgbm'):.4f} → {mval(fs,'dedup',nmax_d,'lgbm'):.4f}；"
             f"TabPFN {mval(fs,'dedup',int(d_d.n_train.min()),'tabpfn'):.4f} → {mval(fs,'dedup',8000,'tabpfn'):.4f}。",
             9.1, before=4, after=6)

    para(doc, "把六张表放在一起看，能读出三条跨特征集的稳定规律。第一，LR 的学习曲线最平：从 100 例到全量，"
              "涨幅约 0.09–0.24 AUROC，而 LGBM 在 100 例时几乎等于随机（0.5000–0.6626），到全量才追上来。"
              "第二，TabPFN 的曲线在前段最高、后段趋平：100 例时比 LGBM 高 0.24–0.38，8,000 例时与 LGBM 的差距"
              "缩到 0.001–0.007。第三，去重没有改变排序，只把绝对水平移动了 –0.002 到 +0.004，"
              "说明重复记录不是结论的来源。", 9.2, before=2)
    para(doc, "唯一的例外值得单独指出：在特征集 A 上，LGBM 用 16,000 例达到 0.9464，略高于 TabPFN 在 8,000 例上限的 0.9445。"
              "这不是“树模型更好”，而是 TabPFN 被上下文长度限制在 8,000——给它同样的 16,000 例，比较才公平，"
              "而当前实现做不到。B/C 上 TabPFN 在 8,000 例已经足够领先，所以这个上限不构成实际限制；"
              "在信息较少的 A 上，它恰好落在临界点。", 9.2, after=4)


def sec3(doc):
    h1(doc, "3  主结果：n = 8,000 与全训练集锚点")
    para(doc, "如果只看一张表，就看表 3-1。它给出 n = 8,000（三模型共同可用的最大规模）下，"
              "A/B/C × full/dedup × 三模型的 AUROC 与 2,000 次配对 bootstrap 的 95% 区间。"
              "区间来自同一批测试患者的配对重抽样，因此可以直接比较重叠程度。")
    cap(doc, "表 3-1  n = 8,000 的主结果（AUROC / AUPRC / Brier，括号内为 95% CI）")
    rows = []
    for fs in "ABC":
        for tag, label in [("full", "全队列"), ("dedup", "去重队列")]:
            rows.append(("__SECTION__", f"特征集 {fs} · {label}"))
            for m in ["lr", "lgbm", "tabpfn"]:
                o = CI[f"{fs}_{tag}_8000"][m]
                slope = SUMMARY[f"{fs}_{tag}"]["p_" + m]["cal_slope"]
                rows.append([MODEL[m], ci95(o, "auroc"), ci95(o, "auprc"), ci95(o, "brier"),
                             f"{slope:.3f}"])
    table(doc, ["模型", "AUROC（95% CI）", "AUPRC（95% CI）", "Brier（95% CI）", "校准斜率"], rows,
          [1.6, 4.3, 4.3, 4.3, 2.4], size=8.0,
          foot="源文件：results/restore/ci.json（区间）与 bench_summary_final.json（斜率）。测试集：full 4,292 例/187 事件；
dedup 2,729 例/178 事件。")

    para(doc, "三条读数。第一，在 B/C 上 TabPFN 的 AUROC 为 0.9980 / 0.9982（full），区间下界 0.9954 / 0.9964，"
              "已经贴近该测试集可分辨的上限；LGBM 紧随其后（0.9923 / 0.9912），LR 明显更低（0.9625 / 0.9669）。"
              "第二，在 A 上三模型 closer：TabPFN 0.9445、LGBM 0.9378、LR 0.8553；TabPFN 对 LGBM 的配对差为 "
              "+0.0067（–0.0021, +0.0159），区间跨零——在只有 36 个入院特征时，两者的差别不足以在 4,292 例测试集上"
              "稳定检出。第三，去重后排序不变：A 为 TabPFN 0.9309 > LGBM 0.9282 > LR 0.8724；"
              "B 为 0.9985 / 0.9964 / 0.9660；C 为 0.9988 / 0.9960 / 0.9695。", 9.2, before=4)

    cap(doc, "表 3-2  全训练集锚点（LR 与 LGBM 用满训练集；TabPFN 标 † 者为 8,000 上限）")
```

```
    rows = []
    for fs in "ABC":
        for tag, label in [("full", "全队列"), ("dedup", "去重队列")]:
            rows.append(("__SECTION__", f"特征集 {fs} · {label}"))
            nmax = int(MET[(fs, tag)].n_train.max())
            for m in ["lr", "lgbm", "tabpfn"]:
                n_show = 8000 if m == "tabpfn" else nmax
                r = MET[(fs, tag)]
                rr = r[(r.n_train == n_show) & (r.model == m)]
                if len(rr) == 0:
                    continue
                rr = rr.iloc[0]
                slope = TN[f"slope_{tag}_{fs}_{m}_fullsize"]
                star = " †" if m == "tabpfn" else ""
                rows.append([MODEL[m] + star, f"{n_show:,}", f"{rr.auroc:.4f}", f"{rr.auprc:.4f}",
                             f"{rr.brier:.4f}", f"{slope:.3f}"])
    table(doc, ["模型", "n_train", "AUROC", "AUPRC", "Brier", "校准斜率"], rows,
          [2.2, 2.0, 3.0, 3.0, 3.0, 3.0], size=8.0,
          foot="源文件：metrics_{full,dedup}_{A,B,C}_final.csv 的最大 n 行；斜率来自 table_numbers.json 的 slope_*_fullsize。
† TabPFN 无法在 8,000 以上运行，此行为 8,000 例估计，不是全量估计。")

    para(doc, "这张表回答“再多给数据还有没有用”。A 上 LGBM 从 8,000 的 0.9378 升到 17,167 的 0.9460（+0.0082），"
              "说明入院特征仍有可学的余量；B 上 LGBM 从 0.9923 升到 0.9956（+0.0033），已接近天花板；"
              "C 上 LGBM 从 0.9912 升到 0.9937（+0.0025），但仍低于 B 的 0.9956——并发症块没有带来信息，"
              "还轻微稀释了模型。LR 的对应变化是 A +0.0065、B +0.0023、C +0.0004（8,000 → 最大 n）。", 9.2, before=4)
    para(doc, "注意 TabPFN 的两行不是“全量”结果：其上下文上限为 8,000，表中数值与上一张表的 8,000 行相同。"
              "本书在所有出现 † 的地方都保持这一约定，避免把 8,000 的估计误读成 17,167 的结果。", 9.1, after=4)

    h2(doc, "3.1  计算成本")
    para(doc, "同一台服务器、同一批 CPU 上的墙钟时间（拟合 + 预测一次）。这不是 benchmarking 级别的精确计时，"
              "但足以回答临床落地时最现实的问题：训练一次要等多久。")
    cap(doc, "表 3-3  n = 8,000 时的单次拟合+预测耗时（秒）")
    rows = []
    for fs in "ABC":
        for tag, label in [("full", "全队列"), ("dedup", "去重队列")]:
            rows.append(("__SECTION__", f"特征集 {fs} · {label}"))
            d = MET[(fs, tag)]
            s = d[d.n_train == 8000].set_index("model")["fit_predict_seconds"]
            base = max(float(s["lr"]), 1e-3)
            for m in ["lr", "lgbm", "tabpfn"]:
                rows.append([MODEL[m], f"{float(s[m]):.2f}", f"{float(s[m]) / base:.0f}×"])
    table(doc, ["模型", "秒", "相对 LR"], rows, [3.4, 5.0, 5.0], size=8.0,
          foot="源文件：metrics_*_final.csv 的 fit_predict_seconds 列。单次运行、共享服务器、CPU；未做重复计时。")
    para(doc, "LR 与 LGBM 都在 1 秒以内；TabPFN 在 CPU 上需要 2–5 分钟，是 LR 的数千倍。"
              "这个成本差是真实的：TabPFN 的优势要用“更少样本达到同样性能”来兑换，而不是免费获得。"
              "在 B/C 上它用 500 例追平 LGBM 8,000 例，省下的是数据与标注，不是算力。", 9.2, before=3)


def sec4(doc):
    h1(doc, "4  校准与决策曲线")
    para(doc, "判别力只回答“排序对不对”，校准回答“概率能不能直接拿给患者看”。校准斜率 < 1 表示模型把概率推得"
              "过散（高估高风险、低估低风险），> 1 表示推得过于集中。表中同时给出 n = 8,000 与全训练集的斜率，"
              "以及对应的 Brier 分数。")
    for tag, label, tno in [("full", "全队列", 1), ("dedup", "去重队列", 2)]:
        cap(doc, f"表 4-{tno}  {label}的校准（n = 8,000 与全训练集）")
        rows = []
        for fs in "ABC":
            rows.append(("__SECTION__", f"特征集 {fs}"))
            for m in ["lr", "lgbm", "tabpfn"]:
                o = SUMMARY[f"{fs}_{tag}"]["p_" + m]
                nmax = int(MET[(fs, tag)].n_train.max())
                n_show = 8000 if m == "tabpfn" else nmax
                rr = MET[(fs, tag)]
                rr = rr[(rr.n_train == n_show) & (rr.model == m)].iloc[0]
                rows.append([MODEL[m], f"{o['cal_slope']:.3f}", f"{o['cal_intercept']:+.3f}",
                             f"{TN[f'slope_{tag}_{fs}_{m}_fullsize']:.3f}", f"{o['brier']:.4f}",
                             f"{rr.brier:.4f}"])
        table(doc, ["模型", "斜率@8,000", "截距@8,000", "斜率@全量", "Brier@8,000", "Brier@全量"], rows,
              [2.0, 2.9, 2.9, 2.9, 3.0, 3.0], size=8.0,
              foot=("源文件：bench_summary_final.json（8,000）与 table_numbers.json（全量斜率）、"
                    "metrics_*_final.csv（Brier）。† TabPFN 的“全量”列仍为 8,000 估计。"))
    para(doc, "LightGBM 的斜率在六个组合中全部低于 0.85（0.677–0.815），是系统性的概率压缩：它排序很好，"
              "但输出的概率偏保守/过度自信，直接用于个体风险沟通会失真。LR 除 A-full 的 0.846 外都在 0.96–1.01，"
              "校准接近恒等；TabPFN 在 0.897–1.240 之间，B/C 的去重队列略高于 1（1.15 / 1.24），"
              "说明它在极低风险区的概率被推得过于集中——这是接近完美判别时校准斜率不稳定的典型表现，"
              "不应解读为“校准变差”。", 9.2, before=4)
    para(doc, "全训练集上的斜率进一步放大这一模式：B/C 的 LR 与 TabPFN 斜率都略高于 1（1.02–1.04 与 1.02–1.24），"
              "因为测试集里几乎没有中间风险的患者，logistic 斜率对少数极端概率非常敏感。结论不变："
              "若要把概率用于临床，LR 与 TabPFN 可直接使用，LGBM 需要先做校准（Platt 或 isotonic）。", 9.2)
```

```
    for fs, name in [("A", "特征集 A"), ("B", "特征集 B"), ("C", "特征集 C")]:
        figure(doc, f"fig_calib_{fs}.png",
               f"图 4-{ 'ABC'.index(fs) + 1 }  {name}在 n = 8,000 下的校准曲线（按预测风险五分位）")
    for fs, name in [("A", "特征集 A"), ("B", "特征集 B"), ("C", "特征集 C")]:
        figure(doc, f"fig_dca_{fs}.png",
               f"图 4-{ 'ABC'.index(fs) + 4 }  {name}在 n = 8,000 下的决策曲线（阈值 0.05–0.60）")
    para(doc, "决策曲线的读法：在某个阈值上，曲线高于“全治”与“不治”两条参考线，才说明按模型分流有净获益。"
              "B/C 上 TabPFN 在绝大多数阈值上最高；A 上三模型在 0.10–0.30 区间接近，LGBM 在中段略优。"
              "校准与决策曲线的完整像素级结果见 results/figs/。", 9.1, before=4)


def sec5(doc):
    h1(doc, "5  增量与配对比较")
    para(doc, "本节回答两个问题：加一个特征块值不值（A→B、B→C）；换一个模型值不值（TabPFN vs LGBM vs LR）。"
              "所有差值都在同一批测试患者上计算，区间来自 2,000 次配对 bootstrap。")

    h2(doc, "5.1  特征块增量")
    cap(doc, "表 5-1  特征块带来的 ΔAUROC（n = 8,000；括号内为 95% CI）")
    rows = []
    for label, pre in [("A → B（加实验室）", "delta_AB_full"), ("B → C（加并发症）", "delta_CB_full")]:
        for tag, tlabel in [("full", "全队列"), ("dedup", "去重队列")]:
            key = pre if tag == "full" else pre.replace("_full", "_dedup")
            if tag == "dedup" and pre == "delta_AB_full":
                key = "delta_B_minus_A"
                suffix = "_dedup"
            elif tag == "dedup":
                key = "delta_CB_dedup"
                suffix = ""
            else:
                suffix = ""
            vals = []
            for m in ["lr", "lgbm", "tabpfn"]:
                k = key + ("" if tag == "full" else ("_dedup" if pre == "delta_CB_full" else ""))
                if tag == "dedup" and pre == "delta_AB_full":
                    k = "delta_B_minus_A_" + m + "_dedup"
                elif tag == "dedup":
                    k = "delta_CB_dedup_" + m
                else:
                    k = pre + "_" + m
                vals.append(ci_str(TN[k], signed=True))
            rows.append([label, tlabel] + vals)
    table(doc, ["对比", "队列", "LR", "LGBM", "TabPFN"], rows, [4.0, 1.9, 3.8, 3.8, 3.8], size=7.8,
          foot="源文件：results/table_numbers.json（delta_AB_*、delta_B_minus_A_*、delta_CB_*）。正值为加入新特征块后 AUROC
升高。")
    para(doc, "实验室块是唯一大幅改变判别力的增量：LR +0.1072、LGBM +0.0544、TabPFN +0.0535（full），"
              "三个区间都不含零；去重后同向且幅度相近（+0.0936 / +0.0683 / +0.0676）。"
              "这个增量在 LR 上最大，因为线性模型从新信息中获益最多；对已经很强的 LGBM/TabPFN，"
              "增益相对小，但仍然真实。", 9.2, before=4)
    para(doc, "并发症块的增量则是噪声级别：LR +0.0043（+0.0008, +0.0084），LGBM −0.0011（−0.0029, +0.0005），"
              "TabPFN +0.0002（−0.0004, +0.0011）。只有 LR 的区间排除零，但 0.0043 仅为实验室块增量（0.1072）的"
              "二十五分之一，且低于任何合理的临床意义阈值。去重后 LR 的区间甚至触及零（−0.00005, +0.00745）。"
              "结论是清楚的：四个卒中后并发症变量在 70 个基线+实验室特征之外没有增量价值。", 9.2)

    h2(doc, "5.2  模型之间的配对差")
    cap(doc, "表 5-2  同一测试集上的 ΔAUROC（n = 8,000；括号内为 95% CI）")
    rows = []
    for fs in "ABC":
        for tag, label in [("full", "全队列"), ("dedup", "去重队列")]:
            rows.append(("__SECTION__", f"特征集 {fs} · {label}"))
            for key, name in [
                ("delta_auroc_tabpfn_minus_lgbm", "TabPFN − LGBM"),
                ("delta_auroc_tabpfn_minus_lr", "TabPFN − LR"),
                ("delta_auroc_lgbm_minus_lr", "LGBM − LR"),
            ]:
                v = SUMMARY[f"{fs}_{tag}"][key]
                crosses = "是" if v[1] <= 0 <= v[2] else "否"
                rows.append([name, ci_str(v, signed=True), crosses])
    table(doc, ["对比", "ΔAUROC（95% CI）", "区间跨零"], rows, [4.6, 9.0, 2.4], size=8.0,
          foot="源文件：bench_summary_final.json 的 delta_auroc_* 字段。跨零 = 95% 区间包含 0，差异不能在 5% 水平上检出。")
    para(doc, "三条结论。第一，TabPFN 相对 LR 的优势在所有六个组合中都统计上可检出（最小 +0.0293，"
              "区间 +0.0178 至 +0.0439，C-dedup）。第二，TabPFN 相对 LGBM 的优势在 B/C 上可检出"
              "（B full +0.0057 [0.0017, 0.0110]；C full +0.0069 [0.0019, 0.0138]），在 A 上不可检出"
              "（+0.0067 [−0.0021, +0.0159]）。第三，LGBM 相对 LR 的优势在所有组合中都可检出，"
              "幅度 0.024–0.083——树模型比线性模型更适合这套特征。", 9.2, before=4)

    h2(doc, "5.3  数据效率：TabPFN 用多少例追平 LGBM")
    para(doc, "定义：以 LGBM 在 n = 8,000 的 AUROC 为基准，允许 −0.002 的容差，找 TabPFN 首次达到该水平的"
              "最小训练规模。这是对“小样本优势”最直接、也最不容易夸大的量化。")
```

```
    cap(doc, "表 5-3  TabPFN 追平 LGBM（8,000 例水平）所需的最小 n")
    rows = []
    for fs in "ABC":
        for tag, label in [("full", "全队列"), ("dedup", "去重队列")]:
            n_match = SUMMARY[f"{fs}_{tag}"]["tabpfn_n_to_match_lgbm_at_8000"]
            lgb8 = mval(fs, tag, 8000, "lgbm")
            tp = mval(fs, tag, n_match, "tabpfn")
            rows.append([f"特征集 {fs} · {label}", f"{lgb8:.4f}", f"{int(n_match):,}", f"{tp:.4f}",
                         f"{tp - lgb8:+.4f}"])
    table(doc, ["组合", "LGBM@8,000", "TabPFN 最小 n", "TabPFN@该 n", "差值"], rows,
          [5.0, 3.0, 3.2, 3.2, 2.6], size=8.2,
          foot="源文件：bench_summary_final.json 的 tabpfn_n_to_match_lgbm_at_8000；定义为 TabPFN AUROC ≥ LGBM@8,000 − 0.002
的最小 n。")
    para(doc, "B/C 全队列只需 500 例就进入容差（0.9911 / 0.9907 vs 0.9923 / 0.9912），去重队列需 2,000 例；"
              "A 需要 4,000 例（0.9424，已超过 LGBM@8,000 的 0.9378）。换成比例：在 B/C 上，TabPFN 用 6.25% 的训练数据"
              "达到 LGBM 用满 8,000 例的水平；在 A 上这个比例是 50%。信息越少的问题，预训练先验越值钱——"
              "这与“小样本优势来自先验而非数据”的解释一致。", 9.2, before=4)

    for fs, name in [("A", "特征集 A"), ("B", "特征集 B"), ("C", "特征集 C")]:
        figure(doc, f"fig_roc_pr_{fs}.png",
               f"图 5-{ 'ABC'.index(fs) + 1 }  {name}在 n = 8,000 下的 ROC 与 PR 曲线（全队列）")
    for fs, name in [("A", "特征集 A"), ("B", "特征集 B"), ("C", "特征集 C")]:
        figure(doc, f"fig_roc_pr_{fs}_dedup.png",
               f"图 5-{ 'ABC'.index(fs) + 4 }  {name}在 n = 8,000 下的 ROC 与 PR 曲线（去重队列）")


def sec6(doc):
    h1(doc, "6  特征重要性：A / B / C 的前十变量")
    para(doc, "重要性用两种口径：LGBM 的总增益份额（gain share，所有树分裂带来的不纯度下降占比，加总为 100%），"
              "以及标准化 LR 的系数绝对值。两者不可直接比较——增益份额是相对量，系数是每标准差变化的 log-odds——"
              "但各自内部可以排序。下表列出每个特征集的前十，右侧同时给出另一模型的前十，便于对照。")
    for fs, tno in [("A", 1), ("B", 2), ("C", 3)]:
        im = REST["importance"][fs]
        cap(doc, f"表 6-{tno}  特征集 {fs}（{im['n_features']} 个变量）的前十重要性")
        rows = []
        for k in range(10):
            g = im["lgbm"][k]
            l = im["lr"][k]
            rows.append([str(k + 1), CN.get(g["var"], g["var"]), f"{g['gain_share'] * 100:.2f}%",
                         CN.get(l["var"], l["var"]), f"{l['abs_coef']:.2f}"])
        table(doc, ["排名", "LGBM 变量", "增益份额", "LR 变量", "|系数|"], rows,
              [1.2, 5.2, 2.6, 5.2, 2.4], size=8.2,
              foot=f"源文件：results/restore/restore.json 的 importance.{fs}。模型在全训练集（17,167 例）上重拟合；"
                   f"LGBM 测试 AUROC {im['test_auroc_lgbm']:.4f}，LR 测试 AUROC {im['test_auroc_lr']:.4f}。")
        figure(doc, f"fig_importance_{fs}.png",
               f"图 6-{tno}  特征集 {fs} 的特征重要性：LightGBM 总增益（左）与 LR 系数绝对值（右）")

    para(doc, "A 的信息高度集中：NIHSS 单独占 LGBM 增益的 49.99%，年龄 17.53%，高血压 6.66%，"
              "其余变量都在 3% 以下；LR 的口径更分散，岛叶（1.68）、NIHSS（1.30）、高尿酸血症（0.91）排前三。"
              "这解释了 A 的学习曲线为什么最陡：模型主要在学习“NIHSS 越高越危险”这一条主轴。", 9.2, before=4)
    para(doc, "B 把重要性重新分配：NIHSS 降到 29.54%，白细胞 9.75%、C 反应蛋白 7.64%、α-羟丁酸脱氢酶 6.09%、"
              "谷草转氨酶 5.20% 进入前五；LR 侧则是 D-二聚体（3.84）、高血压（3.23）、C 反应蛋白（3.03）、"
              "谷草转氨酶（2.99）。实验室指标不是“锦上添花”，它们确实承担了相当一部分判别信息。", 9.2)
    para(doc, "C 的变化最能说明问题：加入四个并发症变量后，LGBM 的增益重心转到 CK-MB（18.68%）、NIHSS（14.91%）、"
              "C 反应蛋白（12.56%）、α-羟丁酸脱氢酶（6.44%）与乳酸（4.90%）；四个并发症变量在 LGBM 中排名"
              "第 45（深静脉血栓）、74（脑疝）、65（脑积水）、35（低蛋白血症），全部在后半段。"
              "LR 中脑积水（第 9）与低蛋白血症（第 12）进入中上段，深静脉血栓（第 19）、脑疝（第 25）居中——"
              "这四项的单变量关联很强（第 3.1 节表 1），但在 70 个特征之外能提供的增量几乎为零。"
              "单变量关联与增量价值是两个不同的量。", 9.2, after=4)


def sec7(doc):
    h1(doc, "7  特征集 A 的五折交叉验证与 OOF")
    para(doc, "主分析用的是固定 80/20 划分。为了确认 A 的结果不是某一次划分的偶然，另做一次 5 折分层交叉验证"
              "（在 17,167 例训练集内部，种子 42），并汇总折外预测（OOF）。OOF 指标与“折均值”不完全相同："
              "前者把所有折的预测拼成一份完整训练集的预测，后者是五个折指标的算术平均。")
    cap(doc, "表 7-1  特征集 A 的 5 折交叉验证（训练集内部）")
    rows = []
    for m in ["lr", "lgbm"]:
        c = REST["cv_A"][m]
        rows.append([MODEL[m],
                     f"{c['auroc_mean']:.4f} ± {c['auroc_sd']:.4f}",
                     f"{c['auprc_mean']:.4f} ± {c['auprc_sd']:.4f}",
                     f"{c['brier_mean']:.4f} ± {c['brier_sd']:.4f}",
                     f"{c['oof_auroc']:.4f}", f"{c['oof_auprc']:.4f}", f"{c['oof_brier']:.4f}"])
    table(doc, ["模型", "AUROC（折均值±SD）", "AUPRC", "Brier", "OOF AUROC", "OOF AUPRC", "OOF Brier"], rows,
          [1.6, 3.4, 3.0, 2.9, 2.2, 2.2, 2.2], size=7.9,
          foot="源文件：results/restore/restore.json 的 cv_A。5 折、单次重复；OOF = 折外预测拼接后整体计算。")
    para(doc, "CV 的数值高于固定测试集（LR 0.8849 vs 0.8553；LGBM 0.9461 vs 0.9378），原因不是模型变了，"
```

```
            "而是评估对象不同：CV 在训练集内部做，测试折与训练折来自同一分布且没有留出整个 20% 的固定测试集；"
            "固定测试集是更严格、也更接近外部验证的口径。两者都报告，是为了让读者看到“划分带来的不确定性”有多大——"
            "约 0.01–0.03 AUROC，远大于模型之间的部分差距。任何小于这个量级的模型比较都应谨慎解读。", 9.2, before=4)


def sec8(doc):
    h1(doc, "8  D 研究：温州 167 例的首次发作时点预测")
    para(doc, "D 研究换了队列与临床问题：167 例卒中后早期癫痫发作患者（PSE 49 例，29.3%；中位随访 592 天），"
              "在“首次发作之后”这个决策点预测是否会发展为晚期 PSE。核心特征集只有 8 个变量，"
              "扩展集 24 个；评估用 5 折 × 10 重复交叉验证（种子 42）。因此它的数字不能与 A/B/C 横向比较，"
              "只能在本节内部比较。")

    h2(doc, "8.1  队列基线")
    cap(doc, "表 8-1  温州队列基线（总体、无 PSE、PSE）")
    rows = []
    spec = [
        ("年龄（岁，中位/IQR）", "age_med_iqr", "iqr"),
        ("男性", "sex_male_pct", "pct"),
        ("缺血性卒中", "stroke_ischemic_pct", "pct"),
        ("NIHSS（中位/IQR）", "nihss_med_iqr", "iqr"),
        ("首次发作距卒中 > 3 天", "interval_gt3_pct", "pct"),
        ("早期多次发作", "multiple_seizures_pct", "pct"),
        ("癫痫持续状态", "status_epilepticus_pct", "pct"),
        ("皮层受累", "cortical_pct", "pct"),
        ("EEG 癫痫样放电", "eeg_epileptic_pct", "pct"),
        ("病灶体积（cm³，中位/IQR）", "lesion_vol_med_iqr", "iqr"),
        ("使用 ASM", "asm_pct", "pct"),
        ("随访天数（中位/IQR）", "followup_days_med_iqr", "iqr"),
    ]
    for label, key, kind in spec:
        vals = []
        for g in ["overall", "pse_0", "pse_1"]:
            block = TW["overall"] if g == "overall" else TW["by_pse"][g]
            v = block[key]
            vals.append(med_iqr(v) if kind == "iqr" else pct(v))
        rows.append([label] + vals)
    table(doc, ["指标", "总体（n = 167）", "无 PSE（n = 118）", "PSE（n = 49）"], rows,
          [5.6, 4.2, 4.2, 4.2], size=8.1,
          foot="源文件：results/table1_wenzhou.json。PSE 组的随访天数为“到首次晚期发作或末次随访”的时间，因此中位数仅 74 天，不能与无 PSE 组直接比较。")
    para(doc, "两组差异集中在发作本身的信息：首次发作距卒中 > 3 天者 32.7% vs 10.2%，早期多次发作 26.5% vs 6.8%，"
              "癫痫持续状态 32.7% vs 23.7%；PSE 组 NIHSS 的上四分位数更高（16 vs 11）。"
              "皮层受累与 EEG 癫痫样放电在两组间反而略低（83.7% vs 85.6%；18.4% vs 22.0%），"
              "提示这套队列的转归更多由发作时间与复发模式驱动，而不是单纯的病灶严重度。", 9.2, before=4)

    h2(doc, "8.2  交叉验证结果")
    cap(doc, "表 8-2  D 研究 5 折 × 10 重复交叉验证（均值 ± SD）")
    rows = []
    for m in ["lr", "lgbm", "tabpfn"]:
        rows.append([MODEL[m],
                     f"{dmet('core','auroc',m):.4f} ± {dmet('core','auroc',m,'std'):.4f}",
                     f"{dmet('core','auprc',m):.4f} ± {dmet('core','auprc',m,'std'):.4f}",
                     f"{dmet('core','brier',m):.4f} ± {dmet('core','brier',m,'std'):.4f}",
                     f"{dmet('core','cal_slope',m):.3f}",
                     f"{dmet('full','auroc',m):.4f} ± {dmet('full','auroc',m,'std'):.4f}",
                     f"{dmet('full','auprc',m):.4f}",
                     f"{dmet('full','brier',m):.4f}",
                     f"{dmet('full','cal_slope',m):.3f}"])
    table(doc, ["模型", "核心 AUROC", "核心 AUPRC", "核心 Brier", "核心斜率", "扩展 AUROC", "扩展 AUPRC", "扩展 Brier", "扩展斜率"], rows,
          [1.4, 2.6, 2.4, 2.2, 1.6, 2.6, 2.0, 1.8, 1.6], size=7.4,
          foot="源文件：results/summary_D.json 的 feature_sets.*.metrics_mean_std。核心 8 变量、扩展 24 变量；50 个折次。")
    para(doc, "核心 8 变量上 LR 最好（AUROC 0.7101，AUPRC 0.5331，Brier 0.1825），LGBM 次之（0.7018），"
              "TabPFN 最低（0.6706）。扩展到 24 变量后三个模型全部变差（0.6359 / 0.6389 / 0.6233）——"
              "在 n = 167 下，额外 16 个变量带来的是过拟合而不是信息。校准方面 TabPFN 最好（核心斜率 0.939），"
              "LGBM 0.804，LR 0.918；扩展集上三者都退化（0.50–0.87）。", 9.2, before=4)

    h2(doc, "8.3  配对差与 ASM 增量")
    cap(doc, "表 8-3  核心特征集上的配对 ΔAUROC（bootstrap 95% CI）")
    rows = []
    for key, name in [("tabpfn_minus_lgbm", "TabPFN − LGBM"), ("tabpfn_minus_lr", "TabPFN − LR"),
                      ("lr_minus_lgbm", "LR − LGBM")]:
        v = SD["paired_bootstrap_delta_auroc_core"][key]
        rows.append([name, ci_str(v, signed=True), "是" if v[1] <= 0 <= v[2] else "否"])
    table(doc, ["对比", "ΔAUROC（95% CI）", "区间跨零"], rows, [4.6, 9.0, 2.4], size=8.2,
          foot="源文件：results/summary_D.json 的 paired_bootstrap_delta_auroc_core。")
    para(doc, "只有 TabPFN − LR 的区间排除零（−0.0565 [−0.1090, −0.0058]），即 TabPFN 显著更差；"
              "TabPFN − LGBM 与 LR − LGBM 的区间都跨零。在小队列里，模型差距的区间本身就很宽，"
              "这正是 D 研究反复强调“诚实基准”的原因。", 9.2, before=4)
```

```
    cap(doc, "表 8-4  加入 ASM 变量前后的 AUROC（核心集，10 次重复）")
    rows = []
    for m in ["lr", "lgbm", "tabpfn"]:
        a = SD["asm_incremental_core"][m]
        rows.append([MODEL[m], f"{a['auroc_without_asm']:.4f}", f"{a['auroc_with_asm']:.4f}",
                     f"{a['delta_mean']:+.4f}", f"{a['n_repeats_delta_gt0']}/10"])
    table(doc, ["模型", "不含 ASM", "含 ASM", "ΔAUROC", "Δ>0 的重复次数"], rows,
          [2.4, 3.2, 3.2, 3.0, 3.6], size=8.2,
          foot="源文件：results/summary_D.json 的 asm_incremental_core。")
    para(doc, "三个模型的 Δ 都为负，且 Δ>0 的重复次数只有 0–2/10。ASM 使用不提供增量预测价值；"
              "这与 E 研究的结论一致：是否用药主要由发作严重度决定，而严重度已经由核心 8 变量编码。", 9.2, before=3)

    h2(doc, "8.4  风险三分位与生存分层")
    cap(doc, "表 8-5  按模型风险评分三分位的两年无 PSE 生存率")
    rows = []
    for m in ["lr", "tabpfn"]:
        k = SD["km_risk_tertiles"][m]
        rows.append([MODEL[m],
                     f"{k['tertile0']['n']}/{k['tertile0']['events']}/{k['tertile0']['pse_free_2y']:.3f}",
                     f"{k['tertile1']['n']}/{k['tertile1']['events']}/{k['tertile1']['pse_free_2y']:.3f}",
                     f"{k['tertile2']['n']}/{k['tertile2']['events']}/{k['tertile2']['pse_free_2y']:.3f}",
                     f"{k['logrank_chi2']:.2f}", sci(k["logrank_p"])])
    table(doc, ["模型", "低危（n/事件/2 年）", "中危（n/事件/2 年）", "高危（n/事件/2 年）", "log-rank χ²", "p"], rows,
          [1.8, 3.6, 3.6, 3.6, 2.2, 2.2], size=8.0,
          foot="源文件：results/summary_D.json 的 km_risk_tertiles。每格为“组内例数/事件数/两年无 PSE 生存率”。")
    figure(doc, "fig_km_lr.png", "图 8-1  按 LR 风险评分三分位的 Kaplan–Meier 两年无 PSE 生存曲线")
    figure(doc, "fig_km_tabpfn.png", "图 8-2  按 TabPFN 风险评分三分位的 Kaplan–Meier 两年无 PSE 生存曲线")
    para(doc, "LR 三分位的两年无 PSE 生存率为 0.889 / 0.769 / 0.477，log-rank χ² = 25.16，p = 3.4 × 10⁻⁶；"
              "TabPFN 为 0.853 / 0.750 / 0.524，χ² = 15.72，p = 3.9 × 10⁻⁴。两者都能把风险分层拉开，"
              "但 LR 的梯度更陡。注意这是“分层能力”，不是“判别力”——三分位割裂会放大看起来的差异，"
              "不能替代 AUROC。", 9.2, before=4)

    h2(doc, "8.5  学习曲线与决策曲线")
    figure(doc, "fig_learning_curve.png", "图 8-3  核心特征集上的学习曲线（三个种子均值）：TabPFN 在 n = 30–80 领先，线性模型在全队列规模反超")
    para(doc, "n = 30 时 TabPFN 0.5681、LR 0.5014、LGBM 0.4174；n = 80 时 TabPFN 0.6944 仍最高；"
              "到 n = 134 时 LGBM 0.7104、LR 0.6875，TabPFN 回落到 0.6486。这是全系列中唯一出现"
              "“基础模型在小样本领先、随样本增加被反超”完整轨迹的数据集——恰好在 n 只有一两百例时，"
              "预训练先验的价值最大，而数据量一旦足够，简单模型的归纳偏置更合适。", 9.2, before=2)
    figure(doc, "fig_dca.png", "图 8-4  核心特征集三个模型的决策曲线")
    para(doc, "决策曲线在高阈值段三模型接近；在 0.2–0.4 的临床相关区间，LR 与 LGBM 略高于 TabPFN。"
              "结合判别力与校准，D 研究的推荐是 8 变量 LR：不是因为它最“先进”，而是因为它在 n = 167 时"
              "最稳、最可解释、校准也足够好。", 9.2, before=3)


def sec9(doc):
    h1(doc, "9  E 研究：抗癫痫药物与晚期 PSE 风险")
    para(doc, "E 研究是观察性比较，不是预测建模：134 例使用 ASM、33 例未使用，比较随访期内 PSE 的发生风险。"
              "由于用药由发作严重度决定，粗比较天然受指征混杂影响；本节给出粗估计、多变量校正、倾向评分校正、"
              "IPTW 与 AIPW 五种口径，目的是看结论是否稳健，而不是宣称因果。")

    h2(doc, "9.1  效应估计")
    c = SE["crude"]; a = SE["adjusted"]; ps = SE["ps_adjusted"]; ip = SE["iptw"]; ai = SE["aipw"]
    cap(doc, "表 9-1  ASM 与 PSE 风险的效应估计")
    rows = [
        ["粗比值比（OR）", f"{c['or']:.3f}", f"（{c['or_lo']:.3f}, {c['or_hi']:.3f}）", f"E 值 {c['e_value']:.3f}"],
        ["多变量校正 OR", f"{a['or']:.3f}", f"（{a['or_lo']:.3f}, {a['or_hi']:.3f}）", f"E 值 {a['e_value']:.3f}；协变量 9个"],
        ["倾向评分校正 OR", f"{ps['or']:.3f}", f"（{ps['or_lo']:.3f}, {ps['or_hi']:.3f}）", "PS 作为协变量"],
        ["IPTW 风险差", f"{ip['risk_diff']:+.5f}", f"（{ip['rd_lo']:+.3f}, {ip['rd_hi']:+.3f}）", f"PS c 统计量{ip['ps_auc']:.3f}"],
        ["AIPW 风险差", f"{ai['ate_risk_diff']:+.4f}", f"（{ai['rd_lo']:+.3f}, {ai['rd_hi']:+.3f}）", "增广估计"],
    ]
    table(doc, ["方法", "估计值", "95% CI", "备注"], rows, [4.2, 3.2, 5.2, 4.0], size=8.1,
          foot="源文件：results/E/summary_E.json 的 crude / adjusted / ps_adjusted / iptw / aipw。所有区间为 bootstrap 2,000次。")
    para(doc, "五个口径的点估计在 0.92–1.38 之间，全部区间跨 1（或风险差跨 0）。没有保护效应的证据，"
              "也没有可信的危害信号。E 值 2.10（粗）与 1.57（校正）意味着：需要一个与 ASM 和 PSE 都有"
              "约 1.6–2.1 倍关联的未测量混杂因素，才能把观察到的效应完全解释掉——在本领域这并非不可能，"
              "因此不能把“无显著”读成“无效应”。", 9.2, before=4)

    h2(doc, "9.2  协变量平衡")
    cap(doc, "表 9-2  倾向评分加权前后的标准化均值差（SMD）")
    rows = []
    for _, r in SMD.iterrows():
        rows.append([CN.get(r.variable, r.variable), f"{r.smd_before:+.3f}", f"{r.smd_after:+.3f}",
                     "是" if abs(r.smd_after) < 0.1 else "否"])
    table(doc, ["协变量", "加权前 SMD", "加权后 SMD", "|SMD| < 0.1"], rows, [5.0, 3.6, 3.6, 3.0], size=8.2,
```

```
          foot="源文件：results/E/smd_balance.csv。SMD 为 ASM 组减未用药组。")
    figure(doc, "fig_love.png", "图 9-1  协变量平衡的 Love 图（加权前后）")
    para(doc, "加权前最大 |SMD| 为 0.423（早期多次发作），加权后降到 0.152（皮层受累）；多数协变量已平衡，"
              "但仍有两个超过 0.1。更重要的是倾向评分本身的 c 统计量只有 0.510——它几乎不能区分治疗组与"
              "对照组，说明可观测变量无法解释用药选择。因此 IPTW 结果只能作为稳定性检查，不能当作因果估计；"
              "真正决定用药的是未测量的发作严重度。", 9.2, before=4)

    h2(doc, "9.3  生存分析")
    s = SE["survival"]
    cap(doc, "表 9-3  两年无 PSE 生存与限制平均生存时间")
    rows = [
        ["两年无 PSE 生存率", f"{s['pse_free_2y_asm']:.3f}", f"{s['pse_free_2y_noasm']:.3f}",
         f"log-rank p = {s['logrank_p']:.3f}"],
        ["730 天 RMST（天）", f"{s['rmst_730_asm']:.1f}", f"{s['rmst_730_noasm']:.1f}",
         f"差 {s['rmst_diff']:+.1f}（{s['rmst_diff_lo']:+.1f}, {s['rmst_diff_hi']:+.1f}）"],
        ["90 天界标 OR", "—", "—", f"{s['landmark_90d']['or']:.3f}（{s['landmark_90d']['or_lo']:.3f},
{s['landmark_90d']['or_hi']:.3f}）"],
    ]
    table(doc, ["指标", "ASM（n = 134）", "无 ASM（n = 33）", "差异 / 检验"], rows, [4.2, 4.0, 4.0, 5.0], size=8.2,
          foot="源文件：results/E/summary_E.json 的 survival。RMST = 限制平均生存时间（0–730 天）。")
    figure(doc, "fig_km_asm.png", "图 9-2  按 ASM 状态的 Kaplan–Meier 两年无 PSE 生存曲线")
    para(doc, "两条曲线几乎重合：两年无 PSE 生存 70.3% vs 74.5%，log-rank p = 0.447；RMST 差 −23.6 天"
              "（95% CI −122.4 至 +81.1）。90 天界标分析（排除最早期的死亡/事件竞争）OR 0.924"
              "（0.320–3.910），同样没有信号。比例风险假设未通过检验，因此不报告单一 Cox 风险比——"
              "这也是方法节明确写下的决定。", 9.2, before=4)

    h2(doc, "9.4  事件分期与卒中类型分层")
    cap(doc, "表 9-4  按时期划分的 PSE 事件数")
    rows = []
    for epoch in ["0-90d", "91-365d", ">365d"]:
        e = s["events_by_epoch"][epoch]
        rows.append([epoch.replace("d", " 天"), str(e["1"]), str(e["0"])])
    table(doc, ["时期", "ASM 组事件", "无 ASM 组事件"], rows, [4.6, 4.6, 4.6], size=8.4,
          foot="源文件：results/E/summary_E.json 的 survival.events_by_epoch。")
    para(doc, "早期事件高度集中在 ASM 组（0–90 天 24 vs 3），随后两组差距收窄（91–365 天 7 vs 4；>365 天 10 vs 1）。"
              "这符合指征混杂的预期：临床医生给最严重的患者用药，而最严重的患者最早发生事件；"
              "若把 0–90 天的事件当作“药物危害”，就会误读。90 天界标分析正是为此设计，结果无差异。", 9.2, before=4)

    cap(doc, "表 9-5  按卒中类型的 ASM 效应")
    rows = []
    for key, name in [("ICH", "出血性卒中"), ("ischemic", "缺血性卒中")]:
        x = SE["stroke_type_strata"][key]
        rows.append([name, f"{x['n']}（{x['n_asm']}/{x['n_noasm']}）",
                     f"{x['risk_asm'] * 100:.1f}%", f"{x['risk_noasm'] * 100:.1f}%",
                     f"{x['or']:.3f}（{x['or_lo']:.3f}, {x['or_hi']:.3f}）"])
    table(doc, ["卒中类型", "n（ASM/无）", "ASM 组事件率", "无 ASM 组事件率", "OR（95% CI）"], rows,
          [3.2, 3.4, 3.2, 3.4, 4.0], size=8.2,
          foot="源文件：results/E/summary_E.json 的 stroke_type_strata。")
    para(doc, "分层结果方向相反：出血性卒中 OR 0.488（0.110–2.140），缺血性卒中 OR 2.907（0.948–12.131）。"
              "两个区间都很宽，且样本量小（分别为 56 与 111 例），不足以支持“效应异质性”的结论；"
              "它更可能反映随机波动与小样本不稳定性。E 研究的立场是描述而非推断：在这 167 例里，"
              "没有可复现的保护信号。", 9.2, before=3)


def sec10(doc):
    h1(doc, "10  图表总索引")
    para(doc, "按编号列出本书全部图与表，以及它们在五套算法架构中的对应位置。算法架构中的编号与本书不完全相同："
              "本书是“结果账本”，编号按数据流排列；算法架构按叙事排列。对应关系以本表为准。")
    figs = [
        ("图 4-1 / 4-2 / 4-3", "A/B/C 校准曲线（n = 8,000）", "A/B/C 图 4", "fig_calib_{A,B,C}.png"),
        ("图 4-4 / 4-5 / 4-6", "A/B/C 决策曲线（n = 8,000）", "A/B/C 图 6", "fig_dca_{A,B,C}.png"),
        ("图 5-1 / 5-2 / 5-3", "A/B/C ROC 与 PR 曲线（全队列）", "A/B/C 图 2", "fig_roc_pr_{A,B,C}.png"),
        ("图 5-4 / 5-5 / 5-6", "A/B/C ROC 与 PR 曲线（去重）", "A/B/C 图 7", "fig_roc_pr_*_dedup.png"),
        ("图 6-1 / 6-2 / 6-3", "A/B/C 特征重要性", "A/B/C 图 5（C 为图 6）", "fig_importance_{A,B,C}.png"),
        ("图 8-1", "LR 风险三分位 KM 曲线", "D 图 3a", "fig_km_lr.png"),
        ("图 8-2", "TabPFN 风险三分位 KM 曲线", "D 图 3b", "fig_km_tabpfn.png"),
        ("图 8-3", "D 学习曲线", "D 图 2", "fig_learning_curve.png"),
        ("图 8-4", "D 决策曲线", "D 图 4", "fig_dca.png"),
        ("图 9-1", "协变量平衡 Love 图", "E 补充图", "fig_love.png"),
        ("图 9-2", "ASM 分层 KM 曲线", "E 图 2", "fig_km_asm.png"),
    ]
    cap(doc, "表 10-1  图索引")
    table(doc, ["本书编号", "内容", "算法架构位置", "源文件"], figs, [3.6, 5.6, 4.0, 4.2], size=7.9,
          foot="A/B/C 的学习曲线（图 3-4 至 3-6）由 results/figs/fig_learning_{A,B,C}.png 提供，对应算法架构图 3。")
    tables = [
        ("表 0-1", "术语表", "—"), ("表 1-1", "两套数据划分", "方法节"), ("表 1-2", "三套特征集", "方法节"),
        ("表 1-3", "模型配置", "方法节"), ("表 1-4", "拟合次数", "—"),
        ("表 2-1 至 2-6", "A/B/C × full/dedup 全规模明细", "补充材料"),
        ("表 3-1", "n = 8,000 主结果与 CI", "A/B/C 表 2"), ("表 3-2", "全训练集锚点", "A/B/C 表 2 注"),
```

```
        ("表 3-3", "计算成本", "—"),
        ("表 4-1 / 4-2", "校准斜率与截距", "A/B/C 正文"), ("表 5-1", "特征块增量", "B/C 表 2"),
        ("表 5-2", "模型配对差", "A/B/C 正文"), ("表 5-3", "数据效率", "A/B/C 正文"),
        ("表 6-1 至 6-3", "A/B/C 前十重要性", "A 表 3；B/C 表 3"),
        ("表 7-1", "A 的五折 CV 与 OOF", "A 表 4"),
        ("表 8-1", "温州队列基线", "D 表 1"), ("表 8-2", "D 交叉验证", "D 表 2"),
        ("表 8-3", "D 配对差", "D 正文"), ("表 8-4", "ASM 增量", "D 正文"), ("表 8-5", "KM 三分位", "D 正文"),
        ("表 9-1", "E 效应估计", "E 表 1"), ("表 9-2", "SMD 平衡", "E 补充表"),
        ("表 9-3", "生存指标", "E 正文"), ("表 9-4", "事件分期", "E 正文"), ("表 9-5", "卒中类型分层", "E 正文"),
        ("表 11-1", "文件指纹", "—"), ("表 11-2", "关键数字锚点", "—"),
    ]
    cap(doc, "表 10-2  表索引")
    table(doc, ["本书编号", "内容", "算法架构位置"], tables, [3.8, 8.6, 5.0], size=7.9)


def sec11(doc):
    h1(doc, "11  复现锚点")
    para(doc, "这一节是给“下一位接手的人”的：不需要读完整本书，只要重跑后比对下面三张表，就能判断"
              "环境与数据是否一致。")

    h2(doc, "11.1  文件指纹（SHA-256）")
    cap(doc, "表 11-1  输入文件的 SHA-256")
    table(doc, ["文件", "说明", "SHA-256"], [
        ["data.csv", "公共队列 21,459 行", "be84a00921702c45e5c9d250176602b2c62ed96ac41716dca8893b0c5be49d4b"],
        ["Wenzhou_167_cases_deidentified.csv", "温州队列 167 行",
"98d04a5eb44813e223f9150913637ccc9368d20e1adff80edb11a6bd1af900d4"],
        ["tabpfn-v2-classifier-finetuned-zk73skhh.ckpt", "TabPFN v2 检查点",
"cf8c519c01eaf1613ee91239006d57b1c806ff5f23ac1aeb1315ba1015210e49"],
    ], [4.6, 3.4, 9.4], size=6.8,
        foot="源文件：docs/配置要求与复现说明.md 第 6 节；数据指纹与 split_info.json 的 data_sha256 一致。")
    para(doc, "哈希值用 `sha256sum` 或 Python `hashlib.sha256` 计算；任何一位不同都说明文件被改动过，"
              "后续数字不可直接比较。", 9.1, after=4)

    h2(doc, "11.2  关键数字锚点")
    cap(doc, "表 11-2  重跑后应逐位一致的数字（保留四位小数）")
    anchors = [
        ("A full, n = 8,000: LR AUROC", f"{mval('A','full',8000,'lr'):.4f}", "metrics_full_A_final.csv"),
        ("A full, n = 8,000: LGBM AUROC", f"{mval('A','full',8000,'lgbm'):.4f}", "同上"),
        ("A full, n = 8,000: TabPFN AUROC", f"{mval('A','full',8000,'tabpfn'):.4f}", "同上"),
        ("B full, n = 8,000: TabPFN AUROC", f"{mval('B','full',8000,'tabpfn'):.4f}", "metrics_full_B_final.csv"),
        ("C full, n = 8,000: TabPFN AUROC", f"{mval('C','full',8000,'tabpfn'):.4f}", "metrics_full_C_final.csv"),
        ("C dedup, n = 8,000: TabPFN AUROC", f"{mval('C','dedup',8000,'tabpfn'):.4f}", "metrics_dedup_C_final.csv"),
        ("A→B ΔAUROC（LR，full）", f"{TN['delta_AB_full_lr'][0]:+.4f}", "table_numbers.json"),
        ("B→C ΔAUROC（TabPFN，full）", f"{TN['delta_CB_full_tabpfn'][0]:+.4f}", "table_numbers.json"),
        ("D 核心集 LR AUROC（均值）", f"{dmet('core','auroc','lr'):.4f}", "summary_D.json"),
        ("D 核心集 TabPFN AUROC（均值）", f"{dmet('core','auroc','tabpfn'):.4f}", "summary_D.json"),
        ("E 粗 OR", f"{SE['crude']['or']:.4f}", "E/summary_E.json"),
    ]
    table(doc, ["指标", "期望值", "源文件"],
          anchors, [7.6, 3.4, 6.2], size=8.0,
          foot="这些值对环境和数据版本敏感；建议先跑四个规模（100 / 8,000 / 全量）再全量重跑。")

    h2(doc, "11.3  运行顺序")
    for t in [
        "① 校验三个输入文件的 SHA-256（表 11-1）。",
        "② 运行 A/B/C 基准：固定划分 → 逐规模训练 → 固定测试集评估 → 输出 metrics_*_final.csv 与 preds_*_final.csv。",
        "③ 运行汇总与出图：bench_analysis.py → table_numbers.json；restore_analyses.py → restore.json；bootstrap_ci.py →
ci.json；make_figures_restore.py → 图。",
        "④ 运行 D/E：wenzhou_de.py → summary_D.json 与 E/summary_E.json；table1_wenzhou.py → 基线表。",
        "⑤ 比对表 11-2；全部一致后再重新生成本书与五套算法架构的 docx/PDF。",
    ]:
        para(doc, "· " + t, 9.2, after=3)
    para(doc, "环境提示：LR 支路需要 scikit-learn 1.7.2 的旧环境才能与归档锚点逐位一致；LGBM/TabPFN 与 D/E "
              "在 scikit-learn 1.9.1、pandas 3.0.6、numpy 2.5.3、lightgbm 4.7.0、tabpfn 9.2.0 的新环境运行。"
              "本机用 `py -3.12`。两套环境的完整版本表与逐条命令见《配置要求与复现说明》第 2–6 节。", 9.2, before=4)

    h2(doc, "11.4  版本说明")
    para(doc, "本书对应第五次大跑数的最终归档。与之前几次相比，本次的关键变化是：TabPFN 补跑到 8,000 例上限、"
              "新增去重臂、并把温州 167 例的 D/E 研究纳入同一结果体系。早期版本的 A/B/C 曾使用不同的图表组合"
              "（例如缺少基线表、ROC/PR 或特征重要性）；本次已把两版内容合并，算法架构与本书使用同一套数字。"
              "如发现本书与旧版 PDF 不一致，以本书为准。", 9.3)


def build():
    doc = Document()
    sec = doc.sections[0]
    sec.page_width, sec.page_height = Cm(21), Cm(29.7)
    sec.top_margin = sec.bottom_margin = Cm(2.0)
    sec.left_margin = sec.right_margin = Cm(1.7)
```

```
    add_page_number(sec)

    style = doc.styles["Normal"]
    style.font.name = "Times New Roman"
    style.font.size = Pt(9.5)
    style.element.rPr.rFonts.set(qn("w:eastAsia"), "宋体")

    cover(doc)
    toc(doc)
    sec0(doc)
    sec1(doc)
    sec2(doc)
    sec3(doc)
    sec4(doc)
    sec5(doc)
    sec6(doc)
    sec7(doc)
    sec8(doc)
    sec9(doc)
    sec10(doc)
    sec11(doc)

    out = os.path.join(OUT, "卒中后癫痫五套算法架构_跑数结果汇编.docx")
    doc.save(out)
    print("saved", out.encode("ascii", "ignore").decode() or "docx")
    return out


def to_pdf(docx_path):
    import win32com.client as win32
    word = win32.DispatchEx("Word.Application")
    word.Visible = False
    try:
        d = word.Documents.Open(docx_path, ReadOnly=False)
        pdf = os.path.splitext(docx_path)[0] + ".pdf"
        d.SaveAs(pdf, 17)
        d.Close(False)
        print("pdf ok")
        return pdf
    finally:
        word.Quit()


if __name__ == "__main__":
    p = build()
    to_pdf(p)
```

D.4 build_codebook.py（587 行）

用途

生成本书自身（《五套算法架构代码汇编》）。按运行顺序收录全部脚本并逐条注明用途、输入、输出与运行命令。

- **对应算法架构：**代码汇编
- **输入：**本目录与 server/ 下全部 .py/.ps1
- **输出：**books/五套算法架构代码汇编.docx/.pdf

运行命令

```
py -3.12 build_codebook.py
```

口径提醒

- **注意：**新增脚本后，把它的条目加进 SCRIPTS 或 APPENDIX_SCRIPTS 再重跑本脚本。

代码全文

```
# -*- coding: utf-8 -*-
"""生成《五套算法架构代码汇编》docx：所有分析脚本 + 用途说明 + 运行命令 + 代码全文。"""
import hashlib
import os
from docx import Document
from docx.enum.text import WD_ALIGN_PARAGRAPH
from docx.oxml import OxmlElement
from docx.oxml.ns import qn
from docx.shared import Cm, Pt, RGBColor

BASE = os.path.dirname(os.path.abspath(__file__))
SERVER = os.path.join(BASE, "server")
OUT = os.path.join(BASE, "books")
os.makedirs(OUT, exist_ok=True)

TITLE = "卒中后癫痫五套算法架构 · 代码汇编"
SUBTITLE = "从原始数据到算法架构全部数字与图表的可执行代码"
VERSION = "v1.0 | 2026-11-07 | 三合医研发团队，河南大学"

SCRIPTS = [
    dict(
        no="2.1", file="tabpfn_bench.py", path=os.path.join(SERVER, "tabpfn_bench.py"),
        papers="A / B / C",
        purpose=("主基准脚本，负责算法架构 A、B、C 三套特征集在 full 与 dedup 两个队列上的全部逐规模结果。"
                 "它按固定种子 42 做分层 80/20 切分，构造嵌套训练子集（n = 100、250、500、1,000、2,000、"
                 "4,000、8,000、16,000 与全训练集），在同一个子集上训练三个模型，并在固定测试集上计算 "
                 "AUROC、AUPRC、Brier，保存 metrics.csv 和逐行预测 test_predictions.csv。TabPFN 只在 "
                 "n ≤ 8,000 上运行（原生上下文上限）；传统模型额外跑到 16,000 与全量。"),
        inputs="data.csv（21,459 例公共队列，SHA-256 见附录 B）",
        outputs="out/<tag>_<features>/metrics.csv；out/<tag>_<features>/test_predictions.csv",
        run="$PY -u tabpfn_bench.py --features A --tag full --out $BASE/out/full_A\n"
            "# 六个组合：--features A/B/C × --tag full/dedup",
        notes=["LR 支路用于交叉核对；算法架构最终 LR 数字由 lr_rerun.py 重算并覆盖，二者预处理一致。",
               "嵌套子集保证不同 n 之间是前缀关系，同一 n 下三模型看到完全相同的患者。",
               "--max-tabpfn-n 默认 10,000，本项目实际评估到 8,000。"],
    ),
    dict(
        no="2.2", file="lr_rerun.py", path=os.path.join(SERVER, "lr_rerun.py"),
        papers="B / C",
        purpose=("只补算 logistic 回归（LR）支路。已有 TabPFN/LightGBM 结果不重跑；脚本读取 "
                 "out/<tag>_<feature>/metrics.csv 与 test_predictions.csv，删掉 model == 'lr' 的旧行与 "
                 "p_lr 旧列，按修正后的预处理顺序重算 B、C × full/dedup 的 LR，再写回原文件。"
                 "关键修复是训练行索引先 np.sort：lbfgs 对行序敏感，不排序会在第 4–5 位小数产生差异。"),
        inputs="已由 tabpfn_bench.py 生成的 4 个组合目录（B/C × full/dedup）",
        outputs="原地覆盖上述目录的 metrics.csv 与 test_predictions.csv（仅 LR 行/列）",
        run="$OLD_PY lr_rerun.py        # sklearn 1.7.2 的 D13 老环境",
        notes=["锚点：B LR 0.9647814390954035 / 0.7872176622723596；C LR 0.9673034710506947 / 0.7984723092638404。",
               "预处理顺序：μ/σ 在原始训练值上估计 → 1/99 缩尾 → 13 列 log1p → 标准化；顺序反了 AUROC 会差到第 3–4 位。",
               "不设置 OMP_NUM_THREADS，保持与归档运行相同的 BLAS 归约顺序。"],
    ),
    dict(
        no="2.3", file="verify_lr_172.py", path=os.path.join(SERVER, "verify_lr_172.py"),
        papers="A / B / C",
        purpose=("最小化验证脚本：只拟合 B、C 全训练集的 LR，打印测试 AUROC/AUPRC/Brier，用来确认当前 "
                 "Python 环境（scikit-learn 1.7.2）能逐位复现归档锚点。它不写文件，只看标准输出。"),
        inputs="data.csv",
        outputs="标准输出（三行指标 + 与锚点的对照）",
```

```
    run="$OLD_PY verify_lr_172.py",
    notes=["预期：B 0.9647814390954035 / 0.7872176622723596；C 0.9673034710506947 / 0.7984723092638404。",
           "任何一位对不上，先检查 sklearn 版本与 OMP_NUM_THREADS，再检查数据文件指纹。"],
),
dict(
    no="2.4", file="bench_analysis.py", path=os.path.join(SERVER, "bench_analysis.py"),
    papers="A / B / C",
    purpose=("汇总与出图。扫描 6 个组合目录，只接受 metrics.csv 与 test_predictions.csv 同时存在、"
             "且三模型 n_train 对齐的组合；在 n = 8,000 上计算校准斜率与截距，在测试行上做 2,000 次"
             "配对 bootstrap，输出 combined_metrics.csv、bench_summary_final.json，以及学习曲线、"
             "ROC/PR、校准曲线、决策曲线四类图（A/B/C 各一套）。"),
    inputs="out/ 下 6 个组合目录（A/B/C × full/dedup）",
    outputs=("out/combined_metrics.csv；out/bench_summary_final.json；"
             "figs/fig_learning_A/B/C.png、fig_roc_pr_*.png、fig_calib_*.png、fig_dca_*.png"),
    run="$PY bench_analysis.py",
    notes=["缺文件或 n 不对齐的组合会被跳过并记入 skipped，不会静默拼数。",
           "校准斜率用 logit(p) 对 y 的 LR（C = 1e6）估计；bootstrap 固定种子 42。",
           "算法架构正文嵌入 ROC/PR、学习、校准、决策四类图；特征重要性图由 2.8 另行生成。"],
),
dict(
    no="2.5", file="table1_baseline.py", path=os.path.join(SERVER, "table1_baseline.py"),
    papers="A",
    purpose=("计算 A 算法架构表 1：36 个入院特征按一年内癫痫结局分组，full 与 dedup 两套；连续变量给"
             "中位数与四分位，二值变量给百分比。结果以 JSON 打到标准输出，重定向保存为 table1.json。"),
    inputs="data.csv",
    outputs="标准输出 → results/table1.json",
    run="py table1_baseline.py data.csv > table1.json",
    notes=["只做描述统计，不建模、不调参。", "dedup 用 df.drop_duplicates()，与主基准完全一致。"],
),
dict(
    no="2.6", file="restore_analyses.py", path=os.path.join(BASE, "restore_analyses.py"),
    papers="A / B / C",
    purpose=("重建三套算法架构的描述性与可解释性结果：A 的表 1（36 个入院特征）、B 的表 1（70 个特征）、"
             "C 的表 1（4 个卒中后并发症）按结局分组的总体/无 PSE/PSE 与 P 值；两模型的特征重要性"
             "（LightGBM 总增益占比、标准化 logistic 回归系数绝对值）；以及 A 的训练集五折交叉验证与"
             "汇总袋外性能。全部在最终口径下重算，输出一个 JSON 供算法架构装配脚本读取。"),
    inputs="data.csv（21,459 例公共队列）",
    outputs="results/restore/restore.json",
    run="py -3.12 restore_analyses.py",
    notes=["连续变量用 Mann–Whitney U、二值变量用 χ²；实验室指标必须按连续变量处理，否则卡方会报负值。",
           "特征重要性在 full 训练集（n = 17,167）上拟合后计算；LR 系数对应协议预处理尺度。",
           "C 的表 1 只含四个并发症变量；A 的表 1 含 range_lobe（任一脑叶受累）。"],
),
dict(
    no="2.7", file="bootstrap_ci.py", path=os.path.join(BASE, "bootstrap_ci.py"),
    papers="A / B / C",
    purpose=("为 A/B/C 的表 2 补 95% bootstrap 置信区间：三个模型 × full/dedup 两个队列 × "
             "n = 8,000 与最大训练规模（17,167 或 10,915）× AUROC/AUPRC/Brier 三项指标，"
             "每项 2,000 次对测试行的重抽样，种子 42。结果写入 ci.json，算法架构表 2 的括号区间即来自这里。"),
    inputs="results/preds_full_*_final.csv、results/preds_dedup_*_final.csv",
    outputs="results/restore/ci.json",
    run="py -3.12 bootstrap_ci.py",
    notes=["公开表无患者标识，只能按行重抽样；这一局限已写进三套算法架构的讨论。",
           "TabPFN 不在 17,167 上评估，对应键不存在，表中以“—”呈现。"],
),
dict(
    no="2.8", file="make_figures_restore.py", path=os.path.join(BASE, "make_figures_restore.py"),
    papers="A / B / C",
    purpose=("生成算法架构图 2（n = 8,000 的 ROC 与精确率–召回率曲线，full 队列）、图 7（同一曲线，"
             "dedup 队列）与图 5/6（特征重要性：LightGBM 总增益与 logistic 回归系数绝对值）。"
             "ROC/PR 一律调用 scikit-learn 的 roc_curve 与 precision_recall_curve，避免手写累积量出错。"),
    inputs="results/preds_{full,dedup}_{A,B,C}_final.csv；results/restore/restore.json",
    outputs="results/figs/fig_roc_pr_{A,B,C}.png、fig_roc_pr_{A,B,C}_dedup.png、fig_importance_{A,B,C}.png",
    run="py -3.12 make_figures_restore.py",
    notes=["三个模型固定配色：LR 蓝、LightGBM 橙、TabPFN 绿。",
           "PR 面板必须用 sklearn 的 precision_recall_curve；早期手写版本把 recall 当 precision 除，纵轴会压到 0–0.005。",
           "生成后把 PNG 复制到 papers/01_A|02_B|03_C/figs/，再由 build_docx5.py 嵌入。"],
),
dict(
    no="3.1", file="wenzhou_de.py", path=os.path.join(SERVER, "wenzhou_de.py"),
    papers="D / E",
    purpose=("温州 167 例的 D、E 两个任务都在这一个脚本里。D：核心 8 变量与全集 24 变量，重复分层 "
             "5 折 × 10 次（种子 42），LR/LightGBM/TabPFN 三模型，输出 CV 指标、学习曲线（种子 "
             "42/43/44，n = 30–134）、校准、决策曲线、风险三分位 KM 与 log-rank、配对 bootstrap。"
             "E：ASM 与 PSE 的粗 OR、多变量校正 OR（bootstrap CI）、倾向评分作为协变量的 OR、"
             "IPTW/AIPW 稳定性检查、SMD 平衡、E 值、KM/RMST（730 天）、90 天界标、分期事件表、"
             "卒中类型分层。"),
    inputs="Wenzhou_167_cases_deidentified.csv（167 例，SHA-256 见附录 B）",
    outputs=("summary_D.json、summary_E.json；cv_summary_core/full.csv、cv_fold_metrics_*.csv、"
```

```
                "learning_curve_*.csv、dca.csv、E/smd_balance.csv；D/E 的 KM、ROC、校准、DCA、学习曲线图"),
        run="$PY wenzhou_de.py --data $DATA --out out/D --task D\n"
            "$PY wenzhou_de.py --data $DATA --out out/E --task E",
        notes=["IPTW/AIPW 只作稳定性检查，不进入算法架构主估计。",
               "E 算法架构不报告单一 Cox 风险比：比例风险假设不成立；脚本中的诊断函数不作为结论。",
               "ASM 增量预测在核心 8 变量上做 50 次重复，逐次比较加/不加 ASM 的 AUROC。"],
    ),
    dict(
        no="3.2", file="table1_wenzhou.py", path=os.path.join(SERVER, "table1_wenzhou.py"),
        papers="D / E",
        purpose=("计算 D、E 算法架构表 1：同一批 167 例，分别按 PSE 结局（D）和 ASM 用药（E）分组，输出总体、"
                 "by_pse、by_asm 三套描述统计与末次发作时间分布。结果 JSON 打到标准输出。"),
        inputs="Wenzhou_167_cases_deidentified.csv",
        outputs="标准输出 → results/table1_wenzhou.json",
        run="py table1_wenzhou.py Wenzhou_167_cases_deidentified.csv > table1_wenzhou.json",
        notes=["连续变量给中位数与四分位，二值变量给百分比。", "列名在脚本开头按固定顺序重命名，防止错列。"],
    ),
]

APPENDIX_SCRIPTS = [
    dict(
        no="A.1", file="tabpfn_smoke.py", path=os.path.join(SERVER, "tabpfn_smoke.py"),
        papers="环境自检",
        purpose=("加载固定权重，检查 TabPFNClassifier 是否支持 ignore_pretraining_limits/device/random_state，"
                 "并在 n = 1,000/4,000/10,000 上各拟合一次，打印耗时与 AUROC。用于确认权重路径、版本和 "
                 "CPU 推理正常，不产生算法架构数字。"),
        inputs="data.csv + 权重文件",
        outputs="标准输出",
        run="$PY tabpfn_smoke.py",
        notes=["第一次在新机器上跑数时先执行它，冒烟通过再跑主基准。"],
    ),
    dict(
        no="D.1", file="build_docx5.py", path=os.path.join(BASE, "build_docx5.py"),
        papers="算法架构装配",
        purpose=("算法架构装配脚本（本机 Windows 运行，不参与科学计算）。读取 papers/ 下的中英文 Markdown 源稿、"
                 "results/ 下的 JSON/CSV 指标、figs/ 图片，生成 10 份算法架构 docx 和《配置要求与复现说明》docx。"
                 "表格中的数字全部来自结果文件，不在脚本里手写。"),
        inputs="papers/*/*.md、papers/*/figs/*.png、results/*.json、results/*.csv",
        outputs="docx/paperA-E_{en,cn}.docx；docx/配置要求与复现说明.docx",
        run="py -3.12 build_docx5.py",
        notes=["表格列宽、跨页重复表头、图注同页等排版规则都写在脚本里。",
               "修改算法架构数字时改 results/ 的源文件，不要改 docx。"],
    ),
    dict(
        no="D.2", file="to_pdf5.ps1", path=os.path.join(BASE, "to_pdf5.ps1"),
        papers="PDF 导出",
        purpose=("用本机 Word 的 COM 接口把 docx/ 下所有 docx 批量另存为 PDF（格式码 17）。只做格式转换，"
                 "不改内容。"),
        inputs="docx/*.docx",
        outputs="docx/*.pdf",
        run="powershell -ExecutionPolicy Bypass -File to_pdf5.ps1",
        notes=["需要本机安装 Microsoft Word。", "转换前先关闭已打开的旧 PDF，避免文件占用。"],
    ),
    dict(
        no="D.3", file="build_resultsbook.py", path=os.path.join(BASE, "build_resultsbook.py"),
        papers="结果汇编",
        purpose=("生成《卒中后癫痫五套算法架构_跑数结果汇编》docx + pdf。全部数字从 results/ 的 JSON/CSV "
                 "读取（144 次 A/B/C 拟合明细、n = 8,000 主结果与 CI、校准、增量、重要性、D/E 全部结果），"
                 "不在脚本里手写任何指标值。"),
        inputs="results/*.json、results/*_final.csv、results/figs/*.png、results/E/*",
        outputs="books/卒中后癫痫五套算法架构_跑数结果汇编.docx/.pdf",
        run="py -3.12 build_resultsbook.py",
        notes=["本书是“账本”：每个表下有源文件脚注，可逐位核对。",
               "修订跑数时先改 results/ 源文件再重跑本脚本，禁止手改表格。"],
    ),
    dict(
        no="D.4", file="build_codebook.py", path=os.path.join(BASE, "build_codebook.py"),
        papers="代码汇编",
        purpose=("生成本书自身（《五套算法架构代码汇编》）。按运行顺序收录全部脚本并逐条注明用途、输入、"
                 "输出与运行命令。"),
        inputs="本目录与 server/ 下全部 .py/.ps1",
        outputs="books/五套算法架构代码汇编.docx/.pdf",
        run="py -3.12 build_codebook.py",
        notes=["新增脚本后，把它的条目加进 SCRIPTS 或 APPENDIX_SCRIPTS 再重跑本脚本。"],
    ),
    dict(
        no="A.2", file="external_transfer.py", path=os.path.join(SERVER, "external_transfer.py"),
        papers="迁移实验（未入册）",
        purpose=("跨队列迁移实验：把 D13 公共队列与温州 167 例共有的 7 个变量对接，检验公共队列上训练的"
```

```
                "模型能否直接迁移到温州队列。结果接近随机，按“不合格的不写进去”的原则未进入任何一册正文。"),
        inputs="data.csv + 温州数据",
        outputs="标准输出（AUROC 等）",
        run="$PY external_transfer.py",
        notes=["保留此脚本是为了完整记录“跑过但不采用”的实验；其阴性结论在 D 的局限节有文字交代。"],
    ),
    dict(
        no="A.3", file="debug_lr.py", path=os.path.join(SERVER, "debug_lr.py"),
        papers="诊断",
        purpose=("一次性诊断脚本：定位两条代码路径的 LR 微小差异（预处理顺序），用于确认最终 LR 数字的"
                 "口径。不产生算法架构数字。"),
        inputs="data.csv",
        outputs="标准输出（对照表）",
        run="$PY debug_lr.py",
        notes=["结论已固化在 lr_rerun.py 的注释里；此脚本仅作归档，无需重跑。"],
    ),
    dict(
        no="A.4", file="debug_loop.py", path=os.path.join(SERVER, "debug_loop.py"),
        papers="诊断",
        purpose=("一次性诊断脚本：复现嵌套子集循环并打印各规模对照，用于核对 metrics CSV 的行数。"
                 "不产生算法架构数字。"),
        inputs="data.csv",
        outputs="标准输出（各规模行数对照）",
        run="$PY debug_loop.py",
        notes=["仅作归档，无需重跑。"],
    ),
    dict(
        no="A.5", file="setup_venv_server.sh", path=os.path.join(SERVER, "setup_venv_server.sh"),
        papers="环境构建",
        purpose=("在服务器上构建 D13 老环境（scikit-learn 1.7.2 的 venv）：先从清华镜像拉取 pip wheel "
                 "直接解包进 venv，再安装 requirements.txt 并自检五个关键包。五套算法架构的 LR 支路"
                 "（lr_rerun.py）就在这个 venv 里复算，才能与归档锚点逐位一致。"),
        inputs="requirements.txt（同目录）",
        outputs="~/D13_20261107/venv（LR 老环境）",
        run="bash setup_venv_server.sh",
        notes=["服务器直连外网被 CMCC 门户劫持时，本脚本的镜像方案即当时的离线安装绕行路径。",
               "新机器复现 LR 数字前先跑它，再跑 lr_rerun.py。"],
    ),
    dict(
        no="A.6", file="requirements.txt", path=os.path.join(SERVER, "requirements.txt"),
        papers="环境依赖",
        purpose=("D13 老环境的依赖清单（numpy/pandas/scikit-learn 1.7.2/lightgbm/matplotlib/joblib 等），"
                 "由 A.5 安装。是 LR 支路可复现性的版本依据。"),
        inputs="—",
        outputs="—",
        run="pip install -r requirements.txt",
        notes=["新环境（LightGBM/TabPFN/D/E 支路）的完整版本表在《配置要求与复现说明》第 2 节。"],
    ),
    dict(
        no="A.7", file="model_d13_server.py", path=os.path.join(SERVER, "model_d13_server.py"),
        papers="前身项目",
        purpose=("D13 前身项目（卒中后癫痫三场景建模：full + dedup 双版）的主脚本。五套算法架构的数据、"
                 "固定划分与 LR 老环境均来自该项目；本脚本归档用于追溯数据管线的来龙去脉。"),
        inputs="~/D13_20261107/data.csv",
        outputs="~/D13_20261107/results/",
        run="venv/bin/python model_d13_server.py",
        notes=["属于前身任务，其数字不进入五套算法架构正文；只作溯源归档。"],
    ),
    dict(
        no="A.8", file="audit_d13.py", path=os.path.join(SERVER, "audit_d13.py"),
        papers="前身项目",
        purpose=("D13 前身项目的审计脚本：核对数据 SHA-256、划分信息与结果文件的一致性。"
                 "五套算法架构沿用其“双端审计”的做法。"),
        inputs="~/D13_20261107/results/",
        outputs="审计报告（标准输出）",
        run="venv/bin/python audit_d13.py",
        notes=["属于前身任务，只作溯源归档。"],
    ),
    dict(
        no="A.9", file="slurm_d13.sh", path=os.path.join(SERVER, "slurm_d13.sh"),
        papers="前身项目",
        purpose=("D13 前身项目的 SLURM 作业提交模板（full / dedup 两个参数化任务）。"
                 "归档用于展示当时的调度方式。"),
        inputs="—",
        outputs="slurm_*.out / slurm_*.err",
        run="sbatch -J d13_full slurm_d13.sh full",
        notes=["属于前身任务，只作溯源归档。"],
    ),
]
```

```
EXCLUDED = []


def set_font(run, size, bold=False, italic=False, mono=False, east="宋体", color=None):
    run.font.name = "Consolas" if mono else "Times New Roman"
    run.font.size = Pt(size)
    run.font.bold = bold
    run.font.italic = italic
    if color:
        run.font.color.rgb = RGBColor(*color)
    rPr = run._element.get_or_add_rPr()
    rFonts = rPr.find(qn("w:rFonts"))
    if rFonts is None:
        rFonts = OxmlElement("w:rFonts")
        rPr.append(rFonts)
    rFonts.set(qn("w:eastAsia"), "宋体" if mono else east)


def para(doc, text="", size=10, bold=False, italic=False, align=None, before=0, after=0,
         indent=None, keep=False, east="宋体", color=None):
    p = doc.add_paragraph()
    if align is not None:
        p.alignment = align
    pf = p.paragraph_format
    pf.space_before = Pt(before)
    pf.space_after = Pt(after)
    pf.line_spacing = 1.0
    if indent is not None:
        pf.left_indent = Cm(indent)
    if keep:
        pf.keep_with_next = True
    if text:
        r = p.add_run(text)
        set_font(r, size, bold=bold, italic=italic, east=east, color=color)
    return p


def add_page_number(section):
    footer = section.footer
    p = footer.paragraphs[0]
    p.alignment = WD_ALIGN_PARAGRAPH.CENTER
    r1 = p.add_run("第 ")
    set_font(r1, 8)
    run = p.add_run()
    set_font(run, 8)
    f1 = OxmlElement("w:fldChar"); f1.set(qn("w:fldCharType"), "begin")
    it = OxmlElement("w:instrText"); it.set(qn("xml:space"), "preserve"); it.text = "PAGE"
    f2 = OxmlElement("w:fldChar"); f2.set(qn("w:fldCharType"), "separate")
    t = OxmlElement("w:t"); t.text = "1"
    f3 = OxmlElement("w:fldChar"); f3.set(qn("w:fldCharType"), "end")
    for el in (f1, it, f2, t, f3):
        run._r.append(el)
    r2 = p.add_run(" 页")
    set_font(r2, 8)


def add_code(doc, text):
    lines = text.splitlines()
    for line in lines:
        p = doc.add_paragraph()
        pf = p.paragraph_format
        pf.space_before = Pt(0)
        pf.space_after = Pt(0)
        pf.line_spacing = Pt(9.2)
        pf.left_indent = Cm(0)
        r = p.add_run(line if line.strip() else " ")
        set_font(r, 7.5, mono=True)


def bullet(doc, label, value):
    p = doc.add_paragraph(style="List Bullet")
    p.paragraph_format.space_before = Pt(1)
    p.paragraph_format.space_after = Pt(1)
    r = p.add_run(label + ": ")
    set_font(r, 9.5, bold=True)
    r2 = p.add_run(value)
    set_font(r2, 9.5)
```

```
def sha256(path):
    h = hashlib.sha256()
    with open(path, "rb") as f:
        for chunk in iter(lambda: f.read(1 << 20), b""):
            h.update(chunk)
    return h.hexdigest()


def script_section(doc, meta, new_page=True):
    if new_page:
        doc.add_page_break()
    p = doc.add_paragraph()
    p.paragraph_format.space_before = Pt(0)
    p.paragraph_format.space_after = Pt(4)
    p.paragraph_format.keep_with_next = True
    r = p.add_run(f"{meta['no']}  {meta['file']}")
    set_font(r, 14, bold=True, east="黑体")
    r2 = p.add_run(f"（{len(open(meta['path'], encoding='utf-8').read().splitlines())} 行）")
    set_font(r2, 9, color=(0x66, 0x66, 0x66))

    para(doc, "用途", 10.5, bold=True, before=6, after=2, keep=True, east="黑体")
    para(doc, meta["purpose"], 9.8, after=4, align=WD_ALIGN_PARAGRAPH.JUSTIFY)

    bullet(doc, "对应算法架构", meta["papers"])
    bullet(doc, "输入", meta["inputs"])
    bullet(doc, "输出", meta["outputs"])

    para(doc, "运行命令", 10.5, bold=True, before=6, after=2, keep=True, east="黑体")
    add_code(doc, meta["run"])

    if meta.get("notes"):
        para(doc, "口径提醒", 10.5, bold=True, before=6, after=2, keep=True, east="黑体")
        for n in meta["notes"]:
            bullet(doc, "注意", n)

    para(doc, "代码全文", 10.5, bold=True, before=8, after=2, keep=True, east="黑体")
    add_code(doc, open(meta["path"], encoding="utf-8").read())


def main():
    doc = Document()
    sec = doc.sections[0]
    sec.page_width, sec.page_height = Cm(21), Cm(29.7)
    sec.top_margin = sec.bottom_margin = Cm(2.0)
    sec.left_margin = sec.right_margin = Cm(1.6)
    add_page_number(sec)

    # ---------------- 封面
    para(doc, "", 10, before=120)
    para(doc, TITLE, 22, bold=True, align=WD_ALIGN_PARAGRAPH.CENTER, after=6, east="黑体")
    para(doc, SUBTITLE, 12, align=WD_ALIGN_PARAGRAPH.CENTER, after=18)
    para(doc, VERSION, 10, align=WD_ALIGN_PARAGRAPH.CENTER, after=4)
    para(doc, "收录 10 个分析脚本 + 13 个辅助/诊断/装配/环境脚本，共 5,700 余行", 9,
         align=WD_ALIGN_PARAGRAPH.CENTER, italic=True, after=2)
    para(doc, "所有代码均为实际运行版本，未做删改；中文注释保留原样", 9,
         align=WD_ALIGN_PARAGRAPH.CENTER, italic=True)

    # ---------------- 使用说明
    doc.add_page_break()
    para(doc, "0. 这份 PDF 是什么", 16, bold=True, before=0, after=8, east="黑体")
    para(doc, ("这份汇编把五套算法架构从原始数据到最终数字、表格与图表的代码按运行顺序完整收录，"
               "目的是让审稿人、合作者或下一位接手的人能对照算法架构逐段找到“这个数字是哪个脚本、哪一行算出来的”。"
               "每一节先说明脚本做什么、吃什么文件、吐什么结果，再贴代码全文。"), 10, after=6,
         align=WD_ALIGN_PARAGRAPH.JUSTIFY)

    para(doc, "0.1 怎么读", 12, bold=True, before=8, after=3, east="黑体")
    for t in ["第一次读：先看第 1 节的“代码总览”，知道一共有哪些脚本、各自对应哪套算法架构。",
              "要复现：按 2.1 → 2.8 的顺序跑 A/B/C；按 3.1 → 3.2 跑 D/E。",
              "要核对某个数字：先在算法架构里找到表/图编号，再到第 1 节的对照表找到脚本，最后在脚本里搜指标名。",
              "代码中的 $PY、$OLD_PY、$BASE、$DATA 是环境变量占位符，具体路径见《配置要求与复现说明》。"]:
        bullet(doc, "提示", t)

    para(doc, "0.2 运行环境（一句话版）", 12, bold=True, before=8, after=3, east="黑体")
    para(doc, ("A/B/C 的 LightGBM 与 TabPFN 在 scikit-learn 1.9.1 的新环境运行；LR 支路在 "
               "scikit-learn 1.7.2 的 D13 老环境复算，才能与归档锚点逐位一致。D/E 全部在新环境运行。"
               "两套环境的完整版本表、三条 SHA-256 与逐条命令见《配置要求与复现说明》第 2–6 节。"),
         10, after=6, align=WD_ALIGN_PARAGRAPH.JUSTIFY)
```

```
    para(doc, "0.3 收录边界（重要）", 12, bold=True, before=8, after=3, east="黑体")
    para(doc, ("本书收录服务器与本机上与该项目相关的全部 23 个脚本：10 个分析脚本 + 13 个辅助/诊断/装配/环境脚本。"
               "其中 A.2 跨队列迁移实验的结果接近随机，按“不合格的不写进去”的原则未进入任何一册正文；"
               "A.3、A.4 是一次性诊断脚本，不产生算法架构数字；A.5–A.6 是 LR 老环境的构建脚本与依赖清单；A.7–A.9 是前身项目 D13 的主脚本、审计与提交脚本（只作溯源，其数字不进入正文）。收进来是为了让读者看到全部跑过的代码，"
               "而不是只看到成功的那一部分——每个条目都注明了自己的用途与边界。"), 10, after=4,
         align=WD_ALIGN_PARAGRAPH.JUSTIFY)

    # ---------------- 总览
    doc.add_page_break()
    para(doc, "1. 代码总览", 16, bold=True, before=0, after=6, east="黑体")
    para(doc, "表 1. 脚本、行数与算法架构对应关系", 9, bold=True, align=WD_ALIGN_PARAGRAPH.CENTER, after=2)

    rows = []
    for m in SCRIPTS + APPENDIX_SCRIPTS:
        n = len(open(m["path"], encoding="utf-8").read().splitlines())
        rows.append([m["no"], m["file"], str(n), m["papers"]])
    tbl = doc.add_table(rows=1, cols=4)
    tbl.style = "Table Grid"
    tbl.alignment = 1
    hdr = ["编号", "脚本", "行数", "对应算法架构/用途"]
    widths = [2.0, 6.6, 1.8, 7.2]
    for j, h in enumerate(hdr):
        c = tbl.rows[0].cells[j]
        c.width = Cm(widths[j])
        pp = c.paragraphs[0]
        pp.alignment = WD_ALIGN_PARAGRAPH.CENTER
        r = pp.add_run(h)
        set_font(r, 9, bold=True)
    for row in rows:
        cells = tbl.add_row().cells
        for j, v in enumerate(row):
            cells[j].width = Cm(widths[j])
            pp = cells[j].paragraphs[0]
            pp.alignment = WD_ALIGN_PARAGRAPH.CENTER if j != 1 else WD_ALIGN_PARAGRAPH.LEFT
            r = pp.add_run(v)
            set_font(r, 8.5)

    para(doc, "", 6, after=4)
    para(doc, "1.1 算法架构–脚本–产出一览", 12, bold=True, before=6, after=3, east="黑体")
    mapping = [
        ("A 入院特征基准", "2.1 + 2.2 + 2.4 + 2.6 + 2.7 + 2.8", "表 1–4；图 2–7"),
        ("B 实验室增量", "2.1 + 2.2 + 2.4 + 2.6 + 2.7 + 2.8", "表 1–3；图 2–7"),
        ("C 并发症边界检验", "2.1 + 2.2 + 2.4 + 2.6 + 2.7 + 2.8", "表 1–3；图 2–7"),
        ("D 首次发作时点预测", "wenzhou_de.py + table1_wenzhou.py", "表 1、表 2；图 2–4"),
        ("E 抗癫痫药物分析", "wenzhou_de.py + table1_wenzhou.py", "表 1；图 2"),
    ]
    t2 = doc.add_table(rows=1, cols=3)
    t2.style = "Table Grid"
    t2.alignment = 1
    for j, h in enumerate(["算法架构", "核心脚本", "算法架构中的表/图"]):
        c = t2.rows[0].cells[j]
        c.width = Cm([5.0, 7.4, 5.2][j])
        pp = c.paragraphs[0]
        pp.alignment = WD_ALIGN_PARAGRAPH.CENTER
        r = pp.add_run(h)
        set_font(r, 9, bold=True)
    for row in mapping:
        cells = t2.add_row().cells
        for j, v in enumerate(row):
            cells[j].width = Cm([5.0, 7.4, 5.2][j])
            pp = cells[j].paragraphs[0]
            pp.alignment = WD_ALIGN_PARAGRAPH.LEFT if j == 0 else WD_ALIGN_PARAGRAPH.CENTER
            r = pp.add_run(v)
            set_font(r, 8.5)

    # ---------------- 正文
    doc.add_page_break()
    para(doc, "2. A / B / C：21,459 例公共队列", 16, bold=True, before=0, after=6, east="黑体")
    para(doc, ("A/B/C 共用一套基准管线：固定切分 → 嵌套子集 → 三模型训练 → 固定测试集评估 → 汇总出图。"
               "差别只在特征集：A 为 36 个入院特征，B 加 34 项实验室指标（70 个），C 再加 4 个卒中后并发症"
               "（74 个）。下面八个脚本按实际运行顺序排列。"), 10, after=4,
         align=WD_ALIGN_PARAGRAPH.JUSTIFY)
    for m in SCRIPTS[:8]:
        script_section(doc, m)

    doc.add_page_break()
    para(doc, "3. D / E：温州 167 例真实队列", 16, bold=True, before=0, after=6, east="黑体")
    para(doc, ("D/E 使用同一个单中心去标识队列（167 例，PSE 49 例，中位随访 592 天）。D 研究首次发作"
```

```
            "时点的预测，E 研究抗癫痫药物与晚期 PSE 的关联。所有分析在一个脚本内按 --task 分流，"
            "表 1 由独立脚本生成。"), 10, after=4, align=WD_ALIGN_PARAGRAPH.JUSTIFY)
    for m in SCRIPTS[8:]:
        script_section(doc, m)

    # ---------------- 附录
    doc.add_page_break()
    para(doc, "附录 A  环境自检脚本", 16, bold=True, before=0, after=6, east="黑体")
    script_section(doc, APPENDIX_SCRIPTS[0])

    doc.add_page_break()
    para(doc, "附录 B  脚本指纹", 16, bold=True, before=0, after=4, east="黑体")
    para(doc, ("下表给出本汇编收录的每个文件的 SHA-256 与行数。改动任何一个字符，指纹都会变；"
               "交接时先核对指纹，再谈复现。"), 9.8, after=4)
    files = [(m["file"], m["path"]) for m in SCRIPTS + APPENDIX_SCRIPTS]
    tb = doc.add_table(rows=1, cols=4)
    tb.style = "Table Grid"
    tb.alignment = 1
    for j, h in enumerate(["文件", "行数", "SHA-256（前 32 位）", "完整 SHA-256"]):
        c = tb.rows[0].cells[j]
        c.width = Cm([3.6, 1.4, 5.6, 7.0][j])
        pp = c.paragraphs[0]
        pp.alignment = WD_ALIGN_PARAGRAPH.CENTER
        r = pp.add_run(h)
        set_font(r, 8.5, bold=True)
    for name, path in files:
        n = len(open(path, encoding="utf-8").read().splitlines())
        digest = sha256(path)
        cells = tb.add_row().cells
        vals = [name, str(n), digest[:32], digest]
        for j, v in enumerate(vals):
            cells[j].width = Cm([3.6, 1.4, 5.6, 7.0][j])
            pp = cells[j].paragraphs[0]
            pp.alignment = WD_ALIGN_PARAGRAPH.LEFT if j in (0, 3) else WD_ALIGN_PARAGRAPH.CENTER
            r = pp.add_run(v)
            set_font(r, 7.2 if j >= 2 else 8)

    para(doc, "", 6, after=4)
    para(doc, "附录 C  未收录脚本说明", 16, bold=True, before=8, after=4, east="黑体")
    for name, why in EXCLUDED:
        bullet(doc, name, why)
    para(doc, ("这三类脚本不改变算法架构任何数字。若需要审计“所有跑过的代码”，它们仍在 server/ 目录中，"
               "与本汇编收录版本同源。"), 9.5, after=4, italic=True)

    para(doc, "附录 D  算法架构装配脚本", 16, bold=True, before=10, after=4, east="黑体")
    para(doc, ("以下两个脚本用于把结果装配成 docx/PDF，不属于统计分析。放在附录是为了在需要重新生成"
               "算法架构文件时不必另找。"), 9.8, after=4)
    for m in APPENDIX_SCRIPTS[1:]:
        script_section(doc, m)

    out = os.path.join(OUT, "五套算法架构代码汇编.docx")
    doc.save(out)
    print("saved", out)


if __name__ == "__main__":
    main()
```

A.2 external_transfer.py（153 行）

用途

跨队列迁移实验：把 D13 公共队列与温州 167 例共有的 7 个变量对接，检验公共队列上训练的模型能否直接迁移到温州队列。结果接近随机，按“不合格的不写进去”的原则未进入任何一册正文。

- **对应算法架构：**迁移实验（未入册）
- **输入：**data.csv + 温州数据
- **输出：**标准输出（AUROC 等）

运行命令

```
$PY external_transfer.py
```

口径提醒

- **注意：**保留此脚本是为了完整记录“跑过但不采用”的实验；其阴性结论在 D 的局限节有文字交代。

代码全文

```
# -*- coding: utf-8 -*-
"""跨队列迁移实验：D13 公共队列（21,459）⇄ 温州 167 例真实队列。

两队列共有的 7 个变量：age, gender, hypertension, diabetes, hyperuricemia,
hyperlipidaemia, nihss。三个模型（LR / LightGBM / TabPFN）在两个方向上都评估：
  D13 → Wenzhou：在大队列上训练，直接迁移到小队列（外部验证）；
  Wenzhou → D13：在小队列上训练，迁移到大队列的固定测试集（小样本迁移）。
指标：AUROC、AUPRC、Brier、校准斜率/截距（logistic 重校准），并保存预测。

用法:
  python external_transfer.py --d13 ~/D13_20261107/data.csv \
      --wenzhou ~/WENZHOU_DE_20261107/data/Wenzhou_167_cases_deidentified.csv \
      --out out/transfer
"""
import argparse
import json
import warnings
from pathlib import Path

import numpy as np
import pandas as pd
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import average_precision_score, brier_score_loss, roc_auc_score
from sklearn.model_selection import train_test_split

warnings.filterwarnings("ignore")
SEED = 42
WEIGHTS = Path.home() / "TABPFN_BENCH_20261107/weights/tabpfn-v2-classifier-finetuned-zk73skhh.ckpt"
COMMON = ["age", "gender", "hypertension", "diabetes", "hyperuricemia",
          "hyperlipidaemia", "nihss"]


def prep_lr(Xtr, Xte):
    lo = np.percentile(Xtr, 1, axis=0)
    hi = np.percentile(Xtr, 99, axis=0)
    Xtr = np.clip(Xtr, lo, hi)
    Xte = np.clip(Xte, lo, hi)
    mu = Xtr.mean(0)
    sd = Xtr.std(0)
    sd[sd == 0] = 1
    return (Xtr - mu) / sd, (Xte - mu) / sd


def fit_pred(name, Xtr, ytr, Xte):
    from lightgbm import LGBMClassifier
    if name == "lr":
        a, b = prep_lr(Xtr.copy(), Xte.copy())
        m = LogisticRegression(C=1.0, max_iter=5000, random_state=SEED).fit(a, ytr)
        return m.predict_proba(b)[:, 1]
    if name == "lgbm":
        g = LGBMClassifier(n_estimators=300, learning_rate=0.05, num_leaves=31,
                           min_child_samples=50, subsample=0.8, colsample_bytree=0.8,
                           reg_alpha=0.1, reg_lambda=0.1, random_state=SEED,
                           n_jobs=8, verbosity=-1)
        g.fit(Xtr, ytr)
        return g.predict_proba(Xte)[:, 1]
```

```
    from tabpfn import TabPFNClassifier
    t = TabPFNClassifier(model_path=str(WEIGHTS), n_estimators="auto", random_state=SEED,
                         ignore_pretraining_limits=True, device="cpu")
    t.fit(Xtr, ytr)
    return t.predict_proba(Xte)[:, 1]


def calib(y, p):
    lp = np.log(np.clip(p, 1e-6, 1 - 1e-6) / (1 - np.clip(p, 1e-6, 1 - 1e-6)))
    m = LogisticRegression(C=1e6, max_iter=1000).fit(lp.reshape(-1, 1), y)
    return float(m.coef_[0, 0]), float(m.intercept_[0])


def metrics(y, p):
    s, i = calib(y, p)
    return {"n": int(len(y)), "events": int(y.sum()),
            "prevalence": float(y.mean()),
            "auroc": float(roc_auc_score(y, p)),
            "auprc": float(average_precision_score(y, p)),
            "brier": float(brier_score_loss(y, p)),
            "cal_slope": s, "cal_intercept": i}


def main():
    ap = argparse.ArgumentParser()
    ap.add_argument("--d13", required=True)
    ap.add_argument("--wenzhou", required=True)
    ap.add_argument("--out", required=True)
    ap.add_argument("--d13-sub", type=int, default=167, help="小样本迁移时 D13 训练子集大小")
    args = ap.parse_args()
    out = Path(args.out); out.mkdir(parents=True, exist_ok=True)

    d13 = pd.read_csv(args.d13)
    d13 = d13.rename(columns={"hyperlipidaemia": "hyperlipidaemia"})
    wx = pd.read_csv(args.wenzhou)
    wx.columns = ["row", "stroke_type", "sex", "age", "age_lt65", "alcohol", "smoking",
                  "hypertension", "diabetes", "hyperlipidemia", "hyperuricemia",
                  "seizure_type", "interval_days", "interval_gt3", "multiple_seizures",
                  "status_epilepticus", "eeg", "asm", "nihss", "nihss_gt14", "infection",
                  "lesion_vol", "lesion_gt10", "cortical", "multilobar", "deep",
                  "ventricular_ext", "surgery", "toast", "thrombolysis", "lipid_lowering",
                  "antiplatelet", "anticoag", "pse", "followup_days", "late_seizure_days",
                  "censor_days"]
    wx = wx.rename(columns={"hyperlipidemia": "hyperlipidaemia", "sex": "gender"})
    Xd = d13[COMMON].to_numpy(float); yd = d13.second_epilepsy.to_numpy(int)
    Xw = wx[COMMON].to_numpy(float); yw = wx.pse.to_numpy(int)

    # D13 固定 80/20 切分（与主分析同种子）
    Xtr, Xte, ytr, yte = train_test_split(Xd, yd, test_size=0.2, stratify=yd, random_state=SEED)
    res, preds = {}, {}

    # 方向一：D13 全量训练 → 温州外部验证
    for name in ["lr", "lgbm", "tabpfn"]:
        Xtr_use, ytr_use = (Xtr, ytr)
        if name == "tabpfn" and len(ytr) > 10000:  # TabPFN 原生上限 10,000
            rng = np.random.default_rng(SEED)
            idx = rng.choice(len(ytr), 10000, replace=False)
            Xtr_use, ytr_use = Xtr[idx], ytr[idx]
        p = fit_pred(name, Xtr_use, ytr_use, Xw)
        preds[f"d13_to_wx_{name}"] = p
        res[f"d13_to_wx_{name}"] = {**metrics(yw, p), "n_train": int(len(ytr_use))}

    # 方向二：D13 小样本（167）→ 温州；同时 D13 自身测试集作参照
    rng = np.random.default_rng(SEED)
    pos = np.where(ytr == 1)[0]; neg = np.where(ytr == 0)[0]
    pn = int(round(args.d13_sub * len(pos) / len(ytr)))
    idx = np.concatenate([rng.choice(pos, pn, replace=False),
                          rng.choice(neg, args.d13_sub - pn, replace=False)])
    for name in ["lr", "lgbm", "tabpfn"]:
        p = fit_pred(name, Xtr[idx], ytr[idx], Xw)
        preds[f"d13small_to_wx_{name}"] = p
        res[f"d13small_to_wx_{name}"] = {**metrics(yw, p), "n_train": int(len(idx))}

    # 方向三：温州（167）训练 → D13 测试集
    for name in ["lr", "lgbm", "tabpfn"]:
        p = fit_pred(name, Xw, yw, Xte)
        preds[f"wx_to_d13_{name}"] = p
        res[f"wx_to_d13_{name}"] = {**metrics(yte, p), "n_train": int(len(yw))}
```

```
    # 方向四：D13 全量 → D13 自身（参照上界）
    for name in ["lr", "lgbm"]:
        p = fit_pred(name, Xtr, ytr, Xte)
        preds[f"d13_to_d13_{name}"] = p
        res[f"d13_to_d13_{name}"] = {**metrics(yte, p), "n_train": int(len(ytr))}

    (out / "transfer_results.json").write_text(
        json.dumps(res, ensure_ascii=False, indent=2), encoding="utf-8")
    wx_cols = {k: v for k, v in preds.items() if k.endswith("_to_wx_lr") or "_to_wx_" in k}
    d13_cols = {k: v for k, v in preds.items() if "_to_d13_" in k}
    pd.DataFrame({"y_wx": yw, **wx_cols}).to_csv(out / "predictions_wenzhou.csv", index=False)
    pd.DataFrame({"y_d13": yte, **d13_cols}).to_csv(out / "predictions_d13.csv", index=False)
    print(json.dumps(res, ensure_ascii=False, indent=2))
    print("DONE", out)


if __name__ == "__main__":
    main()
```

A.3 debug_lr.py（73 行）

用途

一次性诊断脚本：定位两条代码路径的 LR 微小差异（预处理顺序），用于确认最终 LR 数字的口径。不产生算法架构数字。

- **对应算法架构：**诊断
- **输入：**data.csv
- **输出：**标准输出（对照表）

运行命令

```
$PY debug_lr.py
```

口径提醒

- **注意：**结论已固化在 lr_rerun.py 的注释里；此脚本仅作归档，无需重跑。

代码全文

```
# -*- coding: utf-8 -*-
"""对照两条代码路径在 B 全训练集上的 LR 结果，定位微小差异来源。"""
import numpy as np
import pandas as pd
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import average_precision_score, roc_auc_score
from sklearn.model_selection import train_test_split

SEED = 42
A_DEMO = ["age", "gender", "nihss"]
A_COMORB = ["uremia", "fatty_liver", "diabetes", "hypertension", "coronary_disease",
            "atrial_fibrillation", "hyperuricemia", "hyperlipidaemia"]
A_BRAIN = ["frontal_lobe", "parietal_lobe", "temporal_lobe", "occipital_lobe",
           "insular_lobe", "range_lobe", "basal_ganglia", "capsula_interna",
           "brainstem", "epencephalon", "paraventricular", "centrum_semiovale",
           "thalamus", "subcortex_lobe"]
A_VESSEL = ["aca", "mca", "pca", "va", "ba", "cca_plaque", "ica_plaque", "eca_plaque",
            "ant_circle", "post_circle", "large_ves_as"]
FEATURES_A = A_DEMO + A_COMORB + A_BRAIN + A_VESSEL
LABS = ["plt", "wbc", "rbc", "hba1c", "crp", "tg", "ldl", "hdl", "ast", "alt",
        "bilirubin", "albumin", "urea", "creatinine", "bua", "pt", "aptt", "tt",
        "inr", "d_dimer", "fibrinogen", "ck", "ck_mb", "ldh", "hbdh", "ima",
        "na", "k", "cl", "ca", "p", "lactate", "anion_gap", "tco2"]
POST = ["dvt", "cerebral_hernia", "hydrocephalus", "hypoproteinemia"]
FS = {"A": FEATURES_A, "B": FEATURES_A + LABS, "C": FEATURES_A + LABS + POST}
LOG1P_COLS = ["crp", "ast", "alt", "creatinine", "d_dimer", "ck", "ck_mb",
              "ldh", "hbdh", "ima", "anion_gap", "tg", "bua"]
ORD = ["age", "nihss", "range_lobe"]


def lr_transform(Xtr, Xte, cols):
    Xa, Xb = Xtr.copy(), Xte.copy()
    for j, c in enumerate(cols):
        if c in ORD or c in LABS:
            mu, sd = Xa[:, j].mean(), Xa[:, j].std()
            sd = sd if sd > 1e-12 else 1.0
        if c in LABS:
            lo = np.percentile(Xtr[:, j], 1)
            hi = np.percentile(Xtr[:, j], 99)
            Xa[:, j] = np.clip(Xa[:, j], lo, hi)
            Xb[:, j] = np.clip(Xb[:, j], lo, hi)
            if c in LOG1P_COLS:
                Xa[:, j] = np.log1p(np.maximum(Xa[:, j], 0))
                Xb[:, j] = np.log1p(np.maximum(Xb[:, j], 0))
        if c in ORD or c in LABS:
            Xa[:, j] = (Xa[:, j] - mu) / sd
            Xb[:, j] = (Xb[:, j] - mu) / sd
    return Xa, Xb


df = pd.read_csv("/home/zxyaicenter2/D13_20261107/data.csv")
y = df["second_epilepsy"].to_numpy(int)

# 路径一：verify 风格（C 全列切前 70 列）
Xc = df[FS["C"]].to_numpy(float)
Xtr1, Xte1, ytr1, yte1 = train_test_split(Xc, y, test_size=0.2, stratify=y, random_state=SEED)
```

```
Xa1, Xb1 = lr_transform(Xtr1[:, :70].copy(), Xte1[:, :70].copy(), FS["B"])
m1 = LogisticRegression(C=1.0, max_iter=3000).fit(Xa1, ytr1)
p1 = m1.predict_proba(Xb1)[:, 1]
print("path1 auroc", roc_auc_score(yte1, p1), "auprc", average_precision_score(yte1, p1))

# 路径二：lr_rerun 风格（B 列直接取数）
Xb2 = df[FS["B"]].to_numpy(float)
Xtr2, Xte2, ytr2, yte2 = train_test_split(Xb2, y, test_size=0.2, stratify=y, random_state=SEED)
print("split same?", np.array_equal(ytr1, ytr2), np.array_equal(Xtr1[:, :70], Xtr2),
      np.array_equal(Xte1[:, :70], Xte2), len(ytr1), len(ytr2))
Xa3, Xb3 = lr_transform(Xtr2.copy(), Xte2.copy(), FS["B"])
print("transform diff:", np.abs(Xa3 - Xa1).max(), np.abs(Xb3 - Xb1).max())
m2 = LogisticRegression(C=1.0, max_iter=3000, random_state=SEED).fit(Xa3, ytr2)
p2 = m2.predict_proba(Xb3)[:, 1]
print("path2 auroc", roc_auc_score(yte2, p2), "auprc", average_precision_score(yte2, p2))
print("coef diff:", np.abs(m1.coef_ - m2.coef_).max())
print("n_iter:", m1.n_iter_, m2.n_iter_)
```

A.4 debug_loop.py（79 行）

用途

一次性诊断脚本：复现嵌套子集循环并打印各规模对照，用于核对 metrics CSV 的行数。不产生算法架构数字。

- **对应算法架构：**诊断
- **输入：**data.csv
- **输出：**标准输出（各规模行数对照）

运行命令

```
$PY debug_loop.py
```

口径提醒

- **注意：**仅作归档，无需重跑。

代码全文

```
# -*- coding: utf-8 -*-
"""复现 lr_rerun 的嵌套子集循环，打印 full/B 在 n=16000 与 n=17167 的 AUROC，与 metrics.csv 对照。"""
import numpy as np
import pandas as pd
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import average_precision_score, roc_auc_score
from sklearn.model_selection import train_test_split

SEED = 42
A_DEMO = ["age", "gender", "nihss"]
A_COMORB = ["uremia", "fatty_liver", "diabetes", "hypertension", "coronary_disease",
            "atrial_fibrillation", "hyperuricemia", "hyperlipidaemia"]
A_BRAIN = ["frontal_lobe", "parietal_lobe", "temporal_lobe", "occipital_lobe",
           "insular_lobe", "range_lobe", "basal_ganglia", "capsula_interna",
           "brainstem", "epencephalon", "paraventricular", "centrum_semiovale",
           "thalamus", "subcortex_lobe"]
A_VESSEL = ["aca", "mca", "pca", "va", "ba", "cca_plaque", "ica_plaque", "eca_plaque",
            "ant_circle", "post_circle", "large_ves_as"]
FEATURES_A = A_DEMO + A_COMORB + A_BRAIN + A_VESSEL
LABS = ["plt", "wbc", "rbc", "hba1c", "crp", "tg", "ldl", "hdl", "ast", "alt",
        "bilirubin", "albumin", "urea", "creatinine", "bua", "pt", "aptt", "tt",
        "inr", "d_dimer", "fibrinogen", "ck", "ck_mb", "ldh", "hbdh", "ima",
        "na", "k", "cl", "ca", "p", "lactate", "anion_gap", "tco2"]
POST = ["dvt", "cerebral_hernia", "hydrocephalus", "hypoproteinemia"]
FS = {"A": FEATURES_A, "B": FEATURES_A + LABS, "C": FEATURES_A + LABS + POST}
LOG1P_COLS = ["crp", "ast", "alt", "creatinine", "d_dimer", "ck", "ck_mb",
              "ldh", "hbdh", "ima", "anion_gap", "tg", "bua"]
ORD = ["age", "nihss", "range_lobe"]
SIZES = [100, 250, 500, 1000, 2000, 4000, 8000, 16000]


def lr_transform(Xtr, Xte, cols):
    Xa, Xb = Xtr.copy(), Xte.copy()
    for j, c in enumerate(cols):
        if c in ORD or c in LABS:
            mu, sd = Xa[:, j].mean(), Xa[:, j].std()
            sd = sd if sd > 1e-12 else 1.0
        if c in LABS:
            lo = np.percentile(Xtr[:, j], 1)
            hi = np.percentile(Xtr[:, j], 99)
            Xa[:, j] = np.clip(Xa[:, j], lo, hi)
            Xb[:, j] = np.clip(Xb[:, j], lo, hi)
            if c in LOG1P_COLS:
                Xa[:, j] = np.log1p(np.maximum(Xa[:, j], 0))
                Xb[:, j] = np.log1p(np.maximum(Xb[:, j], 0))
        if c in ORD or c in LABS:
            Xa[:, j] = (Xa[:, j] - mu) / sd
            Xb[:, j] = (Xb[:, j] - mu) / sd
    return Xa, Xb


def nested_subsets(y, sizes, seed=SEED):
    rng = np.random.default_rng(seed)
    pos = rng.permutation(np.where(y == 1)[0])
    neg = rng.permutation(np.where(y == 0)[0])
    for n in sizes:
        p = int(round(n * len(pos) / len(y)))
```

```
        p = min(max(p, 1), len(pos))
        q = min(max(n - p, 1), len(neg))
        yield n, np.concatenate([pos[:p], neg[:q]])


df = pd.read_csv("/home/zxyaicenter2/D13_20261107/data.csv")
cols = FS["B"]
X = df[cols].to_numpy(float)
y = df["second_epilepsy"].to_numpy(int)
Xtr_all, Xte, ytr_all, yte = train_test_split(X, y, test_size=0.2, stratify=y, random_state=SEED)
sizes = [s for s in SIZES if s < len(ytr_all)] + [len(ytr_all)]
print("len ytr", len(ytr_all), "pos", int(ytr_all.sum()), "sizes", sizes)
for n, idx in nested_subsets(ytr_all, sizes):
    if n not in (16000, len(ytr_all)):
        continue
    idx_sorted = np.sort(idx)
    print(n, "idx len", len(idx_sorted), "identity?", np.array_equal(idx_sorted, np.arange(len(ytr_all))))
    Xtr, ytr = Xtr_all[idx_sorted], ytr_all[idx_sorted]
    Xa, Xb = lr_transform(Xtr, Xte, cols)
    m = LogisticRegression(C=1.0, max_iter=3000, random_state=SEED).fit(Xa, ytr)
    p = m.predict_proba(Xb)[:, 1]
    print("  loop auroc", roc_auc_score(yte, p), "auprc", average_precision_score(yte, p))
```

A.5 setup_venv_server.sh（24 行）

用途

在服务器上构建 D13 老环境（scikit-learn 1.7.2 的 venv）：先从清华镜像拉取 pip wheel 直接解包进 venv，再安装 requirements.txt 并自检五个关键包。五套算法架构的 LR 支路（lr_rerun.py）就在这个 venv 里复算，才能与归档锚点逐位一致。

- **对应算法架构：**环境构建
- **输入：**requirements.txt（同目录）
- **输出：**~/D13_20261107/venv（LR 老环境）

运行命令

```
bash setup_venv_server.sh
```

口径提醒

- **注意：**服务器直连外网被 CMCC 门户劫持时，本脚本的镜像方案即当时的离线安装绕行路径。
- **注意：**新机器复现 LR 数字前先跑它，再跑 lr_rerun.py。

代码全文

```
#!/bin/bash
# 服务器端：清华镜像引导 pip 到 venv，并安装 D13 依赖（不依赖 ensurepip/系统 pip）
set -euo pipefail
cd ~/D13_20261107
MIRROR=https://pypi.tuna.tsinghua.edu.cn

PIPURL=$(curl -s --max-time 60 "$MIRROR/simple/pip/" \
  | grep -o 'href="[^"]*pip-[0-9][0-9.]*-py3-none-any\.whl[^"]*"' \
  | tail -1 | sed 's/^href="//; s/"$//')
case "$PIPURL" in
  http*) ;;
  *) PIPURL="$MIRROR$PIPURL" ;;
esac
echo "pip wheel: $PIPURL"

curl -s --max-time 120 -O "$PIPURL"
WHEEL=$(basename "$PIPURL")
unzip -q -o "$WHEEL" -d venv/lib/python3.12/site-packages/
rm -f "$WHEEL"

./venv/bin/python -m pip --version
./venv/bin/pip install -q -r requirements.txt
./venv/bin/python -c 'import numpy,pandas,sklearn,lightgbm,matplotlib,joblib; print("deps ok", numpy.__version__,
pandas.__version__, sklearn.__version__, lightgbm.__version__)'
echo "VENV_READY"
```

A.6 requirements.txt（7 行）

用途

D13 老环境的依赖清单（numpy/pandas/scikit-learn 1.7.2/lightgbm/matplotlib/joblib 等），由 A.5 安装。是 LR 支路可复现性的版本依据。

- **对应算法架构：**环境依赖
- **输入：**—
- **输出：**—

运行命令

```
pip install -r requirements.txt
```

口径提醒

- **注意：**新环境（LightGBM/TabPFN/D/E 支路）的完整版本表在《配置要求与复现说明》第 2 节。

代码全文

```
# D13 三场景建模 服务器 venv 依赖（安装后以 env_record.txt 实际版本为准）
numpy>=1.26
pandas>=2.1
scikit-learn>=1.4
lightgbm>=4.3
matplotlib>=3.8
joblib>=1.3
```

A.7 model_d13_server.py（564 行）

用途

D13 前身项目（卒中后癫痫三场景建模：full + dedup 双版）的主脚本。五套算法架构的数据、固定划分与 LR 老环境均来自该项目；本脚本归档用于追溯数据管线的来龙去脉。

- **对应算法架构：**前身项目
- **输入：**~/D13_20261107/data.csv
- **输出：**~/D13_20261107/results/

运行命令

```
venv/bin/python model_d13_server.py
```

口径提醒

- **注意：**属于前身任务，其数字不进入五套算法架构正文；只作溯源归档。

代码全文

```
# -*- coding: utf-8 -*-
r"""
================================================================================
D13 缺血性卒中后一年内继发性癫痫 —— 三场景建模（服务器运行版）
================================================================================

【科学口径】与冻结方案《D13 变量分组与建模起步方案》及原型 model_d13.py 完全一致：
    - 结局 second_epilepsy（一年内继发性癫痫二分类标签）
    - 特征集 A(36) / B(70) / C(74)
    - seed 42 分层 80/20 切分；训练集内 5 折分层 CV
    - LR(C=1.0 L2, max_iter=3000, 无类权重)：折内 winsorize(1%/99%) + log1p(13 列) + 标准化
    - LightGBM(CV: 1000 树 lr0.05 31 叶 min_child50 sub0.8 col0.8 scale_pos_weight
      seed42 验证折早停 50；测试模型: 300 树同参数)：原始数值
    - 指标：AUROC / AUPRC / Brier（CV 均值±SD、OOF 拼接、测试集单值）
      + 测试集 ROC/PR、校准曲线、决策曲线 + 稀疏二值变量 2×2 频数
    - 敏感性分析：full（保留完全重复行，主分析）/ dedup（去重后）

【工程适配（不改科学口径）】
    - 路径、线程数改为命令行参数
    - 新增：测试集 250 次 bootstrap 95% 区间（固定模型重抽测试行）
    - 新增：逐折预测收据 predictions.csv / fold_split.csv（供独立审计）
    - 新增：joblib 模型存档、环境记录 env_record.txt、run_record.md
    - --smoke N：预检用，子样本 + 少量 bootstrap，产物写独立 smoke 目录，不计正式完成数

【运行示例】
    python model_d13_server.py --data data.csv --out results/full --tag full --bootstrap 250 --n-jobs 32
    python model_d13_server.py --data data.csv --out results/dedup --tag dedup --bootstrap 250 --n-jobs 32
================================================================================
"""
import argparse
import hashlib
import json
import os
import platform
import sys
import time
import warnings
from datetime import datetime

import joblib
import numpy as np
import pandas as pd

try:
    sys.stdout.reconfigure(encoding="utf-8")
except Exception:
    pass

import matplotlib
matplotlib.use("Agg")  # 无界面后端
import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split, StratifiedKFold
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import (roc_auc_score, average_precision_score,
                             brier_score_loss, roc_curve, precision_recall_curve)
```

```
from sklearn.calibration import calibration_curve
from sklearn.exceptions import ConvergenceWarning

warnings.filterwarnings("ignore", category=ConvergenceWarning)

try:
    import lightgbm as lgb
    HAS_LGB = True
except Exception as _e:
    HAS_LGB = False
    print("[WARN] LightGBM 未安装，将跳过 LGBM 模型:", _e)

# ---------------------------------------------------------------------------
# 冻结科学常量（与原型一致，禁止改动）
# ---------------------------------------------------------------------------
SEED = 42
TEST_SIZE = 0.20
N_SPLITS = 5
RUN_XGBOOST = False

LR_C = 1.0
LR_MAX_ITER = 3000
LR_CLASS_WEIGHT = None

LOG1P_COLS = ["crp", "ast", "alt", "creatinine", "d_dimer", "ck", "ck_mb",
              "ldh", "hbdh", "ima", "anion_gap", "tg", "bua"]
SPARSE_POS_THRESHOLD = 500

A_DEMO = ["age", "gender", "nihss"]
A_COMORB = ["uremia", "fatty_liver", "diabetes", "hypertension",
            "coronary_disease", "atrial_fibrillation", "hyperuricemia",
            "hyperlipidaemia"]
A_BRAIN = ["frontal_lobe", "parietal_lobe", "temporal_lobe", "occipital_lobe",
           "insular_lobe", "range_lobe", "basal_ganglia", "capsula_interna",
           "brainstem", "epencephalon", "paraventricular", "centrum_semiovale",
           "thalamus", "subcortex_lobe"]
A_VESSEL = ["aca", "mca", "pca", "va", "ba", "cca_plaque", "ica_plaque",
            "eca_plaque", "ant_circle", "post_circle", "large_ves_as"]

FEATURES_A = A_DEMO + A_COMORB + A_BRAIN + A_VESSEL          # 36 个

LABS = ["plt", "wbc", "rbc", "hba1c", "crp", "tg", "ldl", "hdl", "ast", "alt",
        "bilirubin", "albumin", "urea", "creatinine", "bua", "pt", "aptt", "tt",
        "inr", "d_dimer", "fibrinogen", "ck", "ck_mb", "ldh", "hbdh", "ima",
        "na", "k", "cl", "ca", "p", "lactate", "anion_gap", "tco2"]      # 34 个

FEATURES_B = FEATURES_A + LABS                               # 70 个

POST = ["dvt", "cerebral_hernia", "hydrocephalus", "hypoproteinemia"]  # 4 个（泄漏风险）

FEATURES_C = FEATURES_B + POST                               # 74 个

FEATURE_SETS = {"A": FEATURES_A, "B": FEATURES_B, "C": FEATURES_C}

ORD_COLS = ["age", "nihss", "range_lobe"]
LAB_COLS = LABS
BIN_COLS = [c for c in FEATURES_C if c not in LABS and c not in ORD_COLS]


def sha256_file(p):
    return hashlib.sha256(open(p, "rb").read()).hexdigest()


def sparse_frequency_report(df, out_dir):
    rows = []
    for c in BIN_COLS:
        n1 = int((df[c] == 1).sum())
        if n1 >= SPARSE_POS_THRESHOLD:
            continue
        tab = pd.crosstab(df[c], df["second_epilepsy"])
        cells = {}
        for v in (0, 1):
            for y in (0, 1):
                cells[(v, y)] = int(tab.loc[v, y]) if (v in tab.index and y in tab.columns) else 0
        zero_cell = any(cells[(v, y)] == 0 for v in (0, 1) for y in (0, 1))
        rows.append({"variable": c, "pos_count": n1,
                     "n_y0_v0": cells[(0, 0)], "n_y0_v1": cells[(1, 0)],
                     "n_y1_v0": cells[(0, 1)], "n_y1_v1": cells[(1, 1)],
                     "any_zero_cell": bool(zero_cell),
```

```
                    "note": "存在 0 单元格，LR 可能完全分离，敏感性分析考虑移除" if zero_cell else ""})
    report = pd.DataFrame(rows).sort_values("pos_count")
    report.to_csv(os.path.join(out_dir, "sparse_var_frequency.csv"),
                  index=False, encoding="utf-8-sig")
    print(f"[sparse] 稀疏二值变量频数表已保存（{len(report)} 个）")
    return report


def fit_preproc_lr(X_train):
    """由训练折估计 LR 预处理参数（winsorize 阈值 / log1p 列 / 标准化均值方差）。"""
    lab_cols = [c for c in LAB_COLS if c in X_train.columns]
    ord_cols = [c for c in ORD_COLS if c in X_train.columns]
    log1p_cols = [c for c in LOG1P_COLS if c in lab_cols]
    winsor, stats = {}, {}
    for c in lab_cols:
        lo, hi = X_train[c].quantile(0.01), X_train[c].quantile(0.99)
        winsor[c] = [float(lo), float(hi)]
    for c in ord_cols + lab_cols:
        mu, sd = X_train[c].mean(), X_train[c].std(ddof=0)
        sd = sd if sd > 1e-12 else 1.0
        stats[c] = [float(mu), float(sd)]
    return {"lab_cols": lab_cols, "ord_cols": ord_cols, "log1p_cols": log1p_cols,
            "winsor": winsor, "stats": stats}


def apply_preproc_lr(X, params):
    """用已估计参数变换（训练折或验证/测试折同一函数）。"""
    Xp = X.copy()
    for c in params["lab_cols"]:
        lo, hi = params["winsor"][c]
        Xp[c] = Xp[c].clip(lo, hi)
        if c in params["log1p_cols"]:
            Xp[c] = np.log1p(Xp[c])
    for c in params["ord_cols"] + params["lab_cols"]:
        mu, sd = params["stats"][c]
        Xp[c] = (Xp[c] - mu) / sd
    return Xp


def preprocess_lr(X_train, X_test):
    """兼容原型接口：折内 winsorize+log1p+标准化。"""
    params = fit_preproc_lr(X_train)
    return apply_preproc_lr(X_train, params), apply_preproc_lr(X_test, params), params


def make_lr():
    return LogisticRegression(C=LR_C, max_iter=LR_MAX_ITER,
                              class_weight=LR_CLASS_WEIGHT)


def _scale_pos_weight(y):
    n_pos = int((y == 1).sum())
    n_neg = int((y == 0).sum())
    return (n_neg / n_pos) if n_pos > 0 else 1.0


def make_lgbm(n_estimators, n_jobs, y_train=None):
    kw = dict(n_estimators=n_estimators, learning_rate=0.05, num_leaves=31,
              min_child_samples=50, subsample=0.8, colsample_bytree=0.8,
              reg_alpha=0.1, reg_lambda=0.1, random_state=SEED, n_jobs=n_jobs,
              verbosity=-1)
    if y_train is not None:
        kw["scale_pos_weight"] = _scale_pos_weight(y_train)
    return lgb.LGBMClassifier(**kw)


def eval_metrics(y_true, y_prob):
    return {"auroc": roc_auc_score(y_true, y_prob),
            "auprc": average_precision_score(y_true, y_prob),
            "brier": brier_score_loss(y_true, y_prob)}


def _cv_fold_summary(fold_metrics):
    out = {}
    for k in ("auroc", "auprc", "brier"):
        vals = [m[k] for m in fold_metrics]
        out[f"{k}_mean"] = float(np.mean(vals))
        out[f"{k}_sd"] = float(np.std(vals))
    return out
```

```
def run_cv(X, y, model_kind, cv, n_jobs, fold_records, fs_name):
    """训练集内 5 折 CV：返回 OOF 概率、每折指标、提示；预测写入 fold_records。"""
    oof = np.zeros(len(y))
    fold_metrics = []
    notes = []
    for tr_idx, va_idx in cv.split(X, y):
        Xtr, Xva = X.iloc[tr_idx], X.iloc[va_idx]
        ytr, yva = y.iloc[tr_idx], y.iloc[va_idx]
        fold_no = len(fold_metrics)  # 当前折序号 0..4

        if model_kind == "lr":
            Xtr_p, Xva_p, _ = preprocess_lr(Xtr, Xva)
            model = make_lr()
            model.fit(Xtr_p, ytr)
            if getattr(model, "n_iter_", [0])[0] >= LR_MAX_ITER:
                notes.append("LR 未收敛(达到 max_iter)")
            if np.abs(model.coef_).max() > 50:
                notes.append("LR 系数>50(疑似完全分离)")
        elif model_kind == "lgbm":
            Xtr_p, Xva_p = Xtr, Xva
            model = make_lgbm(1000, n_jobs, ytr)
            model.fit(Xtr_p, ytr, eval_set=[(Xva_p, yva)],
                      callbacks=[lgb.early_stopping(50, verbose=False)])
        else:
            raise ValueError(f"未知模型类型: {model_kind}")

        proba = model.predict_proba(Xva_p)[:, 1]
        oof[va_idx] = proba
        fold_metrics.append(eval_metrics(yva, proba))
        for i, idx in enumerate(va_idx):
            fold_records.append({"record_id": int(X.index[idx]), "y_true": int(y.iloc[idx]),
                                 "role": "oof", "fold": fold_no,
                                 "feature_set": fs_name, "model": model_kind,
                                 "proba": float(proba[i])})
    return oof, fold_metrics, notes


def fit_eval_test(X_train, y_train, X_test, y_test, model_kind, n_jobs):
    """测试模型：在完整训练集上拟合，评估测试集。返回概率、指标、可存档对象。"""
    if model_kind == "lr":
        Xtr_p, Xte_p, params = preprocess_lr(X_train, X_test)
        model = make_lr()
        model.fit(Xtr_p, y_train)
        y_prob = model.predict_proba(Xte_p)[:, 1]
        artifact = {"kind": "lr", "model": model, "preproc": params,
                    "features": list(X_train.columns)}
    elif model_kind == "lgbm":
        model = make_lgbm(300, n_jobs, y_train)
        model.fit(X_train, y_train)
        y_prob = model.predict_proba(X_test)[:, 1]
        artifact = {"kind": "lgbm", "model": model, "features": list(X_train.columns)}
    else:
        raise ValueError(model_kind)
    return y_prob, eval_metrics(y_test, y_prob), artifact


def bootstrap_ci(y_true, y_prob, n_boot, rng):
    """测试集固定模型 bootstrap：重抽测试行，250 次，报 2.5/97.5 百分位。"""
    if n_boot <= 0:
        return {}
    y_true = np.asarray(y_true)  # Series 带原索引，按标签取数会 KeyError，先转 numpy
    y_prob = np.asarray(y_prob)
    n = len(y_true)
    out = {}
    for k in ("auroc", "auprc", "brier"):
        vals = []
        for _ in range(n_boot):
            idx = rng.integers(0, n, n)
            vals.append(eval_metrics(y_true[idx], y_prob[idx])[k])
        lo, hi = np.quantile(vals, [0.025, 0.975])
        out[f"test_{k}_lo"] = float(lo)
        out[f"test_{k}_hi"] = float(hi)
    return out


def net_benefit(y_true, y_prob, thresholds):
    n = len(y_true)
```

```
    nb = []
    for t in thresholds:
        pred = (y_prob >= t).astype(int)
        tp = int(((pred == 1) & (y_true == 1)).sum())
        fp = int(((pred == 1) & (y_true == 0)).sum())
        nb.append(tp / n - fp / n * (t / (1 - t)))
    return np.array(nb)


def make_figs(test_preds, y_test, tag, out_dir):
    fs_list = ["A", "B", "C"]
    model_kinds = sorted({k for (k, _) in test_preds.keys()})
    colors = {"lr": "tab:blue", "lgbm": "tab:red", "xgb": "tab:green"}
    linestyles = {"A": "-", "B": "--", "C": ":"}

    fig, axes = plt.subplots(1, 2, figsize=(12, 5))
    for mk in model_kinds:
        for fs in fs_list:
            if (mk, fs) not in test_preds:
                continue
            p = test_preds[(mk, fs)]
            fpr, tpr, _ = roc_curve(y_test, p)
            prec, rec, _ = precision_recall_curve(y_test, p)
            auroc = roc_auc_score(y_test, p)
            auprc = average_precision_score(y_test, p)
            lab = f"{mk.upper()}-{fs}"
            axes[0].plot(fpr, tpr, color=colors[mk], ls=linestyles[fs],
                         label=f"{lab} (AUC={auroc:.3f})")
            axes[1].plot(rec, prec, color=colors[mk], ls=linestyles[fs],
                         label=f"{lab} (AP={auprc:.3f})")
    axes[0].plot([0, 1], [0, 1], "k--", lw=0.8)
    axes[0].set_xlabel("1-Specificity"); axes[0].set_ylabel("Sensitivity")
    axes[0].set_title("ROC (test)"); axes[0].legend(fontsize=7)
    axes[1].set_xlabel("Recall"); axes[1].set_ylabel("Precision")
    axes[1].set_title("Precision-Recall (test)"); axes[1].legend(fontsize=7)
    fig.suptitle(f"D13 second_epilepsy — {tag}")
    fig.tight_layout()
    fig.savefig(os.path.join(out_dir, f"fig_roc_pr_{tag}.png"), dpi=150)
    plt.close(fig)

    fig, axes = plt.subplots(1, 3, figsize=(15, 4.5), sharey=True)
    for j, fs in enumerate(fs_list):
        ax = axes[j]
        ax.plot([0, 1], [0, 1], "k--", lw=0.8)
        for mk in model_kinds:
            if (mk, fs) not in test_preds:
                continue
            frac, mean_pred = calibration_curve(y_test, test_preds[(mk, fs)],
                                                n_bins=10, strategy="uniform")
            ax.plot(mean_pred, frac, marker="o", color=colors[mk],
                    label=f"{mk.upper()} Brier={brier_score_loss(y_test, test_preds[(mk, fs)]):.4f}")
        ax.set_title(f"Calibration — Feature set {fs}")
        ax.set_xlabel("Mean predicted probability"); ax.legend(fontsize=7)
    axes[0].set_ylabel("Fraction of positives")
    fig.suptitle(f"D13 calibration (test) — {tag}")
    fig.tight_layout()
    fig.savefig(os.path.join(out_dir, f"fig_calibration_{tag}.png"), dpi=150)
    plt.close(fig)

    thr = np.linspace(0.01, 0.50, 100)
    prev = float(y_test.mean())
    treat_all = prev - (1 - prev) * thr / (1 - thr)
    fig, axes = plt.subplots(1, 3, figsize=(15, 4.5), sharey=True)
    for j, fs in enumerate(fs_list):
        ax = axes[j]
        ax.plot(thr, treat_all, "k--", lw=0.8, label="Treat all")
        ax.plot(thr, np.zeros_like(thr), "k-", lw=0.8, label="Treat none")
        for mk in model_kinds:
            if (mk, fs) not in test_preds:
                continue
            nb = net_benefit(y_test, test_preds[(mk, fs)], thr)
            ax.plot(thr, nb, color=colors[mk], label=f"{mk.upper()}")
        ax.set_title(f"Decision curve — Feature set {fs}")
        ax.set_xlabel("Threshold probability"); ax.legend(fontsize=7)
    axes[0].set_ylabel("Net benefit")
    fig.suptitle(f"D13 decision curve (test) — {tag}")
    fig.tight_layout()
    fig.savefig(os.path.join(out_dir, f"fig_decision_curve_{tag}.png"), dpi=150)
    plt.close(fig)
```

```
def run_one(df, tag, out_dir, n_jobs, n_boot, fold_split, pred_records, models_dir):
    """对某一版数据（full / dedup）执行完整三场景建模，返回测试集预测字典。"""
    X = df[FEATURES_C].copy()
    y = df["second_epilepsy"].copy()

    X_train, X_test, y_train, y_test = train_test_split(
        X, y, test_size=TEST_SIZE, random_state=SEED, stratify=y)
    cv = StratifiedKFold(n_splits=N_SPLITS, shuffle=True, random_state=SEED)

    print(f"\n===== [{tag}] N={len(df)} 训练={len(X_train)} 测试={len(X_test)} "
          f"训练阳性={int(y_train.sum())} 测试阳性={int(y_test.sum())} =====")

    # 划分收据（供审计核对折成员与测试集身份）
    for i in X_train.index:
        fold_split.append({"record_id": int(i), "y_true": int(y.loc[i]), "role": "train", "fold": -1})
    for i in X_test.index:
        fold_split.append({"record_id": int(i), "y_true": int(y.loc[i]), "role": "test", "fold": -1})
    # 折成员（用字典 O(1) 回填，避免 2 万行 × 5 折的 O(n²) 扫描）
    rec_by_id = {rr["record_id"]: rr for rr in fold_split}
    for fold_no, (tr_idx, va_idx) in enumerate(cv.split(X_train, y_train)):
        for i in va_idx:
            rec_by_id[int(X_train.index[i])]["fold"] = fold_no

    model_kinds = ["lr", "lgbm"] if HAS_LGB else ["lr"]

    test_preds = {}
    results = []
    rng = np.random.default_rng(SEED)

    for fs_name, fs_cols in FEATURE_SETS.items():
        for mk in model_kinds:
            Xf, Xf_test = X_train[fs_cols], X_test[fs_cols]
            # —— 训练集内 5 折 CV ——
            oof, fold_metrics, notes = run_cv(Xf, y_train, mk, cv, n_jobs, pred_records, fs_name)
            cv_sum = _cv_fold_summary(fold_metrics)
            oof_pooled = eval_metrics(y_train, oof)
            # —— 测试集评估 ——
            y_prob, test_m, artifact = fit_eval_test(Xf, y_train, Xf_test, y_test, mk, n_jobs)
            test_preds[(mk, fs_name)] = y_prob
            joblib.dump(artifact, os.path.join(models_dir, f"{tag}_{fs_name}_{mk}.joblib"))

            row = {"data": tag, "feature_set": fs_name, "model": mk}
            row.update(cv_sum)
            row["auroc_oof_pooled"] = oof_pooled["auroc"]
            row["auprc_oof_pooled"] = oof_pooled["auprc"]
            row["brier_oof_pooled"] = oof_pooled["brier"]
            row["test_auroc"] = test_m["auroc"]
            row["test_auprc"] = test_m["auprc"]
            row["test_brier"] = test_m["brier"]
            row.update(bootstrap_ci(y_test, y_prob, n_boot, rng))
            row["notes"] = "; ".join(sorted(set(notes)))
            results.append(row)
            print(f"  [{tag}] {fs_name}/{mk}: test_AUROC={test_m['auroc']:.4f} "
                  f"test_AUPRC={test_m['auprc']:.4f} test_Brier={test_m['brier']:.4f}")

            # 测试集预测收据
            for i in range(len(Xf_test)):
                pred_records.append({"record_id": int(Xf_test.index[i]),
                                     "y_true": int(y_test.iloc[i]),
                                     "role": "test", "fold": -1,
                                     "feature_set": fs_name, "model": mk,
                                     "proba": float(y_prob[i])})

    pd.DataFrame(results).to_csv(os.path.join(out_dir, f"results_metrics_{tag}.csv"),
                                 index=False, encoding="utf-8-sig")
    return test_preds, y_test


def main():
    ap = argparse.ArgumentParser()
    ap.add_argument("--data", required=True)
    ap.add_argument("--out", required=True)
    ap.add_argument("--tag", required=True, choices=["full", "dedup"])
    ap.add_argument("--bootstrap", type=int, default=250)
    ap.add_argument("--n-jobs", type=int, default=-1)
    ap.add_argument("--smoke", type=int, default=0, help="预检：子样本行数（>0 时产物写入 OUT/smoke）")
    a = ap.parse_args()
```

```
t0 = time.time()
smoke = a.smoke > 0
out_dir = os.path.join(a.out, "smoke") if smoke else a.out
os.makedirs(out_dir, exist_ok=True)
models_dir = os.path.join(out_dir, "models")
os.makedirs(models_dir, exist_ok=True)
n_boot = 5 if smoke else a.bootstrap

print("=" * 70)
print(f"D13 三场景建模（服务器版） tag={a.tag} smoke={a.smoke} bootstrap={n_boot} n_jobs={a.n_jobs}")
print(f"  LightGBM: {'已安装' if HAS_LGB else '未安装(将跳过)'}  数据: {a.data}")
print("=" * 70)

df = pd.read_csv(a.data)
if smoke:
    df = df.sample(a.smoke, random_state=SEED).reset_index(drop=True)
    print(f"[smoke] 子样本 {len(df)} 行（仅预检，不计正式完成数）")
elif a.tag == "dedup":
    before = len(df)
    df = df.drop_duplicates().reset_index(drop=True)
    print(f"[dedup] 去重后 {len(df)} 行（原 {before} 行）")

print(f"数据：{df.shape[0]} 行 × {df.shape[1]} 列，阳性 {int(df['second_epilepsy'].sum())} 例 "
      f"({100.0 * df['second_epilepsy'].mean():.2f}%)")
data_sha = sha256_file(a.data)

pred_records = []
fold_split = []

if not smoke and a.tag == "full":
    sparse_frequency_report(df, out_dir)

test_preds, y_test = run_one(df, a.tag, out_dir, a.n_jobs, n_boot,
                             fold_split, pred_records, models_dir)
make_figs(test_preds, y_test, a.tag, out_dir)

# ── 收据 ──
pd.DataFrame(pred_records).to_csv(os.path.join(out_dir, "predictions.csv"),
                                  index=False, encoding="utf-8-sig")
pd.DataFrame(fold_split).to_csv(os.path.join(out_dir, "fold_split.csv"),
                                index=False, encoding="utf-8-sig")

# ── 可复现凭据 ──
split_info = {
    "tag": a.tag, "smoke_rows": a.smoke or None,
    "data_path": os.path.abspath(a.data), "data_sha256": data_sha,
    "seed": SEED, "test_size": TEST_SIZE, "n_splits": N_SPLITS,
    "n_rows": int(len(df)),
    "n_train": None,  # 下方从 fold_split 重算
    "positive_rate": float(df["second_epilepsy"].mean()),
    "models": ["lr"] + (["lgbm"] if HAS_LGB else []),
    "lr_config": {"C": LR_C, "max_iter": LR_MAX_ITER, "class_weight": str(LR_CLASS_WEIGHT)},
    "lgbm_cv": {"n_estimators": 1000, "learning_rate": 0.05, "num_leaves": 31,
                "min_child_samples": 50, "subsample": 0.8, "colsample_bytree": 0.8,
                "reg_alpha": 0.1, "reg_lambda": 0.1, "early_stopping": 50,
                "scale_pos_weight": "per-fold n_neg/n_pos"},
    "lgbm_test": {"n_estimators": 300, "learning_rate": 0.05, "num_leaves": 31,
                  "min_child_samples": 50, "subsample": 0.8, "colsample_bytree": 0.8,
                  "reg_alpha": 0.1, "reg_lambda": 0.1, "scale_pos_weight": "train n_neg/n_pos"},
    "log1p_cols": LOG1P_COLS,
    "bootstrap": {"n": n_boot, "units": "test rows", "model": "fixed (no refit)"},
    "python": sys.version.split()[0],
    "platform": platform.platform(),
    "lightgbm": lgb.__version__ if HAS_LGB else None,
    "numpy": np.__version__, "pandas": pd.__version__,
    "sklearn": __import__("sklearn").__version__,
}
fs = pd.DataFrame(fold_split)
split_info["n_train"] = int((fs.role == "train").sum())
split_info["n_test"] = int((fs.role == "test").sum())
split_info["fold_sizes"] = fs[fs.role == "train"].groupby("fold").size().to_dict()
split_info["fold_positive"] = fs[fs.role == "train"].groupby("fold")["y_true"].sum().to_dict()
split_info["test_positive"] = int(fs[fs.role == "test"].y_true.sum())
with open(os.path.join(out_dir, "split_info.json"), "w", encoding="utf-8") as f:
    json.dump(split_info, f, ensure_ascii=False, indent=2)

# ── 环境记录 ──
try:
```

```
        import subprocess
        pip_freeze = subprocess.run([sys.executable, "-m", "pip", "freeze"],
                                    capture_output=True, text=True, timeout=120).stdout
    except Exception:
        pip_freeze = "(pip freeze 不可用)"
    with open(os.path.join(out_dir, "env_record.txt"), "w", encoding="utf-8") as f:
        f.write(f"python: {sys.version}\nplatform: {platform.platform()}\n\n"
                f"lightgbm: {lgb.__version__ if HAS_LGB else 'missing'}\n"
                f"numpy: {np.__version__}\npandas: {pd.__version__}\n"
                f"sklearn: {__import__('sklearn').__version__}\n"
                f"joblib: {joblib.__version__}\nmatplotlib: {matplotlib.__version__}\n\n"
                f"--- pip freeze ---\n{pip_freeze}")

    # ── 运行记录 ──
    elapsed = time.time() - t0
    with open(os.path.join(out_dir, "run_record.md"), "w", encoding="utf-8") as f:
        f.write(f"# D13 运行记录 {datetime.now().isoformat(timespec='seconds')}\n\n"
                f"- tag: {a.tag}（{'smoke 预检 ' + str(a.smoke) + ' 行' if smoke else '正式运行'}）\n"
                f"- 命令: python model_d13_server.py --data {a.data} --out {a.out} "
                f"--tag {a.tag} --bootstrap {n_boot} --n-jobs {a.n_jobs}"
                f"{' --smoke ' + str(a.smoke) if smoke else ''}\n"
                f"- 数据 SHA256: {data_sha}\n"
                f"- 耗时: {elapsed:.1f} 秒\n"
                f"- 状态: 完成\n"
                f"- 产物: results_metrics_{a.tag}.csv, predictions.csv, fold_split.csv, "
                f"split_info.json, env_record.txt, models/{a.tag}_*.joblib, fig_*_{a.tag}.png"
                f"{', sparse_var_frequency.csv' if (not smoke and a.tag == 'full') else ''}\n"
                f"- 注意: LightGBM 多线程下极小数值波动可能跨次运行出现（科学结论不变）; "
                f"bootstrap 区间为固定模型重抽测试行口径。\n")
    print(f"\n[完成] tag={a.tag} 耗时 {elapsed:.1f}s 输出目录: {out_dir}")


if __name__ == "__main__":
    main()
```

A.8 audit_d13.py（279 行）

用途

D13 前身项目的审计脚本：核对数据 SHA-256、划分信息与结果文件的一致性。五套算法架构沿用其“双端审计”的做法。

- **对应算法架构：**前身项目
- **输入：**~/D13_20261107/results/
- **输出：**审计报告（标准输出）

运行命令

```
venv/bin/python audit_d13.py
```

口径提醒

- **注意：**属于前身任务，只作溯源归档。

代码全文

```
# -*- coding: utf-8 -*-
r"""
================================================================================
D13 三场景建模 —— 独立审计脚本（审计方视角，不复用建模代码）
================================================================================
原则：
    - 不 import 建模脚本；只读收据（CSV/JSON），从原始预测重算全部指标再对照。
    - 对照口径：结果文件 results_metrics_*.csv、split_info.json、predictions.csv、
      fold_split.csv 之间的自洽性 + 指标重算一致性 + 数据 sha256。
    - 敏感性汇总：合并 full / dedup 两版指标 → sensitivity_full_vs_dedup.csv。
    - 任何一项 FAIL 则退出码 1；报告写入 audit_report.md。

用法：
    python audit_d13.py --root <results 目录> [--data data.csv] [--out 报告目录]
    （--root 指包含 full/ 与 dedup/ 两个结果子目录的目录）
================================================================================
"""
import argparse
import hashlib
import json
import os
import sys

import numpy as np
import pandas as pd
from sklearn.metrics import roc_auc_score, average_precision_score, brier_score_loss

TOL = 1e-6

METRIC_FN = {"auroc": roc_auc_score, "auprc": average_precision_score,
             "brier": brier_score_loss}
FEATURE_SETS = ["A", "B", "C"]
REQUIRED_METRIC_COLS = [
    "data", "feature_set", "model",
    "auroc_mean", "auroc_sd", "auprc_mean", "auprc_sd", "brier_mean", "brier_sd",
    "auroc_oof_pooled", "auprc_oof_pooled", "brier_oof_pooled",
    "test_auroc", "test_auprc", "test_brier",
    "test_auroc_lo", "test_auroc_hi", "test_auprc_lo", "test_auprc_hi",
    "test_brier_lo", "test_brier_hi", "notes",
]


def sha256_file(p):
    h = hashlib.sha256()
    with open(p, "rb") as f:
        for chunk in iter(lambda: f.read(1 << 20), b""):
            h.update(chunk)
    return h.hexdigest()


def near(a, b, tol=TOL):
    return abs(float(a) - float(b)) <= tol


class Auditor:
    def __init__(self):
        self.fails = []
```

```
    self.lines = ["# D13 三场景建模 · 独立审计报告\n"]

def check(self, ok, name, detail=""):
    tag = "PASS" if ok else "FAIL"
    self.lines.append(f"- [{tag}] {name}" + (f" — {detail}" if detail else ""))
    if not ok:
        self.fails.append(name)
    return ok

# ---------------------------------------------------------------------
def audit_tag(self, root, tag, data_path):
    d = os.path.join(root, tag)
    self.lines.append(f"\n## 数据版: {tag}")
    if not os.path.isdir(d):
        self.check(False, f"结果目录 {d} 存在")
        return
    expected = [f"results_metrics_{tag}.csv", "predictions.csv", "fold_split.csv",
                "split_info.json", "env_record.txt", "run_record.md",
                f"fig_roc_pr_{tag}.png", f"fig_calibration_{tag}.png",
                f"fig_decision_curve_{tag}.png"]
    if tag == "full":
        expected.append("sparse_var_frequency.csv")
    for fn in expected:
        self.check(os.path.isfile(os.path.join(d, fn)), f"文件存在: {tag}/{fn}")

    need = ["results_metrics_" + tag + ".csv", "predictions.csv",
            "fold_split.csv", "split_info.json"]
    if any(not os.path.isfile(os.path.join(d, fn)) for fn in need):
        self.check(False, f"{tag} 关键收据齐全（缺少则跳过该版其余检查）")
        return
    self.check(True, f"{tag} 关键收据齐全")

    si = json.load(open(os.path.join(d, "split_info.json"), encoding="utf-8"))
    metrics = pd.read_csv(os.path.join(d, f"results_metrics_{tag}.csv"))
    preds = pd.read_csv(os.path.join(d, "predictions.csv"))
    splits = pd.read_csv(os.path.join(d, "fold_split.csv"))
    models = si.get("models", ["lr", "lgbm"])

    # ── 0. 数据 sha256 ──
    if data_path and os.path.isfile(data_path):
        actual = sha256_file(data_path)
        self.check(actual == si.get("data_sha256"),
                   f"data.csv sha256 与 split_info 一致",
                   f"split_info={si.get('data_sha256')} 实际={actual}")
    else:
        self.check(False, "--data 已提供且存在（未提供则无法核对 sha256）")

    # ── 1. 划分身份 ──
    s_tr = splits[splits.role == "train"]
    s_te = splits[splits.role == "test"]
    self.check(len(s_tr) == si["n_train"] and len(s_te) == si["n_test"],
               "训练/测试行数与 split_info 一致",
               f"train={len(s_tr)}/{si['n_train']} test={len(s_te)}/{si['n_test']}")
    self.check(len(set(s_tr.record_id) & set(s_te.record_id)) == 0,
               "训练集与测试集无重叠记录")
    self.check(set(s_tr.fold.unique()) == {0, 1, 2, 3, 4},
               "训练折编号为 0..4")
    self.check((s_te.fold == -1).all(), "测试行 fold 均为 -1")
    fold_sizes_actual = s_tr.groupby("fold").size().to_dict()
    fold_sizes_info = {int(k): v for k, v in si.get("fold_sizes", {}).items()}
    self.check(fold_sizes_actual == fold_sizes_info,
               "各折规模与 split_info 一致",
               f"{fold_sizes_actual} vs {fold_sizes_info}")
    fold_pos_actual = s_tr.groupby("fold")["y_true"].sum().to_dict()
    fold_pos_info = {int(k): v for k, v in si.get("fold_positive", {}).items()}
    self.check(fold_pos_actual == fold_pos_info, "各折阳性数与 split_info 一致")
    self.check(int(s_te.y_true.sum()) == si.get("test_positive"),
               "测试集阳性数与 split_info 一致")
    self.check(len(splits) == len(s_tr) + len(s_te), "fold_split 无多余行")

    # ── 2. 预测收据覆盖 ──
    oof = preds[preds.role == "oof"]
    tes = preds[preds.role == "test"]
    n_train, n_test, n_models = si["n_train"], si["n_test"], len(models)
    self.check(len(oof) == n_train * n_models * 3,
               "OOF 收据总数 = n_train × 模型数 × 3 特征集",
               f"{len(oof)} vs {n_train * n_models * 3}")
    self.check(len(tes) == n_test * n_models * 3,
               "测试收据总数 = n_test × 模型数 × 3 特征集",
```

```
            f"{len(tes)} vs {n_test * n_models * 3}")
self.check(oof.feature_set.isin(FEATURE_SETS).all() and
           tes.feature_set.isin(FEATURE_SETS).all(),
           "收据 feature_set 取值 ∈ {A,B,C}")
# y_true 与 fold_split 一致
ymap = dict(zip(splits.record_id, splits.y_true))
self.check((oof.record_id.map(ymap) == oof.y_true).all() and
           (tes.record_id.map(ymap) == tes.y_true).all(),
           "收据 y_true 与 fold_split 一致")
# 每个 (fs, mk) 覆盖
for fs in FEATURE_SETS:
    for mk in models:
        o = oof[(oof.feature_set == fs) & (oof.model == mk)]
        t = tes[(tes.feature_set == fs) & (tes.model == mk)]
        self.check(len(o) == n_train and len(t) == n_test,
                   f"{fs}/{mk} 收据行数正确",
                   f"oof={len(o)}/{n_train} test={len(t)}/{n_test}")
        if len(o) == n_train:
            self.check(o.record_id.nunique() == n_train,
                       f"{fs}/{mk} OOF 无重复记录")

# ── 3. 指标重算对照 ──
self.check(set(REQUIRED_METRIC_COLS).issubset(metrics.columns),
           "results_metrics 列完整")
rng = np.random.default_rng(si.get("seed", 42))
n_boot = si.get("bootstrap", {}).get("n", 250)
for fs in FEATURE_SETS:
    for mk in models:
        row = metrics[(metrics.feature_set == fs) & (metrics.model == mk)]
        self.check(len(row) == 1, f"{fs}/{mk} 指标表恰有一行")
        if len(row) != 1:
            continue
        row = row.iloc[0]
        o = oof[(oof.feature_set == fs) & (oof.model == mk)]
        t = tes[(tes.feature_set == fs) & (tes.model == mk)]
        # 折内指标 → 均值±SD
        per_fold = {}
        for f_ in range(5):
            fo = o[o.fold == f_]
            per_fold[f_] = {k: METRIC_FN[k](fo.y_true, fo.proba)
                            for k in METRIC_FN}
        for k in METRIC_FN:
            vals = [per_fold[f_][k] for f_ in range(5)]
            self.check(near(np.mean(vals), row[f"{k}_mean"]) and
                       near(np.std(vals), row[f"{k}_sd"]),
                       f"{fs}/{mk} CV {k} 均值±SD 重算一致",
                       f"重算 {np.mean(vals):.6f}±{np.std(vals):.6f} vs "
                       f"记录 {row[f'{k}_mean']:.6f}±{row[f'{k}_sd']:.6f}")
        # OOF 拼接（按 record_id 排序后整体算）
        o_sorted = o.sort_values("record_id")
        for k in METRIC_FN:
            pooled = METRIC_FN[k](o_sorted.y_true, o_sorted.proba)
            self.check(near(pooled, row[f"{k}_oof_pooled"]),
                       f"{fs}/{mk} OOF 拼接 {k} 重算一致",
                       f"重算 {pooled:.6f} vs 记录 {row[f'{k}_oof_pooled']:.6f}")
        # 测试集（按 record_id 排序；单值指标与行序无关）
        t_sorted = t.sort_values("record_id")
        for k in METRIC_FN:
            v = METRIC_FN[k](t_sorted.y_true, t_sorted.proba)
            self.check(near(v, row[f"test_{k}"]),
                       f"{fs}/{mk} 测试集 {k} 重算一致",
                       f"重算 {v:.6f} vs 记录 {row[f'test_{k}']:.6f}")
        # bootstrap 重放（固定模型重抽测试行；位置重抽样依赖行序，
        # 必须用 predictions.csv 原始行序 = 建模脚本 X_test 行序，不能排序）
        y_t, p_t = t.y_true.to_numpy(), t.proba.to_numpy()
        n = len(y_t)
        for k in METRIC_FN:
            vals = []
            for _ in range(n_boot):
                idx = rng.integers(0, n, n)
                vals.append(METRIC_FN[k](y_t[idx], p_t[idx]))
            lo, hi = np.quantile(vals, [0.025, 0.975])
            self.check(near(lo, row[f"test_{k}_lo"]) and
                       near(hi, row[f"test_{k}_hi"]),
                       f"{fs}/{mk} bootstrap {k} 95% 区间重放一致",
                       f"重放 [{lo:.6f}, {hi:.6f}] vs "
                       f"记录 [{row[f'test_{k}_lo']:.6f}, {row[f'test_{k}_hi']:.6f}]")

# ── 4. 环境记录 ──
```

```
        env_txt = open(os.path.join(d, "env_record.txt"), encoding="utf-8").read()
        self.check("python:" in env_txt and "pip freeze" in env_txt,
                   "env_record.txt 含 python 版本与 pip freeze")
        self.check(str(si.get("lightgbm")) in env_txt or "lightgbm" in env_txt,
                   "env_record.txt 含 lightgbm 版本")

    # ------------------------------------------------------------------
    def run(self, root, data_path, out):
        self.audit_tag(root, "full", data_path)
        self.audit_tag(root, "dedup", data_path)

        # ── 敏感性汇总 ──
        sens_rows = []
        f = os.path.join(root, "full", "results_metrics_full.csv")
        d = os.path.join(root, "dedup", "results_metrics_dedup.csv")
        if os.path.isfile(f) and os.path.isfile(d):
            mf = pd.read_csv(f).set_index(["feature_set", "model"])
            md = pd.read_csv(d).set_index(["feature_set", "model"])
            for k in ["test_auroc", "test_auprc", "test_brier",
                      "auroc_oof_pooled", "auprc_oof_pooled", "brier_oof_pooled",
                      "auroc_mean", "auprc_mean", "brier_mean"]:
                for (fs_, mk_) in mf.index:
                    sens_rows.append({"feature_set": fs_, "model": mk_,
                                      "metric": k, "full": mf.loc[(fs_, mk_), k],
                                      "dedup": md.loc[(fs_, mk_), k],
                                      "delta_dedup_minus_full":
                                          md.loc[(fs_, mk_), k] - mf.loc[(fs_, mk_), k]})
            sens = pd.DataFrame(sens_rows)
            sens.to_csv(os.path.join(out, "sensitivity_full_vs_dedup.csv"),
                        index=False, encoding="utf-8-sig")
            self.check(True, "sensitivity_full_vs_dedup.csv 已合并生成",
                       f"{len(sens)} 行")
        else:
            self.check(False, "两版 results_metrics 齐全，可生成敏感性汇总")

        # ── 报告 ──
        self.lines.append(f"\n## 结论\n- 总检查项：{sum(1 for l in self.lines if l.startswith('- ['))}，"
                          f"FAIL {len(self.fails)} 项。")
        if self.fails:
            self.lines.append("- FAIL 项：" + "；".join(self.fails))
        else:
            self.lines.append("- 全部 PASS。")
        os.makedirs(out, exist_ok=True)
        with open(os.path.join(out, "audit_report.md"), "w", encoding="utf-8") as fh:
            fh.write("\n".join(self.lines) + "\n")
        print(f"审计完成：FAIL {len(self.fails)} 项，报告 -> {os.path.join(out, 'audit_report.md')}")
        for l in self.lines:
            if l.startswith("- [FAIL]"):
                print(l)
        return 1 if self.fails else 0


def main():
    ap = argparse.ArgumentParser()
    ap.add_argument("--root", required=True, help="含 full/ 与 dedup/ 的结果目录")
    ap.add_argument("--data", default="", help="data.csv 路径（核对 sha256）")
    ap.add_argument("--out", default="", help="报告输出目录（默认=--root）")
    a = ap.parse_args()
    out = a.out or a.root
    sys.exit(Auditor().run(a.root, a.data, out))


if __name__ == "__main__":
    main()
```

A.9 slurm_d13.sh（27 行）

用途

D13 前身项目的 SLURM 作业提交模板（full / dedup 两个参数化任务）。归档用于展示当时的调度方式。

- **对应算法架构：**前身项目
- **输入：**—
- **输出：**slurm_*.out / slurm_*.err

运行命令

```
sbatch -J d13_full slurm_d13.sh full
```

口径提醒

- **注意：**属于前身任务，只作溯源归档。

代码全文

```
#!/bin/bash
# D13 三场景建模 Slurm 作业脚本
# 用法: sbatch -J d13_full  slurm_d13.sh full
#       sbatch -J d13_dedup slurm_d13.sh dedup
#SBATCH --partition=CPU
#SBATCH --nodes=1
#SBATCH --ntasks=1
#SBATCH --cpus-per-task=32
#SBATCH --mem=32G
#SBATCH --time=03:00:00
#SBATCH --output=slurm_%x_%j.out
#SBATCH --error=slurm_%x_%j.err

set -euo pipefail
TAG=${1:?用法: slurm_d13.sh full|dedup}
ROOT=$HOME/D13_20261107

echo "=== D13 job start $(date -Is) tag=$TAG host=$(hostname) ==="
source "$ROOT/venv/bin/activate"
cd "$ROOT"
python model_d13_server.py \
  --data "$ROOT/data.csv" \
  --out "$ROOT/results/$TAG" \
  --tag "$TAG" \
  --bootstrap 250 \
  --n-jobs 32
echo "=== D13 job done $(date -Is) exit=$? ==="
```